



D3 User Guide Version 10.3.4

Notices

Copyright

© 1996-2026 Rocket Software, Inc. or its affiliates. All Rights Reserved.

Trademarks

Rocket is a registered trademark of Rocket Software, Inc. For a list of Rocket registered trademarks go to: www.rocketsoftware.com/about/legal. All other products or services mentioned in this document may be covered by the trademarks, service marks, or product names of their respective owners.

Examples

This information might contain examples of data and reports. The examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to the names and addresses used by an actual business enterprise is entirely coincidental.

License agreement

This software and the associated documentation are proprietary and confidential to Rocket Software, Inc. or its affiliates, are furnished under license, and may be used and copied only in accordance with the terms of such license.

Note: This product may contain encryption technology. Many countries prohibit or restrict the use, import, or export of encryption technologies, and current use, import, and export regulations should be followed when exporting this product.

Corporate information

Rocket Software, Inc. develops enterprise infrastructure products in four key areas: storage, networks, and compliance; database servers and tools; business information and analytics; and application development, integration, and modernization.

Website: www.rocketsoftware.com

Rocket Global Headquarters

77 4th Avenue, Suite 100

Waltham, MA 02451-1468

USA

To contact Rocket Software by telephone for any reason, including obtaining pre-sales information and technical support, use one of the following telephone numbers.

Country and Toll-free telephone number

- United States: 1-855-577-4323
- Australia: 1-800-823-405
- Belgium: 0800-266-65
- Canada: 1-855-577-4323
- China: 400-120-9242
- France: 08-05-08-05-62
- Germany: 0800-180-0882
- Italy: 800-878-295
- Japan: 0800-170-5464
- Netherlands: 0-800-022-2961
- New Zealand: 0800-003210
- South Africa: 0-800-980-818
- United Kingdom: 0800-520-0439

Contacting Technical Support

The Rocket Community is the primary method of obtaining support. If you have current support and maintenance agreements with Rocket Software, you can access the Rocket Community and report a problem, download an update, or read answers to FAQs. To log in to the Rocket Community or to request a Rocket Community account, go to www.rocketsoftware.com/support. In addition to using the Rocket Community to obtain support, you can use one of the telephone numbers that are listed above or send an email to support@rocketsoftware.com.

Table of contents

D3 User Guide overview	9
Introduction to D3.	9
Logging on and off D3	11
Accessing files and items	13
File Control Block	13
Data input and output specifications.	15
File-defining item or D-pointer	16
Master dictionary.	16
Account dictionary	17
Data file inventory	18
Sharing dictionaries.	18
Accessing items in a file	19
Hashing	20
B-trees	21
Indexes	22
Internal structure of an item.	22
Item header	22
Item-ID.	23
Item body	23
Item types	24
D3 dictionary structure	26
D3 file system	26
System dictionary.	26
Master dictionary.	27
Data file dictionary.	28
Dictionary item types	29
File-defining items.	29
System dictionary.	29
Master dictionary.	34
File dictionary	40
Attribute-defining items.	45
File dictionary	45
Default values.	49
Suppress attribute item	51
Synonym item (Q-pointers)	52
Command item.	52
Proc item.	53
Macro item	54
Menu item.	55
Connectives.	56
File definition item	57
Definition item	58

Connective, macro, and menu items	59
Update processor prestore, proc, and Q-pointer items	60
Super Q-pointer and cataloged FlashBASIC items	62
Item definition summary	63
Controlling and dependent structures	64
Processing codes	66
Input conversions, correlatives, and output conversions.	66
Input format	66
Internal format	67
Intermediate format	67
Output format	67
Processing code summary.	68
Designing a database	70
Details	70
Database rules	71
Attribute name considerations	72
Item-ID considerations	73
Creating a database in D3	76
Creating the data file	76
Sample database design and creation.	78
Deciding on attributes and definitions	78
Creating the file	80
Creating attribute-defining items	81
Editing the file-defining item	85
Creating indexes	86
Connecting to the database	87
Virtual machine environment in Windows	89
Accessing the VME	89
Starting D3 VME	89
Stopping D3 VME	90
File system access	91
Frame size	92
Restoring accounts	92
Saving accounts	93
Serial terminals	94
Telnet emulation	94
VME tapes	95
VME printers	96
Form queue	98
Print file	98
VME supported OSFI drivers	99
File System Interface	101
FSI on D3 Windows	102
Configuring the FSI server	102
Starting the FSI server	103

Stopping the FSI server	104
Logging on to the FSI	105
Users file	106
Using the FSI tools	107
Using automatic file resizing	107
Resizing control files	108
Setting inactivity timeouts on a remote server connection	110
D3 Windows utilities	112
D3 File Manager	112
File Manager layout	112
Editing File Manager options	115
Domains	116
Opening a domain	116
Selecting the MDS server name	117
Databases	118
Creating a database	120
Deleting a database	121
Renaming a database	122
Creating and editing synonyms	123
Tables	124
Creating a table	125
Listing the contents of a table	127
Copying a table	128
Counting a table	130
Deleting a table	131
Creating a new data level for a table	132
Clearing a table	134
Renaming a table	134
Moving a table	135
Table correlatives	136
Records	136
Copying records	137
Creating a record	138
Importing records	139
Editing a record	139
Deleting records	142
Editing a D-pointer	142
Additional File Manger operations	144
Listing or clearing record locks	144
Editing General table properties	146
Viewing Usage table properties	147
Displaying Statistics table properties	148
Listing servers	150
Displaying Flash shared code	150
Creating indexes	153

Checking overflow	155
Running a hash test	156
Resizing files	158
Environment variables	161
Path environment variables	162
Server environment variables.....	163
Performance monitoring tools	164
Windows performance monitoring tools	165
D3 performance monitoring tools	166
System structure	169
Frames	169
Linking frames	170
I/O devices.....	170
Monitor	170
Memory management	171
Frame requests and frame faults	172
Concept of a process.....	172
Process identification blocks	173
User pibs and phantom pibs	174
Scheduler	175
Scheduling algorithm	175
Super high priority process	176
Major parts of the system	177
Boot record.....	177
ABS.....	177
Process control blocks and workspaces.....	180
Proc	182
Structure of the PROC processor	183
Proc I/O buffers	183
Parameters	184
Proc command format.....	185
Proc command summary	185
Editor	187
Entering the Editor	188
Editor command summary	188
Editor error messages	190
Update Processor	191
Update Processor as text editor.....	191
Cursor movement	192
Overtyping and insert modes	194
Inputting text using the spelling checker.....	194
Deleting text	195
Searching and replacing text	195
Cutting and pasting text	196
Prestore commands.....	197

Exiting items.	198
Update Processor as a data processor	199
Overloaded commands for data files	201
Update processor command summary.	202

D3 User Guide overview

The D3 User Guide focuses on the concepts you need to understand the D3 data model and provides information on how a database is designed, updated, and used.

The purpose of this publication is to:

- Familiarize the reader with the basic data structures used
- Present the structure and use of dictionaries
- Describe the text editors available
- Show a session on creating a database of different files in D3

Introduction to D3

D3 is a powerful, efficient, and compact multidimensional DBMS. The entire system includes several modules that make D3 the most powerful and flexible DBMS available, for example:

- Specialized data storage system
- Data retrieval language
- Several command processors
- Line editor
- Full-screen editor
- Spooler
- FlashBASIC compiler
- ODBC and SQL connections

The fundamental concepts essential to understanding how the D3 system works are:

- Smallest identifiable piece of data is called a value. Values are stored in a structure called an attribute (field).
- Attributes can contain no value (null), one value, or several values.
- Group of zero or more attributes make up an item (record). Each item has the same number of attributes, and each attribute is stored in the same order within the item as all the other items.

-
- Items are stored in a file.
 - Definitions of the item attributes and how they are handled is stored in a dictionary. Every file has a corresponding dictionary.

The unique dictionary file contains at least one definition for each attribute in the corresponding file. It describes what kind of data the attribute holds and how the data is sorted, displayed, and interrelates with other attributes. The dictionary has features that allow a:

- value to be converted to a short form during data entry, then reconverted to the value for display.
- value to be referred to if that value is stored in another attribute, another item, or another file.
- value to be logically linked to another value in a different attribute.
- FlashBASIC subroutine to be executed in conjunction with recalling the value.

There must be at least one definition for each of the item attributes. There can be more than one definition for the same attribute. This allows an attribute value to be handled in more than one way, depending on the needs of the moment.

- New attributes can be added to an existing file by merely creating a new attribute definition in the dictionary for a new attribute.
- Manual data entry can be performed by writing a FlashBASIC program, by using the Update Processor and specifying the list of attributes to enter, or by using the Editor. Automatic data entry is performed by writing a FlashBASIC program or a Proc.
- Data is retrieved from a file by either writing a FlashBASIC program, or by using AQL, the report generating language.
- Additional system operations, such as device management or file maintenance, is handled through the Terminal Control Language (TCL).
- AQL sentences can be incorporated into simpler command structures, such as a macro or menu. AQL commands may also be performed from within a FlashBASIC program or Proc routines.
- AQL reports can be sent to the terminal for display or to the Spooler for printing. The Spooler manages all of the printer resources on the D3 system.
- The ODBC and OSFI connections allow non-D3 applications to access the D3 database.

This document describes:

- What a database is
- D3 data model, file structure, and system architecture
- Introduction on navigating the D3 system
- Internal structure of D3 data
- How D3 accesses data in a file
- D3 dictionaries and their subsidiary codes and structures
- Line editor and full-screen editor
- How to design and build a sample database

Other books describe the AQL and TCL languages; the FlashBASIC compiler; the Spooler, ODBC, and SQL capabilities; and other information on D3.

 **Note:** D3 uses a programming language called FlashBASIC, which is an extension of the original Dartmouth BASIC. Unlike its predecessor, Pick/BASIC, in which the source code was tokenized and interpreted at run time, FlashBASIC translates the source code into C and employs the host C compiler to produce object code which improves performance by as much as 500%. Earlier versions of Pick/BASIC source code or compiled object code can be compiled to FlashBASIC without any necessary code modification.

Related tasks

[Logging on and off D3](#)

Logging on and off D3

When D3 has started and the terminals are ready to be logged on as D3 terminals, pressing any key at the terminal produces the following display:

```
Enter your user id:
```

Logging on is a two step process.

1. Enter a D3 user-ID.

The system administrator should assign a unique user-ID to each person who uses the computer.

2. Enter the master dictionary or account wanted.

This account prompt can be bypassed by defining an initial master dictionary (md) in each user-ID in the user's file. The D3 system has a user-ID and account name defined:

dm - user-ID

and the master dictionary entry:

dm - account name

```
D3 Database Management System - Enter your D3 user id: dm
master dictionary: dm
3
<<< 09:50:04 -- D -- 13 Nov 2009
```

3. Logon is complete.

The system is either at the TCL prompt (:) or at a menu provided by the application developer.



Note:

- If the logon procedure fails because the maximum allowed users limit has been reached, an error is logged to the errors file.
- To log off from D3, exit any D3 application running from the TCL prompt. A similar message displays.

```
Connect time = 60 Mins.; CPU=4240 Units; LPTR pages= 0
logged off at 09:00:00 on 14 Mar 2003
```

Note that this logs off a single terminal; it does not shut down the D3 MDBMS. To shut down the virtual machine, see the *D3 System Administration Guide*.

Related concepts

[Introduction to D3](#)

Accessing files and items

Files can contain any number of items and automatically expand to any size. A file should be created by determining how many items are stored in it, and the average size of one item. When the file is created, the entire space is allocated out of contiguous disk frames, even though the file is initially empty. This is the minimum amount of disk space the file will ever take up. It can grow but it cannot shrink below this size.

The file is accessed by looking in the account master dictionary and obtaining the base address, or base Frame Identifier (FID). The size of the file in frames is called the *modulo*. The D3 system uses the account md to keep track of where files are and how much disk space the files use. It uses a *file dictionary* to keep track of how the data is stored and displayed for each item in the file.



The space allocated remains empty until items are added to the file.

File Control Block

At the end of each file's data section is a FCB that contains specific information about the data file. The FCB starts immediately after the last frame in the data section of the file. These frames contain information about the file, including index pointers and their compiled algebraic-processing codes.

Important flags are also contained in the FCB, as are fields indicating that the file is open, closed or in the process of being deleted, and so on. This protects the system from deleting a file while another process is using that file.

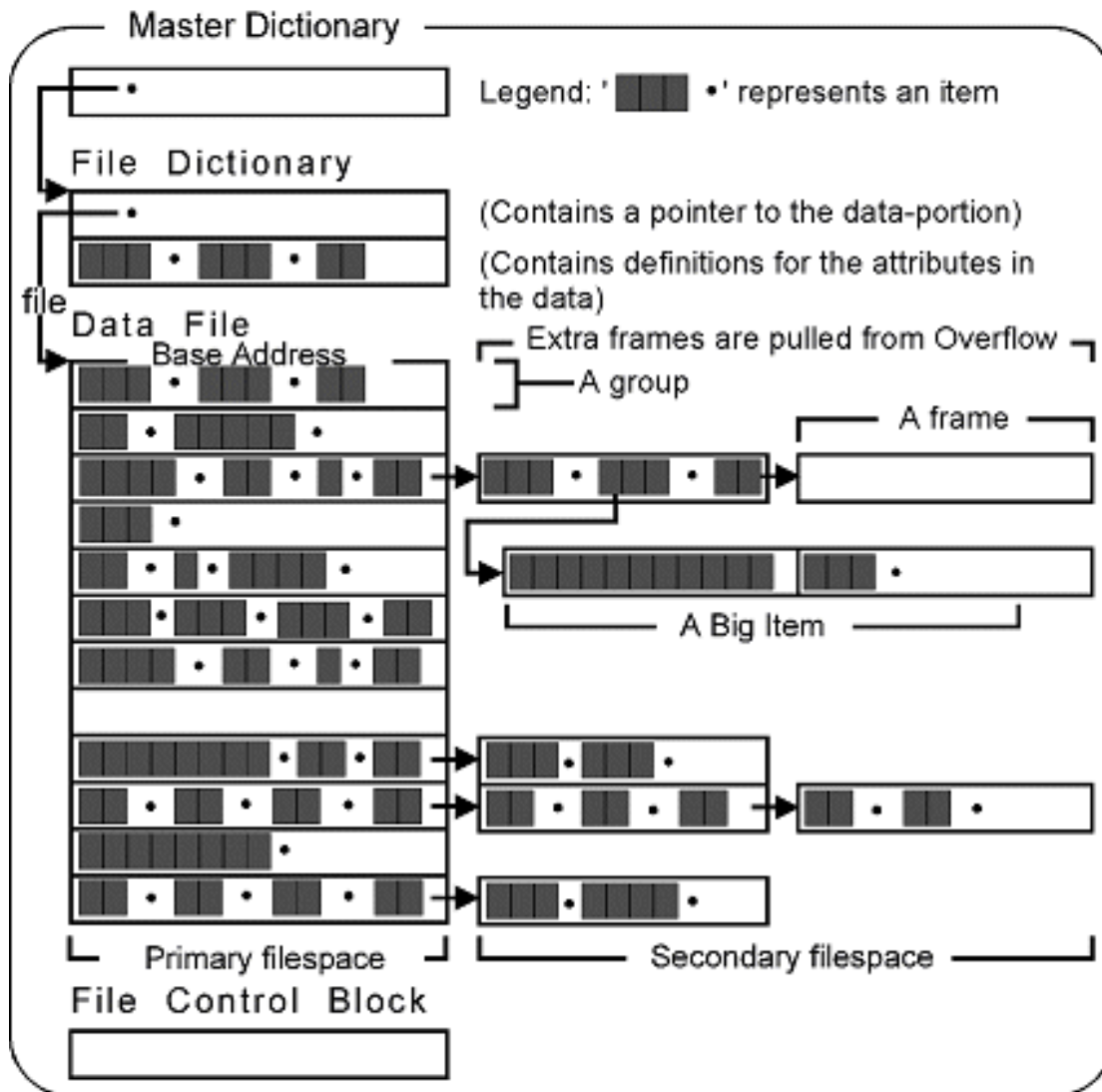
The FCB is used for internal housekeeping and is not meant to be edited.



Files and items

Items are added to a file using an algorithm to calculate a location within the file, called hashing, where the item is to be inserted. These spots are called groups. A group can hold more than one item depending on the size of the group and the size of the item.

As the file fills up, several items can come to rest in the same group and the space in that group is completely used up. At this point, additional frames are retrieved from the overflow table and appended to that group.



The initial allocation of disk space is called primary file space, and items overflow into secondary file space. The speed of a system is reduced if the system has to traverse a lot of secondary file space to find an item. Files can be resized

to make this process more efficient.

Transporting a file to another system with a different frame size can affect the system's performance.

Related concepts

[Data input and output specifications](#)

[File-defining item or D-pointer](#)

[Internal structure of an item](#)

Data input and output specifications

To conserve keystrokes and disk file space, D3 has the capability to modify the value and attribute before saving the item. There also is a method to read the data and reformat it before it displays. These data input and output processes are defined by processing codes, also called conversions and correlatives.

For example:

Input data as:	102003
D3 stores it as:	13077
Displayed as:	20-Oct-2003

Lists

D3 uses the list to make it possible to view a portion of a file when the entire file content is not needed.

Select Lists

To retrieve a subset of a file, the required data items are specified with the **select** command.

```
: select invoices with inv.total < "100.00"  
> sort invoice by customer.name customer.name inv.no inv.date inv.total
```

When a list is selected, the system builds a list of item-IDs in the order specified in the **select** statement. After the list has been selected, a TCL command can be issued to operate on that list, or the list can be saved with the **save-list** command.

Saved Lists

Select lists can be temporary or can be saved for later reference using the **save-list** command. The list contains only references to items that are in the database at the time the selection is executed. The items could easily change between the time the list is saved and the time the list is used. Lists can be saved in any file. The default is the pointer file. Lists are records containing one item-ID per selected record, each in an attribute.

The **get-list** command is used to retrieve a copy of a saved list. Use the **edit-list** command to modify items in the list and the **delete-list** command to delete a list.

Active Lists

Once a list has been selected or retrieved using the **get-list** command, the list is called an active list. The TCL prompt character changes from : to > when a list is active. After a TCL command is executed, the prompt reverts

back to the : prompt.

Related concepts

[File Control Block](#)

[File-defining item or D-pointer](#)

[Internal structure of an item](#)

File-defining item or D-pointer

There is an entry in the master dictionary that is the file-defining item for the file. It is also called a D-pointer because D3 uses a D dictionary code within the master dictionary item to specify that this item points to a data file. The D-pointer contains the name of the file, its address on disk, how large the file is, and other relevant information pertinent to the entire file.

Related concepts

[File Control Block](#)

[Data input and output specifications](#)

[Internal structure of an item](#)

Master dictionary

The base (base address) shows the disk address where the dictionary section of this file begins, and the modulo frames long. In the example below, the file starts at address 12194 and is 11 frames long.

```
{verbs}
{file defining items}
{etc.}

'...inventory^d^12194^11^L^10^_'
```

UD MD inventory(,inventory)

```
dictionary-code    D
base               12194
modulo             11
structure
retrieval-lock
update-lock
output-conversion
correlative
attribute-type     L
column-width       10
input-conversion
macro
output-macro
description
reallocation
hotkey.all
:
```

Related concepts

[Account dictionary](#)

[Data file inventory](#)

[Sharing dictionaries](#)

[Accessing items in a file](#)

[Hashing](#)

[B-trees](#)

[Indexes](#)

Account dictionary

The dictionary section of a data file contains the file-defining items, attribute-defining items, and other definition items.

- File-defining item *inventory* base is the disk address where the data section of this file begins. Its modulo is the number of contiguous frames reserved for this file. In the above example, the data section of the inventory file starts at address 12210 and uses 337 frames.
- Attribute-defining item part number describes the qualities of just one attribute in the data portion of this file. In this example, the part number item defines how the attribute one value is output. The report heading is Part Number, left-aligned, and the system allows 15 spaces for the value to display.

File Dictionary (dictionary section) dict inventory

```
12194
'....inventory^d^12210^337^^^^L^10^p
art.number desc qty units price so#
po# stocking. min stocking.max
this.yr^This is the Inventory file^_
(other file-defining items)
(verbs)
(etc.)

(attribute defining items)
'....part.number^A^1^Part
Number^^^^L^15^_
'....desc^A^2^...^_
'....qty^A^3^...^_
'....units^A^4^...^_
'....price^A^5^...^_
'....so#^A^6^...^_
'....po#^A^7^...^_
'....stocking.min^A^8^...^_
'....stocking.max^A^9^...^_
'....this.yr^A^10^...^_
'....last.yr^A^11^...^_
12204
```

File-defining Item (or 'd-pointer') UD inventory{,inventory}

```
dictionary-code D
base 12210
modulo 337
structure
retrieval-lock
update-lock
output-conversion
correlative
attribute-type L
column-width 10
input-conversion
macro part.number desc
output-macro
description This is the Inve
reallocation
hotkey.all
```

Attribute-defining Item UD inventory part.number

```
Dictionary-code A
attribute count 1
substitute-header Part
Number
structure
output conversion
correlative
attribute type L
column width 15
input-conversion
macro
output-conversion
description
hotkey.all
```

Related concepts

[Master dictionary](#)

[Data file inventory](#)

- [Sharing dictionaries](#)
- [Accessing items in a file](#)
- [Hashing](#)
- [B-trees](#)
- [Indexes](#)

Data file inventory

The structure of the item and the attributes within is determined by knowing that each separate piece of information within an item is an attribute and that an attribute must be defined by an attribute-defining item.

12210

(data)

'....022-47-6391^flange assy^4^ea^1250^409603^609211^1^6^0^0^_'

12546

Data Item

U inventory '022-47-6391'

part.number	022-47-6391
desc	flange assy
qty	4
unit	ea
price	12.50
S.O.#	409603
P.O.#	609211
stocking.min	1
stocking.max	6
this.yr	
last.yr	

Creating a series of attribute-defining items to specify a data section of the file is where the data is actually stored. The data can be internally organized in one of several ways, but the process is transparent to the user. The item is returned to the user as a series of attribute values.

The information in the dictionary section associates the attribute number to its attribute name. There must be at least one attribute-defining item for each attribute.

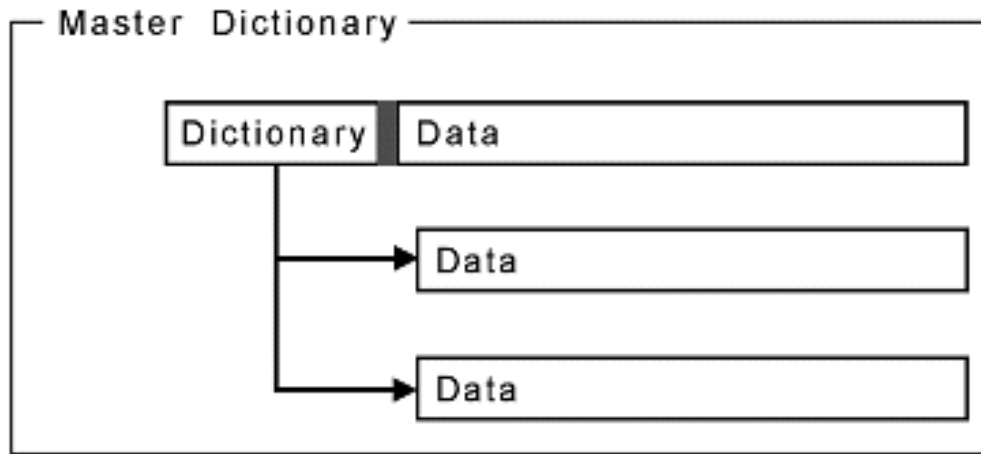
Related concepts

- [Master dictionary](#)
- [Account dictionary](#)
- [Sharing dictionaries](#)
- [Accessing items in a file](#)
- [Hashing](#)
- [B-trees](#)
- [Indexes](#)

Sharing dictionaries

A dictionary file can be shared by multiple data files.

Sharing Dictionaries



Referring to these files requires specifying the dictionary name as part of the filename. For example, a data file is called `sales`, which refers to all of the sales orders; however, a different method is to use two files, where the `sales` file (or the `sales,sales` file) refers to all sales orders for the current year, and the `sales,history` file refers to the orders from all the previous years. A major advantage is a reduction in data retrieval time for all of the more common functions dealing with current year-to-date data, since all of the history data is in a separate file. Since both files share the same structure, both files can share the same dictionary.

Related concepts

[Master dictionary](#)

[Account dictionary](#)

[Data file inventory](#)

[Accessing items in a file](#)

[Hashing](#)

[B-trees](#)

[Indexes](#)

Accessing items in a file

In summary, accounts contain files. Files use dictionaries to describe how the data is arranged. Files arrange data in items. Items are made up of attributes.

Items that are stored in a file can be accessed:

- directly using the item-ID to access an item.
- sequentially in a pseudo random hashing sequence.

- using any dictionary-defined field in sorted sequence.
- using indexes that are currently available from within FlashBASIC, AQL, and Update Processor.

The direct file access technique, using the item-ID to locate the item within the file, is an efficient method of locating data and lends itself to the online nature of the D3 system. The system overhead required to access an item using direct file access is essentially constant, independent of the actual file size.

Related concepts

[Master dictionary](#)

[Account dictionary](#)

[Data file inventory](#)

[Sharing dictionaries](#)

[Hashing](#)

[B-trees](#)

[Indexes](#)

Hashing

Hashing is the internal function that performs a mathematical computation, which yields an address indicating where the record resides.

D3 defines the concept of groups as dividing the linear space for data into equally sized blocks. The number of groups in a file should always be a prime number to facilitate efficient hashing. An item hashes to a specific group. To add the item, it is appended to the series of items already residing within that group. When the group (frame) becomes full, additional blocks of space are linked to that frame as needed. D3 Hashing Algorithm is very efficient:

```
x = 0
for j=1 to len( item-ID )
x = ( x*10 ) + seq( item-ID[ j,1 ] )
next j
group = rem( x, modulo )
frame.id = group + base.fid
```

where

- **item-ID** is the key to accessing the item
- **group** is the relative frame where the item hashes
- **modulo** is the number of groups (frames) in the data file primary file space
- **frame.id** is the final address where the item hashes
- **base.fid** is the starting frame address of the data file

D3 converts ASCII into a numeric value, divides it by the number of available groups, and takes the remainder. That

remainder is the relative group number to which the item resides (hashes).

Adding an item

The system hashes to the group and scans the group for an empty spot to write the item. If it finds the item already there, it reports a duplicate item. If it finds an empty spot, it puts the item in. If it reaches the end of the group, or if there is not enough remaining space in the group to add this item, the item overflows the capacity of the group. D3 must make the group bigger. It grabs a spare frame and links it to the group using a linked-list method. Then it adds the item using the new group space. Items are always added at the end of the group, or at the end of the chain of overflow frames. Each group, therefore, is a potential linked-list chain of frames.

Deleting an item

The system hashes to the group and scans through the group and successive overflow frames until it finds the item. If the item does not exist, the system reports the item not found. If found, the item is deleted and all following items in the group are shifted to eliminate the gap. If an overflow frame becomes empty from repositioning, it is unlinked and returned to overflow.

D3 actually does not delete items right away. Items are initially marked as deleted, then *actually* deleted during the file backup process when the system is sure an undelete is not wanted.

Related concepts

[Master dictionary](#)

[Account dictionary](#)

[Data file inventory](#)

[Sharing dictionaries](#)

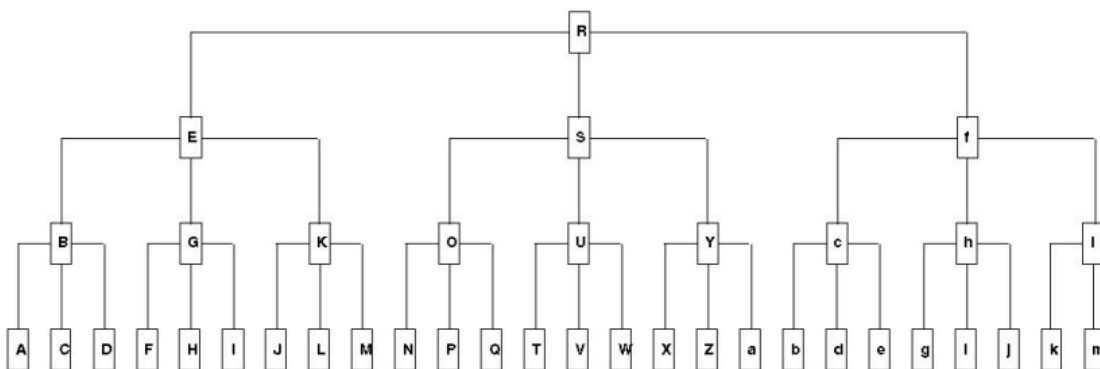
[Accessing items in a file](#)

[B-trees](#)

[Indexes](#)

B-trees

The scheme maintains a smaller, separate file that contains pointers and is maintained in a sorted order. D3 index files are kept in a linear sorted order and maintained as a balanced tree, or B-tree. B-trees are sorted linked lists, so search time is kept to a minimum.



Related concepts

[Master dictionary](#)

[Account dictionary](#)

[Data file inventory](#)

[Sharing dictionaries](#)
[Accessing items in a file](#)
[Hashing](#)
[Indexes](#)

Indexes

There can be as many indexes as needed. Indexes are maintained by the system as the database evolves. Items can be indexed by any field (real or calculated) in the item. The create-index command is used to create an index. When the file is changed, the indexes also change. For example, if an item is deleted, all index entries that point to that item are also deleted.

FlashBASIC programs can reference the indexes by employing the root statements. Indexes also are used by the AQL commands **select**, **sselect**, and **sort** if there is an index corresponding to the sort or select keys.

Related concepts

[Master dictionary](#)
[Account dictionary](#)
[Data file inventory](#)
[Sharing dictionaries](#)
[Accessing items in a file](#)
[Hashing](#)
[B-trees](#)

Internal structure of an item

The D3 system achieves a high level of performance by using storage space efficiently. Data is economically stored using only the necessary data and delimiter characters to separate fields and records.

An item (record) in D3 is made up an item-ID and zero or more attributes (fields), which make up the data stored for this item. Each atomic piece of data or value is separated by a delimiter to separate it from other values. The entire item also has a special delimiter to separate it from other items.

The full structure of an item as it is stored within the D3 system contains the following:

Related concepts

[File Control Block](#)
[Data input and output specifications](#)
[File-defining item or D-pointer](#)

Item header

Each item or record requires some number of bytes for system overhead. This is called the item header.

The item header consists of eight bytes:

- 4 bytes (unused)
- 2 bytes for the item size pointer points to the (trailing) segment mark, not the beginning of the next item

- 1 byte for item flags

0								
1	item deleted since last full save	if D-pointer, file stolen from another account	if D-pointer, file cleared since last full save	if D-pointer, file renamed since last full save, else new item-locked	item was updated since last full save	a D-pointer	a binary item	a pointer item
	7	6	5	4	3	2	1	0

- 1 byte for the number of file-saves performed

At the time a full save is completed, the system clears bits 3, 4, 5, and 7 from the item flags in all items. The message **clearing dirty bits**, seen during the **file-save** process, indicates this procedure is taking place. It is desirable not to perform this cleanup until after the full save is known to be valid. This byte is used to identify whether the item was updated before or after the full save.

Related concepts

[Item-ID](#)
[Item body](#)
[Item types](#)

Item-ID

The item-ID (item identifier) is used to access an item. It must be unique to the file with no duplicate item-IDs. Retrieval time can be minimized by using the item-ID to locate specific items.

Item-IDs are created when the item is created. It may be created automatically by the system, automatically generated by specifications in the file's dictionary, or it may be specified at data entry.

The item-ID is a unique string of characters that identifies this item from all the others in this file. The item-ID can be as short as a single character (byte) or as long as 101 characters, but only the first 24 characters are used for uniqueness. Within the item, the item-ID is referred to as attribute zero. The item-ID is terminated with one or two attribute marks, so that the following item begins on an even byte address.

Related concepts

[Item header](#)
[Item body](#)
[Item types](#)

Item body

Following the item header, the item body contains zero or more attributes. Usually, trailing null-value attributes do not need an attribute mark. Attribute marks are necessary for null values only if they precede a non-null valued attribute to ensure proper placement within the item.

Feature	Description
Attributes can contain	• Null value • One value • Many values, each separated by a value mark
Values can contain	• No data or null value • One piece of data • Several pieces of data, each separated by a subvalue mark

What kind of information the data represents is defined by another item (one that resides in the file's dictionary). The item body contains all of the data associated with this item in attributes. The item can contain zero or more attributes.

Feature	Description
Attribute	An attribute is made up of zero or more values followed by an attribute mark.
Value	A value can contain zero or more data, each piece is separated by a value mark. What kind of information the values represent is defined elsewhere. There are separate data structures used to define each attribute and the values therein. These structures are called attribute-defining items and reside in the file's dictionary.
Segment Mark	Items are terminated with a special delimiter called a segment mark. To improve data retrieval speed, items are required to begin on a word boundary (even byte address), which means that every item's length must be an even number of bytes. A second padding segment mark makes up the difference when necessary.

Related concepts

[Item header](#)

[Item-ID](#)

[Item types](#)

Item types

Externally, there are only items. Internally, items are classified into three different types for maximum efficiency. The type is specified in the item header.

Each type has its own internal structure:

- Ordinary item

The item is stored within the group. The item body must be less than 1720 bytes for a 2K-frame system.

Header	item-ID	<am>{<am>}	item body	<sm>{<sm>}
--------	---------	------------	-----------	------------

- Pointer item

If the item body exceeds 1720 bytes, then a small pointer item is created with a link pointing to the location of the item.

Header	item-ID	<am>{<am>}	frame address	<sm>{<sm>}
--------	---------	------------	---------------	------------

Frame address:	
item body	<sm>{<sm>}

Frame address points to the beginning frame of the item body.

- Pointer item (binary item)

Binary items such as FlashBASIC object code contain hexadecimal values, which may be misinterpreted as segment marks, attribute marks or value marks. To avoid this, a pointer item is created and a frame count is added that indicates the size of the item. The binary item is then stored in a contiguous block of frames.

Header	item-ID	<am>{<am>}	frame address <vm> frame count	<sm>{<sm>}
--------	---------	------------	--------------------------------	------------

Frame address:		frame count:	
item body	...	...	...

Related concepts

[Item header](#)

[Item-ID](#)

[Item body](#)

D3 dictionary structure

The D3 file structure is made up entirely of files with two parts: the data section and the dictionary section.

Section	Description
Data section	Describes where the information resides and contains items that, in turn, can contain zero or more attributes. The raw data in each attribute has no definition as to what the data represents. In D3, the definitions are stored in the dictionary section. Each data attribute has a corresponding dictionary entry describing what that data represents and how the data is entered and displayed.
Dictionary section	Describes how the data is to be viewed or manipulated. These entries are dictionary items and refer to data attributes (fields) by their position within the item, for example, first attribute, second attribute, and so on. The meanings are fully described in a dictionary item called attribute-defining item. There may be several attribute-defining items defining the same attribute, offering different <i>views</i> of the same data.

To enter or output data, the attribute must have a dictionary item that defines it or it can use the system default attribute-defining items. The predefined attribute-defining items are a1, a2, a3, and so on. A1 references attribute 1, a2 references attribute 2, and so on.

Since the dictionary section of a file is also a series of items made up of attributes, there are corresponding definitions for them.

D3 file system

The hierarchy of the D3 file system is illustrated below.

Level	Description
Level 0	System Dictionary
Level 1	Master Dictionary
Level 2	Data File Dictionary
Level 3	Data File Data

System dictionary

The System Dictionary, **mds file**, defines where all the accounts reside. There is only one system dictionary.

The system dictionary can contain:

- File-defining items (accounts)
- Attribute-defining items
- Attribute substitute items

-
- Attribute-suppressing items
 - Synonym items (account Q-pointers)
 - Connectives
 - Macros
 - Menus
 - Update Processor prestore items

The System Dictionary is accessible *only* from the dm account.

Related concepts

[Master dictionary](#)

[Data file dictionary](#)

Master dictionary

Each account on the D3 system has its own master dictionary. This file defines the account environment.

Master dictionaries contain:

- File-defining items (data files)
- Synonym-defining items (file Q-pointers)
- Commands
- Connectives
- Procs
- Macros
- Menus
- Update Processor prestore items
- BASIC program catalog pointers

There is only one master dictionary per account.

Related concepts

[System dictionary](#)

[Data file dictionary](#)

Data file dictionary

A data file is made up of two sections: the dictionary section and the data section.

Section	Description
Dictionary	The dictionary section contains pointers to the associated data sections of the files. It also contains items that describe the structure of the data in the data files.
Data	Data file dictionaries can contain: • Attribute-defining items • Attribute-suppressing items • Synonym-defining items • FlashBASIC object code

One dictionary can be used for several data files.

Data File Data

These files contain all the actual data that are input. Data file data sections can contain items that contain data.

The data is stored in items (records). Each item can have zero or more attributes (fields). The actual data within the attribute is referred to as its value. An attribute can be null (empty) or contain one value or several values (MultiValues).

There can be an unlimited number of data files in an account.

For more information, see section *File-defining items* in *D3 Reference Manual* document.

Related concepts

[System dictionary](#)

[Master dictionary](#)

Dictionary item types

This section discusses the various dictionary item types in D3.

File-defining items

The item-ID and each attribute of the file-defining item contain information that points to and describes the lower level dictionary file or data file. Because it has a D dictionary code, the file-defining item is sometimes referred to as a D-pointer.

There are two file-defining item types:

- System Dictionary, mds

File definition items define each master dictionary (account) on the system.

- Master Dictionary, md

File definition items define data files.

In a file dictionary, file definition items define the file's data section.

Related concepts

[Attribute-defining items](#)

Related reference

[File definition item](#)

System dictionary

The table below describes the attributes for the System Dictionary.

#	Attribute	Description
0	item-ID	Name of the file being defined.
1	dictionary-code	Defines the type of dictionary item. For a system dictionary file-defining item, this <i>must</i> be a d. The create-account process automatically inserts the d. Optional additional codes are:
		1 Log any updates to the transaction logger. This option is not compatible with x or y options.
		p Primary file space contains only pointer items, regardless of the number of bytes in the item.
		s Item-IDs are case sensitive. The s code must be specified when the file is created; it cannot be added or deleted later. This option is compatible with all options.
		x Do not save on file-save tapes. The file does not exist after a file restore.

#	Attribute	Description	
			Known as a DXed file. This option is not compatible with the y option.
		y	Do not save the data in this file on file-save tapes. On a file restore, recreate the file in an empty state. Known as a DYed file. This option is not compatible with the x option.
2	base	Contains the disk address of where the file begins (base FID). The create-file process automatically inserts this address; it <i>must not</i> be altered.	
3	modulo	The number of contiguous frames in primary file space. The create-file process automatically inserts 37.	
4	structure	Not Used.	
5	retrieval-lock	Controls access to the file for retrieval (reading). At least one of the lock codes must match at least one of the user's key codes, found in the users file attribute 6 (keys), before access is granted. This attribute may be MultiValued for multiple access codes.	
6	update-lock	Controls access to the file for updating (writing). At least one of the lock codes must match at least one of the user's key codes, found in the users file attribute 6, before access is granted. This attribute may be MultiValued for multiple access codes.	
7	password	Contains the password required to grant access to (log to) this master dictionary (account). The password displays in encrypted form. There is no way to decrypt the password.	
8	correlative	The processing codes that modify the data <i>before</i> it is sorted or selected. These codes are executed regardless of whether there is a value in the attribute when the item is filed (written), regardless of whether the item is updated using FlashBASIC or the Update Processor.	
9	attribute-type	Data output formatting codes can be one of the following:	
		1	Left align/attribute 10 limits/no break—Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows. <pre> now is the time now is the time for all g ood men to come to me </pre>
		1x	Left align/expand column/word break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time </pre>

#	Attribute	Description
		now is the time for all good men to come to me
	n	Right align/ignore attribute 10/no break—(Numeric specification—obsolete) Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns, if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. now is the time now is the time for all good men to come to me
	r	Right align/ignore attribute 10/no break—Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. now is the time now is the time for all good men to come to me
	rx	Right align/expand and ignore column/no break—Data is right aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. now is the time now is the time for all good men to come to me
	t	Left align/attribute 10/word break—(Text specification) Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. now is the time now is the time for all good men to come to me
	tx	Left align/expand column/word break—(Same as 1x) Data is left aligned. Disregard the attribute 10 column width and make a new column width

#	Attribute	Description	
			based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
		u	Left align/ignore attribute 10/no break—Data is left aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. If the text is wider than the display column width, cascade data into the adjacent column to the right. <pre> now is the time now is the time for all good men to come to me </pre>
		ux	Left align/expand and ignore column/no break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
		w	Output processor is attribute or noncolumnar. Process this attribute through the Output processor before displaying. Any valid Output processor dot (.) command can follow this code. Dot commands following the w are prefixed to the attribute. The attribute itself may contain embedded Output processor dot commands. A second value here can contain dot commands to be performed <i>after</i> the attribute is output. The Output processor .char command is used to output unprintable characters and D3 segment, value, MultiValue, and subvalue marks. For example, w.c.bf centers text and makes it boldface (].xbf turns off the boldface after it outputs the attribute). This code only works for noncolumnar formats. It reverts to code 1 for columnar reports.

#	Attribute	Description
		Output processor all attributes to end of item or noncolumnar. Process all attributes from the current attribute through the end of the item through Output processor before displaying. Any valid Output processor dot command may follow this code. Dot commands following the w are prefixed to the attribute. The following attributes themselves may contain embedded Output processor dot commands. A second value here may contain dot commands to be performed <i>after</i> the attributes are output. As an example, w.c.bf centers text and makes it boldface. w.c.bf].xbf.skip2 centers the first line of text, turns on boldface, prints the text. At the end of the text, turns off boldface and prints two blank lines. This code only works for noncolumnar formats. It reverts to code 1 for columnar reports. If the justification code is unrecognizable, it defaults to the 1 code.
10	column-width	Defines the number of characters that are to be displayed on one line. If the number of characters in the substitute heading (attribute-defining item attribute 3) exceeds the column width, the column width is expanded to the heading size. If the number of characters in the heading is less than the column width, the heading is padded with periods (.) to fill the column width.
11-12	(Reserved)	
13	reallocation	This is used to specify a new modulo when resizing a file. This modulo is used during the file-restore process to change the size of the file's primary file space. To specify a new modulo, enter the modulo enclosed in parentheses. To specify that this modulo not be changed, enter a period.
14	input-conversion	Processing codes applied at data entry time. Processing codes can be used to validate data entry as well as transform data before storing it.
15	macro	This attribute can contain any number of the attribute names to be used as default data entry specifications. These attributes are listed when the Update Processor is invoked to edit or update an item in this file. Multiple attribute names are separated by spaces. This field can be MultiValued for multiple views of the data. To specify which value to use, a FlashBASIC subroutine must be called from the input-conversion attribute. In this subroutine, the function access(18) must be set to the value number needed.
16	output-macro	This contains a list of the attribute names to be used as default output specifications in an AQL report. If this attribute has no value, the system uses the attribute list in the macro attribute. These attributes are listed automatically when any TCL or AQL command that takes output specifications is invoked and no output specifications are included in the command. Attribute names in the list are separated by spaces. If an attribute name does not exist in the dictionary, it is ignored. Output specifications listed in the TCL or AQL command override attributes defined in the output-macro dictionary.
17	description	Contains a description of this attribute use. When using Update Processor, typing ? and pressing ENTER displays the contents of this field.
18	(Reserved)	
19	hotkey.all	Global hotkey subroutine call. Pressing any hotkey calls the subroutines placed in this attribute first, calls the subroutines placed in the specific hotkey attribute, and then returns. A subroutine can be a macro, proc, or cataloged FlashBASIC program.

#	Attribute	Description
20	hotkey.1	Hotkey ^X1 (CTRL+X 1) subroutine call. Pressing ^X1 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
21	hotkey.2	Hotkey ^X2 (CTRL+X 2) subroutine call. Pressing ^X2 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
22	hotkey.3	Hotkey ^X3 (CTRL+X 3) subroutine call. Pressing ^X3 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
23	hotkey.4	Hotkey ^X4 (CTRL+X 4) subroutine call. Pressing ^X4 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
24	hotkey.5	Hotkey ^X5 (CTRL+X 5) subroutine call. Pressing ^X5 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
25	hotkey.6	Hotkey ^X6 (CTRL+X 6) subroutine call. Pressing ^X6 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
26	hotkey.7	Hotkey ^X7 (CTRL+X 7) subroutine call. Pressing ^X7 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
27	hotkey.8	Hotkey ^X8 (CTRL+X 8) subroutine call. Pressing ^X8 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
28	hotkey.9	Hotkey ^X9 (CTRL+X 9) subroutine call. Pressing ^X9 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
29	hotkey.0	Hotkey ^X0 (CTRL+X 0) subroutine call. Pressing ^X0 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.

Related reference

[Master dictionary](#)

[File dictionary](#)

Master dictionary

The table below describes the attributes for the Master Dictionary.

#	Attribute	Description
0	item-ID	Name of the file being defined.
1	dictionary-code	Defines the type of dictionary item. For a master dictionary file-defining item, this <i>must</i> be a d. The create-file process automatically inserts the d. Optional additional codes are:

#	Attribute	Description	
		1	Log any updates to the transaction logger. This option is not compatible with x or y options.
		p	Primary file space contains only pointer items, regardless of the number of bytes in the item.
		s	Item-IDs are case sensitive. The s code must be specified when the file is created; it cannot be added or deleted later. This option is compatible with all options.
		x	Do not save on file-save tapes. The file does not exist after a file restore. Known as a DXed file. This option is not compatible with the y option.
		y	Do not save the data in this file on file-save tapes. On a file restore, recreate the file in an empty state. Known as a DYed file. This option is not compatible with the x option.
2	base	Contains the disk address of where the file begins (base FID). The create-file process automatically inserts this address; it must not be altered.	
3	modulo	The number of contiguous frames in primary file space. The create-file process automatically inserts 37. This field should not be altered. To resize a file, see attribute 13 (reallocation) below.	
4	structure	Not Used.	
5	retrieval-lock	Controls access to the file for retrieval (reading). At least one of the lock codes must match at least one of the users,user-ID file, attribute 6 (keys), before access is granted. This attribute may be MultiValued for multiple access codes.	
6	update-lock	Controls access to the file for updating (writing). At least one of the lock codes must match at least one of the user's key codes, found in the user-ID file attribute 6 (keys), before access is granted. This attribute may be MultiValued for multiple access codes.	
7	password	Contains the password required to grant access to (log to) this master dictionary (account). The password displays in encrypted form. There is no way to decrypt the password.	
8	correlative	The processing codes that modify the data <i>before</i> it is sorted or selected. These codes are executed regardless of whether there is a value in the attribute when the item is filed (written), regardless of whether the item is updated using FlashBASIC or Update Processor.	
9	attribute-type	Data output formatting codes can be one of the following:	
		1	Left align/attribute 10 limits/no break—Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows. <pre> now is the time now is the time for all g ood men to come to me </pre>

#	Attribute	Description
		1x Left align/expand column/word break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
		n Right align/ignore attribute 10/no break—(Numeric specification—obsolete) Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns, if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
		r Right align/ignore attribute 10/no break—Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
		rx Right align/expand and ignore column/no break—Data is right aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
		t Left align/attribute 10/word break—(Text specification) Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character.

#	Attribute	Description
		<pre> now is the time now is the time for all good men to come to me </pre>
	tx	Left align/expand column/word break—(Same as 1x) Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
	u	Left align/ignore attribute 10/no break—Data is left aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. If the text is wider than the display column width, cascade data into the adjacent column to the right. <pre> now is the time now is the time for all good men to come to me </pre>
	ux	Left align/expand and ignore column/no break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
	w	Output processor is attribute or noncolumnar. Process this attribute through the Output processor before displaying. Any valid Output processor dot (.) command can follow this code. Dot commands following the w are prefixed to the attribute. The attribute itself may contain embedded Output processor dot commands. A second value

#	Attribute	Description
		here can contain dot commands to be performed <i>after</i> the attribute is output. The Output processor .char command is used to output unprintable characters and D3 segment, value, MultiValue, and subvalue marks. For example, w.c.bf centers text and makes it boldface (].xbf turns off the boldface after it outputs the attribute). This code only works for noncolumnar formats. It reverts to code 1 for columnar reports.
		Output processor all attributes to end of item or noncolumnar. Process all attributes from the current attribute through the end of the item through Output processor before displaying. Any valid Output processor dot command may follow this code. Dot commands following the w are prefixed to the attribute. The following attributes themselves may contain embedded Output processor dot commands. A second value here may contain dot commands to be performed <i>after</i> the attributes are output. As an example, w.c.bf centers text and makes it boldface. w.c.bf].xbf.skip2 centers the first line of text, turns on boldface, prints the text. At the end of the text, turns off boldface and prints two blank lines. This code only works for noncolumnar formats. It reverts to code 1 for columnar reports. If the justification code is unrecognizable, it defaults to the 1 code.
10	column-width	Defines the number of characters that are to be displayed on one line. If the number of characters in the substitute heading (attribute-defining item attribute 3) exceeds the column width, the column width is expanded to the heading size. If the number of characters in the heading is less than the column width, the heading is padded with periods to fill the column width. If the attribute data is wider than this, then it displays according to the justification directions (attribute 9).
11-12	(Reserved)	
13	reallocation	This is used to specify a new modulo when resizing a file. This modulo is used during the file-restore process to change the size of the file's primary file space. To specify a new modulo, enter the modulo enclosed in parentheses. To specify that this modulo not be changed, enter a period.
14	input-conversion	Processing codes applied at data entry time. Processing codes can be used to validate data entry, as well as transform data before storing it.
15	macro	This attribute can contain any number of the attribute names used as default data entry specifications. These attributes are listed when the Update Processor is invoked to edit/update an item in this file. Multiple attribute names are separated by spaces. The attribute names are listed in one value, separated by spaces. Which values of attribute 15 to use can be specified by assigning to FlashBASIC function access(18) (from 1 to max_value #) in a FlashBASIC program called before displaying the item (input conversion).
16	output-macro	This contains a list of the attribute names to be used as default output specifications in an AQL report. If this attribute has no value, the system uses

#	Attribute	Description
		the attribute list in the macro attribute. These attributes are listed automatically when any TCL or AQL command that takes output specifications is invoked and no output specifications are included in the command. Attribute names in the list are separated by spaces. If an attribute name does not exist in the dictionary, it is ignored. Output specifications listed in the macro attribute override the default attributes defined as a1, a2, a3, and so on.
17	description	Contains a description of this attribute use. When using Update Processor, typing ? and pressing ENTER displays this field as online help. It is usually the application programmer who enters information into this attribute.
18	(Reserved)	
19	hotkey.all	Global hotkey subroutine call. Pressing any hotkey calls the subroutines placed in this attribute first, calls the subroutines placed in the specific hotkey attribute, and then returns. A subroutine can be a macro, proc, or cataloged FlashBASIC program.
20	hotkey.1	Hotkey ^X1 (CTRL+X 1) subroutine call. Pressing ^X1 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
21	hotkey.2	Hotkey ^X2 (CTRL+X 2) subroutine call. Pressing ^X2 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
22	hotkey.3	Hotkey ^X3 (CTRL+X 3) subroutine call. Pressing ^X3 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
23	hotkey.4	Hotkey ^X4 (CTRL+X 4) subroutine call. Pressing ^X4 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
24	hotkey.5	Hotkey ^X5 (CTRL+X 5) subroutine call. Pressing ^X5 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
25	hotkey.6	Hotkey ^X6 (CTRL+X 6) subroutine call. Pressing ^X6 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
26	hotkey.7	Hotkey ^X7 (CTRL+X 7) subroutine call. Pressing ^X7 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
27	hotkey.8	Hotkey ^X8 (CTRL+X 8) subroutine call. Pressing ^X8 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
28	hotkey.9	Hotkey ^X9 (CTRL+X 9) subroutine call. Pressing ^X9 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
29	hotkey.0	Hotkey ^X0 (CTRL+X 0) subroutine call. Pressing ^X0 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.

Related reference

[System dictionary](#)

[File dictionary](#)

File dictionary

The table below describes the attributes for the File Dictionary.

#	Attribute	Description
0	item-ID	Name of the file being defined.  CAUTION: Using any of the reserved characters from Char(247) through Char(255) in item-IDs can produce unpredictable results.
1	dictionary-code	Defines the type of dictionary item. For a file dictionary file-defining item, this <i>must</i> be a d. This item also is called a D-pointer for this reason. The create-file process automatically inserts the d. Optional additional codes are:
		1 Log any updates to the transaction logger. This option is not compatible with x or y options.
		p Primary file space contains only pointer items, regardless of the number of bytes in the item.
		s Item-IDs are case sensitive. The s code must be specified when the file is created; it cannot be added or deleted later. This option is compatible with all options.
		x Do not save on file-save tapes. The file does not exist after a file restore. Known as a DXed file. This option is not compatible with the y option.
	y	Do not save the data in this file on file-save tapes. On a file restore, recreate the file in an empty state. Known as a DYed file. This option is not compatible with the x option.
2	base	Contains the disk address of where the file begins (base FID). The create-file process automatically inserts this address; it must not be altered.
3	modulo	The number of contiguous frames in primary file space. Once set, it must not be altered. This field should not be altered. To resize a file, see attribute 13 (reallocation) below.
4	structure	It is always 1.
5	retrieval-lock	Controls access to the file for retrieval (reading). At least one of the lock codes must match at least one of the user's key codes, found in the user-ID file attribute 6 (keys), before access is granted. This attribute may be MultiValued for multiple access codes.
6	update-lock	Controls access to the file for updating (writing). At least one of the lock codes must match at least one of the user's key codes, found in the user-ID file attribute 6 (keys), before access is granted. This attribute may be MultiValued for multiple access codes.
7	output-conversion	The processing codes that modify the data <i>after</i> it is sorted or selected prior to output. The correlative (attribute 8) is performed first, and then the data is sorted or selected. The output-conversion (attribute 7) is performed, and then the data is output. These codes are executed only if there is a non-null value in the attribute.
8	correlative	The processing codes that modify the data <i>before</i> it is sorted or selected. The correlative

#	Attribute	Description
		is performed first, then the data is sorted or selected. The output-conversion is performed, and then the data is output. These codes are executed regardless of whether there is a value in the attribute when the item is filed (written), regardless of whether the item is updated using FlashBASIC or Update Processor.
9	attribute-type	Data output formatting codes can be one of the following:
		1 Left align/attribute 10 limits/no break—Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows. <pre> now is the time now is the time for all g ood men to come to me </pre>
		1x Left align/expand column/word break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
		n Right align/ignore attribute 10/no break—(Numeric specification—obsolete) Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns, if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
		r Right align/ignore attribute 10/no break—Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
		rx Right align/expand and ignore column/no break—Data is right aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it

#	Attribute	Description
		evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
	t	Left align/attribute 10/word break—(Text specification) Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
	tx	Left align/expand column/word break—(Same as 1x) Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
	u	Left align/ignore attribute 10/no break—Data is left aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. If the text is wider than the display column width, cascade data into the adjacent column to the right. <pre> now is the time now is the time for all good men to come to me </pre>
	ux	Left align/expand and ignore column/no break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows.

#	Attribute	Description
		<pre> now is the time now is the time for all good men to come to me </pre>
		Output processor is attribute or noncolumnar. Process this attribute through the Output processor before displaying. Any valid Output processor dot (.) command can follow this code. Dot commands following the w are prefixed to the attribute. The attribute itself may contain embedded Output processor dot commands. A second value here can contain dot commands to be performed <i>after</i> the attribute is output. The Output processor .char command is used to output unprintable characters and D3 segment, value, MultiValue, and subvalue marks. For example, w.c.bf centers text and makes it boldface (].xbf turns off the boldface after it outputs the attribute). This code only works for noncolumnar formats. It reverts to code 1 for columnar reports.
		Output processor all attributes to end of item or noncolumnar. Process all attributes from the current attribute through the end of the item through Output processor before displaying. Any valid Output processor dot command may follow this code. Dot commands following the w are prefixed to the attribute. The following attributes themselves may contain embedded Output processor dot commands. A second value here may contain dot commands to be performed <i>after</i> the attributes are output. As an example, w.c.bf centers text and makes it boldface. w.c.bf].xbf.skip2 centers the first line of text, turns on boldface, prints the text. At the end of the text, turns off boldface and prints two blank lines. This code only works for noncolumnar formats. It reverts to code 1 for columnar reports. If the justification code is unrecognizable, it defaults to the 1 code.
10	column-width	Defines the number of characters that are to be displayed on one line. If the number of characters in the substitute heading (attribute-defining item attribute 3) exceeds the column width, the column width is expanded to the heading size. If the number of characters in the heading is less than the column width, the heading is padded with periods to fill the column width. If the data attribute is wider than this, then it displays according to the justification directions (attribute 9).
11-12	(Reserved)	
13	reallocation	This is used to specify a new modulo when resizing a file. This modulo is used during the file-restore process to change the size of the file's primary file space. To specify a new modulo, enter the modulo enclosed in parentheses.
14	input-conversion	Processing codes applied at data entry time. Processing codes may be used to validate data entry, as well as transform data before storing it.
15	macro	List of the attribute names to be used as default data entry specifications. These attributes are listed automatically when the Update Processor is invoked to edit/update

#	Attribute	Description
		an item in this file. The attribute names are listed in one value separated by spaces. Which values of attribute 15 to be used may be specified by assigning to FlashBASIC function access(18) (from 1 to max_value #) in a FlashBASIC program called before displaying the item (input conversion).
16	output-macro	This contains a list of the attribute names to be used as default output specifications in an AQL report. If this attribute has no value, the system uses the attribute list in the macro attribute. These attributes are listed automatically when any TCL or AQL command that takes output specifications is invoked and no output specifications are included in the command. Attribute names in the list are separated by spaces. If an attribute name does not exist in the dictionary, it is ignored.
17	description	Contains a description of this attribute use. When using Update Processor, typing ? and pressing ENTER displays this field as online help. It is usually the application programmer who enters information into this attribute.
18	(Reserved)	
19	hotkey.all	Global hotkey subroutine call. Pressing any hotkey calls the subroutines placed in this attribute first, calls the subroutines placed in the specific hotkey attribute, and then returns. A subroutine can be a macro, proc, or cataloged FlashBASIC program.
20	hotkey.1	Hotkey ^X1 (CTRL+X 1) subroutine call. Pressing ^X1 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
21	hotkey.2	Hotkey ^X2 (CTRL+X 2) subroutine call. Pressing ^X2 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
22	hotkey.3	Hotkey ^X3 (CTRL+X 3) subroutine call. Pressing ^X3 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
23	hotkey.4	Hotkey ^X4 (CTRL+X 4) subroutine call. Pressing ^X4 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
24	hotkey.5	Hotkey ^X5 (CTRL+X 5) subroutine call. Pressing ^X5 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
25	hotkey.6	Hotkey ^X6 (CTRL+X 6) subroutine call. Pressing ^X6 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
26	hotkey.7	Hotkey ^X7 (CTRL+X 7) subroutine call. Pressing ^X7 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
27	hotkey.8	Hotkey ^X8 (CTRL+X 8) subroutine call. Pressing ^X8 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
28	hotkey.9	Hotkey ^X9 (CTRL+X 9) subroutine call. Pressing ^X9 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.
29	hotkey.0	Hotkey ^X0 (CTRL+X 0) subroutine call. Pressing ^X0 calls the subroutines placed in this attribute. A subroutine may be a macro, proc, or cataloged FlashBASIC program.

Related reference

[System dictionary](#)

[Master dictionary](#)

Attribute-defining items

The item-ID of the attribute-defining item is the attribute name. The next value of an attribute-defining item is a number *n* which refers to the *n*th attribute in the item. For example, an attribute count of 5 refers to the fifth attribute in the data item.

There can be several attribute-defining items that refer to the same attribute in a data item. For example, perhaps the attribute should be sorted as a right-aligned entry, but printed left aligned. In one case the attribute should be displayed in dollars and cents, and in another case, the attribute should be displayed in thousands of dollars. In such a situation, two attribute-defining items can be defined, as `amount-$` and `amount-$1000`. There is no limit to the number of attribute-defining items that can point to an attribute.

The two most common forms of data stored are:

Data Form	Description
ASCII text	For sorting purposes, data should be left aligned. The <code>t</code> code may be used for justification instead, which is left aligned and breaks at spaces (instead of in the middle of the word) if the text is wider than the display column.
Numeric values	For sorting purposes, data should be right aligned. For dollar (<code>\$000.00</code>) values, use the <code>mr2</code> correlative.

Related concepts

[File-defining items](#)

Related reference

[File definition item](#)

File dictionary

The table below describes the attributes for the File Dictionary.

#	Attribute	Description
0	item-ID	Symbolic name used to refer to this attribute. This name is used to reference the defined attribute. This name is not necessarily what displays on the reports (see substitute-header below). Each attribute-defining item in a file must have a unique attribute name.
1	dictionary-code	Defines the type of dictionary item. For a file dictionary file-defining item, this must be an <code>a</code> , although <code>s</code> for synonym is sometimes used to distinguish between primary and alternate attribute-defining items.
2	attribute-count	The attribute-number within the item being defined (for example, attribute <code>#nn</code>). There are some default attribute numbers available for special use:
		0 - References the Item-ID.
		9998 - Returns the current item counter (<i>nn</i> th item processed).
		9999 - Returns the current item size.
3	substitute-header	The descriptive name used by the Update Processor and AQL report headings. If left null, the item-ID is used as the column heading. To create multiline headings, separate each line with a value mark (<code>]</code>). If no heading is desired, enter a backslash (<code>\</code>)

#	Attribute	Description
		as the substitute header.
4	structure	The controlling and dependent structure code. The attribute value begins with either a c, d, or it is empty. c is followed by the attribute numbers of the associated dependent attribute and d is followed by the controlling attribute number. This establishes internal relationships within a file. Each MultiValue for the controlling attribute is linked to and controls the corresponding values for a dependent attribute. The code c specifies the controller and d specifies the controlled. The usage format is: <pre>c; attrib#; attrib#;... d; attrib#</pre>
5-6	Reserved	
7	output-conversion	The processing codes that modify the data <i>after</i> it is sorted or selected prior to output. These codes are executed only if there is a non-null value in the attribute.
8	correlative	The processing codes that modify the data <i>before</i> it is sorted or selected. The correlative is performed first, then the data is sorted or selected. The output-conversion is performed, and then the data is output. These codes are executed regardless of whether there is a value in the attribute when the item is filed (written), regardless of whether the item is updated using FlashBASIC or Update Processor.
9	attribute-type	Data output formatting codes can be one of the following:
		1 Left align/attribute 10 limits/no break—Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows. <pre>..... now is the time now is the time for all g ood men to come to me</pre>
		1x Left align/expand column/word break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre>..... now is the time now is the time for all good men to come to me</pre>
		n Right align/ignore attribute 10/no break—(Numeric specification—obsolete) Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns, if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows.

#	Attribute	Description
		<pre> now is the time now is the time for all good men to come to me </pre>
	r	Right align/ignore attribute 10/no break—Data is right aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
	rx	Right align/expand and ignore column/no break—Data is right aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
	t	Left align/attribute 10/word break—(Text specification) Data is left aligned, presented in a column width specified by attribute 10. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> now is the time now is the time for all good men to come to me </pre>
	tx	Left align/expand column/word break—(Same as 1x) Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. If the data width exceeds the column width, wrap the data in successive rows, initiating the wrap at the space character. <pre> </pre>

#	Attribute	Description
		<pre> now is the time now is the time for all good men to come to me </pre>
	u	Left align/ignore attribute 10/no break—Data is left aligned. Ignore the attribute 10 column width and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. If the text is wider than the display column width, cascade data into the adjacent column to the right. <pre> now is the time now is the time for all good men to come to me </pre>
	ux	Left align/expand and ignore column/no break—Data is left aligned. Disregard the attribute 10 column width and make a new column width based on the available space remaining in the display (specified by the term width), dividing it evenly between the other x output displays. Ignore the new column width anyway, and overwrite the adjacent columns if necessary. Data is truncated only when it hits the term width limit. Data does not wrap in successive rows. <pre> now is the time now is the time for all good men to come to me </pre>
	w	Output processor is attribute or noncolumnar. Process this attribute through the Output processor before displaying. Any valid Output processor dot (.) command can follow this code. Dot commands following the w are prefixed to the attribute. The attribute itself may contain embedded Output processor dot commands. A second value here can contain dot commands to be performed <i>after</i> the attribute is output. The Output processor .char command is used to output unprintable characters and D3 segment, value, MultiValue, and subvalue marks. For example, w.c.bf centers text and makes it boldface (].xbf turns off the boldface after it outputs the attribute). This code only works for noncolumnar formats. It reverts to code 1 for columnar reports.
	ww	Output processor all attributes to end of item or noncolumnar. Process all attributes from the current attribute through the end of the item through Output processor before displaying. Any valid Output processor dot command may follow this code. Dot commands following the w are prefixed to the attribute. The following attributes themselves may contain embedded Output processor dot commands. A second value here may contain dot commands to

#	Attribute	Description
		be performed <i>after</i> the attributes are output. As an example, <code>w.c.bf</code> centers text and makes it boldface. <code>w.c.bf].xbf.skip2</code> centers the first line of text, turns on boldface, prints the text. At the end of the text, turns off boldface and prints two blank lines. This code only works for noncolumnar formats. It reverts to code 1 for columnar reports. If the justification code is unrecognizable, it defaults to the 1 code.
10	column-width	Defines the number of characters that are to be displayed on one line. If the number of characters in the substitute heading (attribute-defining item attribute 3) exceeds the column width, the column width is expanded to the heading size. If the number of characters in the heading is less than the column width, the heading is padded with periods to fill the column width. If the attribute data is wider than this, then it displays according to the justification directions (attribute 9).
11-13	(Reserved)	
14	input-conversion	Processing codes applied at data entry time. Processing codes can be used to validate data entry, as well as transform data before storing it. This attribute is only used with Update Processor.
15	macro	The information in this attribute is used to pass an attribute name list to Update Processor when zooming to another item. The values named in this attribute are separated by spaces—not punctuation. These secondary file attribute names are used in a zoom when input-conversion of the same attribute-defining item has a translate to a secondary file. If the specified attribute name does not exist in the dictionary, it is ignored.
16	Reserved	
17	description	Contains a description of this attribute use. When using Update Processor, typing ? and pressing ENTER displays this field as online help. It is usually the application programmer who enters information into this attribute.
18-29	(Reserved)	

Related reference

[Default values](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Proc item](#)

[Macro item](#)

[Menu item](#)

[Connectives](#)

Default values

Here is a list of predefined attribute names that may be used in AQL statements on data files.

Item-ID	D/CODE	A/AMC	Notes
A0	A	00	(item-ID)
A1	A	01	(attribute specifier)
A2	A	02	
A3	A	03	
A4	A	04	
A5	A	05	
A6	A	06	
A7	A	07	
A8	A	08	
A9	A	09	
A10	A	10	
A11	A	11	
A12	A	12	
A13	A	13	
A9998	A	9998	(<i>nn</i> th item processed)
A9999	A	9999	(size of item)
0	A	00	
1	A	01	
2	A	02	
3	A	03	
4	A	04	
5	X	05	
6	X	06	
7	A	07	
8	A	08	
9	A	09	
10	A	10	

Here is a list of predefined attribute names that may be used in AQL statements on dictionary files:

Item-ID	D/CODE	A/AMC	Notes
D/CODE	A	01	
A/AMC	A	02	
F/BASE	A	02	

Item-ID	D/CODE	A/AMC	Notes
F/MOD	A	03	
S/NAME	A	03	
V/TAG	A	03	
V/STRUC	A	04	
L/RET	A	05	
L/UPD	A	06	
V/CONV	A	07	
V/CORR	A	08	
V/TYP	A	09	
V/MAX	A	10	
F/REALLOC	A	13	
D/SIZE	A	9999	(<i>nn</i> th item processed)
SIZE	A	9999	(size of item)

Related reference

[File dictionary](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Proc item](#)

[Macro item](#)

[Menu item](#)

[Connectives](#)

Suppress attribute item

These define specific attributes that are not to be displayed in AQL reports, such as list-item reports.

Master Dictionary

#	Attribute	Description
0	item-ID	Attribute name which is the symbolic name used to refer to this attribute. This name is used to reference the defined attribute. Each attribute-defining item in a file that must have a unique attribute name.
1	dictionary-code	Defines the type of this dictionary item. For an attribute-suppressing item, this must be an x.
2-16	(Reserved)	
17	description	Contains a description of this attribute use. When using Update Processor, typing ? and

#	Attribute	Description
		pressing ENTER displays this field as online help. It is usually the application programmer who enters information into this attribute.

Related reference

[File dictionary](#)

[Default values](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Proc item](#)

[Macro item](#)

[Menu item](#)

[Connectives](#)

Synonym item (Q-pointers)

Q-pointers are used in account master dictionaries to point to other files.

Master Dictionary

#	Attribute	Description
0	item-ID	Name to be used as the synonym.
1	dictionary-code	Defines the type of this dictionary item. For a file dictionary file-defining item, this must be q. See the set-file command topic in the <i>D3 Reference Manual</i> for information on creating Q-pointer items automatically.
2	master dictionary	Contains the name of the master dictionary (account) where the destination file resides. If the account name is not specified, the current account name is assumed.
3	filename	Contains the name of the destination file to which this item points. If the filename is not specified, assume it has the same name as the account name (specified in master dictionary attribute).

Related reference

[File dictionary](#)

[Default values](#)

[Suppress attribute item](#)

[Command item](#)

[Proc item](#)

[Macro item](#)

[Menu item](#)

[Connectives](#)

Command item

Commands that the user can perform and each account has its own environment and can have its own custom set of commands.

Command definition entries are supplied with D3, and can be generated when a FlashBASIC program is cataloged.

Master Dictionary

#	Attribute	Description
0	item-ID	Name of the command.
1	dictionary-code	For a command-defining item, this must be v.
2	transfer-point	TCL passes control to this process at this address (for example, the mode-id in hexadecimal).
3	secondary-transfer	Its use depends on attribute 1 and 2.
4	tertiary-transfer	Its use depends on attribute 1, 2, and 3.
5	parameter-string	TCL parameters control treatment of the items retrieved by TCL commands to be passed to the processor, whose entry point is defined in attribute 3. Parameter-string can be zero or more of the following:
		c Copy item to a work area.
		f D3 Update Processor file parameters only (ignore item-list).
		n Okay if item is not on file.
		p Print the item-ID if the item-list is and asterisk (*) (all items) or a selected item-list.
		s Ignore the select-list, because an explicit item-list is required.
		u Items are updated by processor.
		v Final entry required on end of input.
6 - 29	(Reserved)	

Related reference

[File dictionary](#)

[Default values](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Proc item](#)

[Macro item](#)

[Menu item](#)

[Connectives](#)

Proc item

Procs are stored sets of TCL or AQL commands. Each account has its own environment and can have its own custom set of procs.

Proc is a programming language, and includes the capability to execute TCL or AQL commands, much like UNIX's shell language or Job Control Language (JCL).

Master Dictionary

#	Attribute	Description
0	item-ID	Name of the proc.
1	dictionary-code	For a command-defining item, this must be pq.
2-n	<i>proc-statement-1</i> to <i>proc-statement-n</i>	TCL passes control to the PROC processor and it executes the Proc statements.

The Proc can contain as many statement attributes as desired.

For a more detailed description of the Proc language, see [Proc](#).

Related reference

[File dictionary](#)

[Default values](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Macro item](#)

[Menu item](#)

[Connectives](#)

Macro item

The master dictionary can include items called macros.

A macro can contain:

- One or more TCL statements
- Input required by the statements

It is invoked by entering its name (item-ID) at the TCL prompt. In addition, any text that is typed following the macro item-ID is appended to the first TCL statement in the macro line. This allows for the creation of a general macro and for assigning specific criteria in the TCL statement.

To execute the macro, type the macro name (item-ID) at the TCL prompt and press ENTER. After the TCL instructions are executed the system returns to the TCL prompt. Macros are created directly in the master dictionary using Update Processor or by invoking the **create-macro** command, which creates a macro from the last TCL sentence executed.

Master Dictionary

#	Attribute	Description
0	item-ID	Name of the macro.

#	Attribute	Description
1	dictionary-code	For a macro item, this must be either an <code>m</code> or <code>n</code>. Comments may follow after the dictionary-code to document the macro. These comments also may be displayed as a help message if this menu is invoked by another menu and you type <code>?</code> and press ENTER. At least one space must separate the dictionary-code from the comments. <code>m</code> The macro is to display each statement first and await editing and a carriage return before executing. <code>n</code> The nonstop macro executes all statements immediately after pressing ENTER.
2 - n	<i>statement-1</i> {<vm> input-1} to <i>statement-n</i> {<vm> input-n}	The next TCL statement to be executed is optionally followed by a value mark and any input needed for the statement. As many input entries as required may be entered, separated with value marks. The macro can contain as many statement or input attributes as desired. If the macro contains more than one TCL statement, it is recommended that the macro not include procs or other macros as part of the TCL sentence, since control is not returned to the current macro.

Related reference

[File dictionary](#)

[Default values](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Proc item](#)

[Menu item](#)

[Connectives](#)

Menu item

A menu is an item stored in the account's master dictionary. The item is used by the menu processor to produce a formatted menu that includes presentation of options. The menu is created directly in the master dictionary and is invoked by entering its item-ID at TCL, similar to a TCL command. Each account has its own environment, and can have its own custom set of menus.

Master Dictionary

#	Attribute	Description
0	item-ID	Name of the menu.
1	dictionary-code	For a menu-defining item, this must be <code>me</code>. Comments can follow the <code>me</code> dictionary-code to document the menu. These comments also can be displayed as a help message if this menu is invoked by another menu and you type <code>?</code> and press ENTER. At least one space must separate <code>me</code> from the comments.

#	Attribute	Description
2	title	Text to be displayed as the menu title.
3 - n	option-1 {<vm>help-text-1} TCL-statement {<vm> TCL Statement...}} to option-n {<vm> help-text-n <vm> TCL-statement {<vm> TCL Statement...}}	Text of the option as it is appears on the menu, followed by a value mark, optional help text followed by a value mark, and one or more TCL statements each followed by a value mark. To request help, enter the option number for which help is desired, followed or preceded by a question mark. The help entry is optional. If the help entry is missing, the menu processor scans the TCL statement to determine if it invokes a macro or menu. If it does, the processor then scans the first line of the invoked macro or menu for comments. If it has comments, they are displayed as the help information. The TCL statements are the statements to be executed for that option. These are any statements that can be entered at TCL, including another menu name. Each option can include as many TCL statements as desired. The entries for each menu option are separated by value marks. The menu can contain as many options as desired.

Related reference

[File dictionary](#)

[Default values](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Proc item](#)

[Macro item](#)

[Connectives](#)

Connectives

Connectives are special words that have a relational meaning in AQL statements. Each account has its own environment, and can have its own custom set of connectives. Connectives are supplied by D3, and there is no provision for modifying them.

Master Dictionary

#	Attribute	Description
0	item-ID	Name of the connective.
1	dictionary-code	For a connective defining item, this must be a c.

Related reference

[File dictionary](#)

[Default values](#)

[Suppress attribute item](#)

[Synonym item \(Q-pointers\)](#)

[Command item](#)

[Proc item](#)

[Macro item](#)

File definition item

The table below lists and describes the attributes for file definition items.

#	Attribute Name	System Dictionary	Master Dictionary	Ordinary Data File
0	item-ID	item-ID	item-ID	item-ID
1	d/code	d{lpsxy}	d{lpsxy}	d{lpsxy}
2	f/base or a/amc	Base FID of file	Base FID of file	Base FID of file
3	f/mod or v/tag	Modulo of file	Modulo of file	Modulo of file
4	v/struc	...	...	...
5	l/ret	Retrieval lock codes	Retrieval lock codes	Retrieval lock codes
6	l/upd	Updates lock codes	Updates lock codes	Updates lock codes
7	v/conv	System Dictionary Password	Account Password	Output processing codes
8	v/corr	Correlative processing codes	Correlative processing codes	Correlative processing codes
9	v/type	Output justification {lnrtuwwwx}	Output justification {lnrtuwwwx}	Output justification {lnrtuwwwx}
10	v/max	Column width for file name	Column width for account name	Column width for file name
11	v/min	...	...	...
12	(Reserved)			
13	f/realloc	Reallocation modulo	Reallocation modulo	Reallocation modulo
14	input-conversion	Input processing codes	Input processing codes	Input processing codes
15	f/attrib or input macro	Default input attribute list	Default input attribute list	Default input attribute list
16	output macro	Default output attribute list	Default output attribute list	Default output attribute list
17	description	Description	Description	Description
18	output-macro	macros	macros	macros
19	Global Hotkey	macros	macros	macros
20	Hotkey 1	macros	macros	macros
21	Hotkey2	macros	macros	macros
22	Hotkey3	macros	macros	macros
23	Hotkey4	macros	macros	macros
24	Hotkey5	macros	macros	macros
25	Hotkey6	macros	macros	macros

#	Attribute Name	System Dictionary	Master Dictionary	Ordinary Data File
26	Hotkey7	macros	macros	macros
27	Hotkey8	macros	macros	macros
28	Hotkey9	macros	macros	macros
29	Hotkey0	macros	macros	macros

Related concepts

[File-defining items](#)

[Attribute-defining items](#)

Definition item

The table below lists and describes the attributes for definition items.

#	Name	Attribute	Substitute	Suppress-Attribute
0	item-ID	attribute name	attribute name	attribute name
1	d/code	a	s	x
2	f/base or a/amc	Attribute mark count	Attribute mark count	Attribute mark count
3	f/mod or v/tag	Column heading	Column heading	...
4	v/struc	Controlling/Dependent attribute	Controlling/Dependent attribute	...
5	l/ret	...	...	...
6	l/upd	...	...	...
7	v/conv	Output processing codes	Output processing codes	...
8	v/corr	Correlative processing codes	Correlative processing codes	...
9	v/type	Output justification {1nrtuwwx}	Output justification {1nrtuwwx}	...
10	v/max	Column width	Column width	...
11	v/min	...	...	...
12	(Reserved)	...	...	...
13	f/realloc	Reallocation modulo	Reallocation modulo	...
14	input-conversion	Input processing code	Input processing code	...
15	f/attrib or input macro	Zoom/Update Processor attribute list	Zoom/Update Processor attribute list	...
16	output macro	...	...	...
17	description	Description	Description	Description
18	(Reserved)	...	...	...

#	Name	Attribute	Substitute	Suppress-Attribute
19	(Reserved)	...	...	...
20	(Reserved)	...	...	...
21	(Reserved)	...	...	...
22	(Reserved)	...	...	...
23	(Reserved)	...	...	...
24	(Reserved)	...	...	...
25	(Reserved)	...	...	...
26	(Reserved)	...	...	...
27	(Reserved)	...	...	...
28	(Reserved)	...	...	...
29	(Reserved)	...	...	...

Related reference

[Connective, macro, and menu items](#)

[Update processor prestore, proc, and Q-pointer items](#)

[Super Q-pointer and cataloged FlashBASIC items](#)

[Item definition summary](#)

Connective, macro, and menu items

The table below lists and describes the attributes for connective, macro and menu items.

#	Name	Connective	Macro	Menu
0	item-ID	connective name	macro name	menu name
1	d/code	c	m or n	me
2	f/base or a/amc	...	TCL/AQL commands	Menu title
3	f/mod or v/tag	...	...	Menu options
4	v/struc	...	...	...
5	l/ret	...	...	...
6	l/upd	...	...	...
7	v/conv	...	...	...
8	v/corr	...	...	...
9	v/type	...	...	...
10	v/max	...	...	...
11	v/min	...	...	...

#	Name	Connective	Macro	Menu
12	(Reserved)	...	...	...
13	f/realloc	...	...	...
14	input-conversion	...	...	...
15	f/attrib or input macro	...	...	...
16	output macro	...	...	...
17	description	Description	...	...
18	(Reserved)	...	...	...
19	(Reserved)	...	...	...
20	(Reserved)	...	...	...
21	(Reserved)	...	...	...
22	(Reserved)	...	...	...
23	(Reserved)	...	...	...
24	(Reserved)	...	...	...
25	(Reserved)	...	...	...
26	(Reserved)	...	...	...
27	(Reserved)	...	...	...
28	(Reserved)	...	...	...
29	(Reserved)	...	...	...

Related reference

[Definition item](#)

[Update processor prestore, proc, and Q-pointer items](#)

[Super Q-pointer and cataloged FlashBASIC items](#)

[Item definition summary](#)

Update processor prestore, proc, and Q-pointer items

The table below lists and describes the attributes for update processor prestore, proc, and Q-pointer items.

#	Name	Update Processor Prestore	Proc	Synonym (Q-Pointer)
0	item-ID	prestore name	proc name	synonym (source) file name
1	d/code	p	pg	q
2	f/base or a/amc	Update Processor prestore commands	proc statements	target (destination) account
3	f/mod or v/tag	...	...	target (destination) file name

#	Name	Update Processor Prestore	Proc	Synonym (Q-Pointer)
4	v/struc	...	...	...
5	l/ret	...	...	...
6	l/upd	...	...	...
7	v/conv	...	...	output processing codes
8	v/corr	...	...	...
9	v/type	...	...	...
10	v/max	...	...	...
11	v/min	...	...	...
12	(Reserved)	...	...	...
13	f/realloc	...	...	...
14	input-conversion	...	...	...
15	f/attrib or input macro	...	...	...
16	output macro	...	...	...
17	description	Description	...	...
18	(Reserved)	...	...	...
19	(Reserved)	...	...	...
20	(Reserved)	...	...	...
21	(Reserved)	...	...	...
22	(Reserved)	...	...	...
23	(Reserved)	...	...	...
24	(Reserved)	...	...	...
25	(Reserved)	...	...	...
26	(Reserved)	...	...	...
27	(Reserved)	...	...	...
28	(Reserved)	...	...	...
29	(Reserved)	...	...	...

Related reference

[Definition item](#)

[Connective, macro, and menu items](#)

[Super Q-pointer and cataloged FlashBASIC items](#)

[Item definition summary](#)

Super Q-pointer and cataloged FlashBASIC items

The table below lists and describes the attributes for Super Q-pointer and cataloged FlashBASIC items.

#	Name	Synonym (Super Q-Pointer)	Cataloged FlashBASIC Definition Item	Cataloged FlashBASIC Definition Item
0	item-ID	synonym (source) file name	command name	cataloged program name
1	d/code	q{s}	v{x}	cc or dc
2	f/base or a/amc	...	transfer point	(Reserved)
3	f/mod or v/tag	target host system : target file name	secondary transfer point	f
4	v/struc	...	tertiary transfer point	file path to program object code
5	l/ret	...	parameter string	...
6	l/upd	...	parameters {cfnpsuz}	...
7	v/conv	output processing codes	...	...
8	v/corr	...	...	...
9	v/type	...	...	...
10	v/max	...	...	...
11	v/min	...	...	...
12	(Reserved)	...	...	...
13	f/realloc	...	...	...
14	input-conversion	...	...	...
15	f/attrib or input macro	...	...	...
16	output macro	...	...	...
17	description	Description	...	...
18	(Reserved)	...	...	...
19	(Reserved)	...	...	...
20	(Reserved)	...	...	...
21	(Reserved)	...	...	...
22	(Reserved)	...	...	...
23	(Reserved)	...	...	...
24	(Reserved)	...	...	...
25	(Reserved)	...	...	...

#	Name	Synonym (Super Q-Pointer)	Cataloged FlashBASIC Definition Item	Cataloged FlashBASIC Definition Item
26	(Reserved)	...	...	...
27	(Reserved)	...	...	...
28	(Reserved)	...	...	...
29	(Reserved)	...	...	...

Related reference

[Definition item](#)

[Connective, macro, and menu items](#)

[Update processor prestore, proc, and Q-pointer items](#)

[Item definition summary](#)

Item definition summary

Here is a chart showing all the definition types and their applicability:

Dictionary Codes		System Dictionary	Master Dictionary	File Dictionary
A	Attribute-defining item			*
C	Connective	*	*	
CC	Compiled FlashBASIC programs	*	*	
D	File-defining item (D-pointer)	*	*	
DC	(Obsolete) Compiled FlashBASIC programs)	*	*	
M	Macro	*	*	
ME	Menu	*	*	
N	Macro (nonstop)	*	*	
P	Update Processor prestore item	*	*	
PQ	Proc	*	*	
Q	Q-pointer	*	*	
S	Substitute attribute-defining item			*
V	Command	*	*	
X	Suppress attribute-defining item			*

Related reference

[Definition item](#)

[Connective, macro, and menu items](#)

[Update processor prestore, proc, and Q-pointer items](#)

[Super Q-pointer and cataloged FlashBASIC items](#)

Controlling and dependent structures

Controlling and dependent structures allow attributes to be linked together within an item. One attribute controls one or more other attributes. The controlling/dependent structure is defined in attribute 4 (structure) of the attribute-defining item.

- For a controlling attribute, the structure is:

```
c; attribute#{; attribute#}...
```

where *attribute#* is the attribute number of the dependent attribute. This attribute may control more than one other attribute. Multiple attribute numbers are separated by semicolons.

- For a dependent attribute, the structure is:

```
d; attribute#
```

where *attribute#* is the attribute number of the controlling attribute. There may be only one attribute number on which this is dependent.

The rules for a controlling/dependent relationship are:

- Controlling/dependent relationship always involves at least two attribute-defining items:
 - one controller
 - one dependent
- One controlling attribute controls one or more dependent attributes.

There *must* be a dependent attribute.

- When inputting data, both controlling and dependent attributes must be entered even if one of the values is null.
- When displaying data, you must specify the controlling attribute to show a dependent attribute.

Controlling/dependent relationships can be very useful when using a more complicated item structure. For example, an invoice file can be created with the following attributes:

invoice.number invoice.date customer.account.number part.number quantity price freight.charges taxes

The part number, quantity, and price attributes are prime candidates for a controlling/dependent structure. The part number is the controller and both the quantity and price attributes depend on the part number. In this case, part number must be part of the output list in order to output quantity or price.

When using AQL or Update Processor, *not every* dependent attribute must be listed. However, the controlling

attribute must be output to see any or all dependent attributes. When MultiValues are entered, a part number value must have corresponding quantity and price values, even if these are null. This assures that the third value in the price attribute corresponds to the third value in the part number attribute.

The Update Processor recognizes the controlling/dependent structure and tries to list the attribute names across the screen, reminding you that these are controlling/dependent attributes.

To use a controlling or dependent structure some of the time and avoid it other times, create an additional attribute-defining item for these attributes and keep the controlling or dependent structure empty.

As the controlling or dependent structure links the n th value in the controlling attribute to the n th value in the dependent attributes, it would be unwise to permit these values to be independently edited, or the number of values can become unbalanced. If it does, the system may report it with error [68] from the messages file.

Processing codes

Input processing codes (also called input conversions), correlatives, and output processing codes (also called output conversions) are special codes used in the file-defining and attribute-defining items to allow data to be stored in one way and retrieved or displayed in another way. They can also be used to convert data as it is entered into a more efficient format for storage, and then unconvert them for display.

Processing codes can:

- Interact with other attributes to return summary information.
- Perform functions on portions of the data.
- Specify a range within which the data must fall.
- Place constraints on the length of the data to be input or output.
- Mask off certain characters, convert others, and scan for specific patterns.

Input conversions, correlatives, and output conversions

To understand the difference between a Conversion and a Correlative, consider the different formats of data with which D3 deals.

Related concepts

[Processing code summary](#)

Input format

This is the actual text typed by the user.

The D3 system can either:

- Leave the data exactly as it is entered.
- Perform functions and manipulate the data before storing it. The manipulation function is stored in the input conversion attribute, specified in the dictionary attribute-defining item.

Related concepts

[Internal format](#)

[Intermediate format](#)

[Output format](#)

Internal format

This is the data exactly as it is stored in the item. When data is retrieved from an item in a file, it is retrieved in internal format. When the generic attribute names of a0 through a99 are used, there are no processing codes performed, so the data displays exactly as it was stored.

Related concepts

[Input format](#)

[Intermediate format](#)

[Output format](#)

Intermediate format

After the data has been retrieved and is still in internal format, the D3 system has the opportunity to manipulate or transform the data to an intermediate format. These manipulations and transformations are specified in the dictionary attribute-defining item, correlative attribute. If there is no correlative, then manipulation is not performed.

The intermediate format is used in an AQL sentence in the following situations:

- Attribute name is part of a sort key
- Data is compared to a selection criteria
- Attribute name is part of a print limiter
- Attribute name is part of a `total` or `grand-total` connective
- Data produces a control break in an AQL report
- Data is printed except on control break lines

AQL Sorts and Compares occur on data in its intermediate format.

Related concepts

[Input format](#)

[Internal format](#)

[Output format](#)

Output format

Once the data has been retrieved and any correlatives have been performed, the system can perform additional manipulative functions on the intermediate form of the data using the function codes specified in the dictionary attribute-defining item, output-conversion. When those functions, if any, have been completed the data is in output format.

If there is a processing code in both the correlative and output conversion, the correlative result does not feed into the output conversion as its input. The correlative is processed and used in any sort or comparison operation. If there

is no output conversion, the correlative result displays. If there is an output conversion, the result of the correlative is discarded, the output conversion is processed, and the result displays.

If you use MultiValued processing codes, the result of the last processing code can be used as input data by the next processing code.

More than one processing code can be specified in the attribute. Separate each processing code with a value mark.

All arithmetic done within correlatives is integer arithmetic. To use values with decimal places such as 41.25, 3.55, 8.00, 22.25, declare a fixed decimal position for all data within the file using the `m1nm` or `mrnm` code. The data can be stored as 4125, 355, 800, 2225, and so on. When the values are retrieved, the decimal place is reinserted.

If there are internal values with a varying decimal point embedded in the values such as 4.125, 3.5, 8, 22.25, scale the values from the 000.00 form to a 00000 form, perform the arithmetic, and descale it back to the 000.00 form.

The following `f` code scales a number with up to five decimal places:

```
F x(G0.1); x(G1.1);(S;*‘0’);(ML%5);:
```

where `x` is the attribute-mark count.

Related concepts

[Input format](#)

[Internal format](#)

[Intermediate format](#)

Processing code summary

The table below summarizes the processing codes.

Code	Description
a	Arithmetic function
b	Bridge correlative
c	Concatenate attribute/literal values
call	Call a FlashBASIC subroutine
callx	Call a FlashBASIC subroutine
cu	Character update
d	Data Conversion
f	Function definition
g	Group extraction code
i	Index correlative
id	Item-ID correlative
if	If then else correlative
l	Attribute length limits

Code	Description
mc	Mask character conversions
mi	Mask input
ml	Mask character conversion; left-aligned string (used primarily for text)
mr	Mask character conversion; right-aligned string (used primarily for numbers and dollar amounts)
ms	Mask sort conversion; sort attribute according to the sequence of characters in <code>errmsg seq</code> item
mt	Mask time conversion; convert time of day from internal to external format
mx	Mask hexadecimal conversion; convert decimal numbers to hexadecimal numbers
my	Mask decimal conversion; convert hexadecimal numbers to decimal numbers
o	Sort in ascending order
p	Pattern match
r	Range match
s	Substitute specified values
t	Extract a fixed field from an attribute value
<i>tfile.name</i>	Translate attribute values from another file or current file
u	User defined conversions or attributes (User-Exits)
v	Value code
x	Display only
zc	Zip code

Related concepts

[Input conversions, correlatives, and output conversions](#)

Designing a database

Design is the most important part of database application development. No amount of clever processing and computer speed can make up for a poorly designed database. While a poorly designed database may get the job done, the system works harder to process the work, costing time and disk space. Over time, it is difficult to adapt to changing needs. A well designed database is fast, efficient, easy to use, and easy to maintain.

The two biggest considerations in designing a database are:

- What is the data (attributes) that each item has?
- How is the data related?

Details

The D3 DBMS has a unique file structure that is universal throughout the system. All files, whether they are system, program, spreadsheets, text files or data files have this same structure. This consistent file structure saves programming time because FlashBASIC and AQL are optimized to interface with it.

D3 file structure allows great flexibility in designing databases. The forgiving nature of the system tolerates inefficient use of the file structure. It is amazing how well a D3 database functions even when it is poorly designed. In the D3 system, an attribute can be added later. If the file is too small, it can be resized. The data displayed can be reformatted by updating the attribute-defining item.

To design a good database, consider how:

- Attributes interact with other attributes
- Attributes are stored on disk
- Items are entered initially
- Items are updated and modified
- To minimize data storage
- To minimize search response time
- To restrict access to certain users

Good database design requires more knowledge of how D3 dictionaries work, and how to take advantage of this power. Knowing how to link attributes together with the controlling/dependent structure helps ensure data integrity. Translate correlatives help link attributes with other local or external attributes for faster speed and smaller items.

Bridge correlatives let you update more than one item at a time.

Related concepts

[Database rules](#)

[Attribute name considerations](#)

[Item-ID considerations](#)

Database rules

When designing a database, follow the rules described below.

Avoid duplicating data.

There is no need to store information in an attribute if it is available in another attribute in this file or another file. For example, an Invoice file may not need to keep customer name, address and telephone number attributes. All of these attributes are available from the Customer file. Keeping this information in a separate file reduces the amount of disk space and only requires one update to change the customer information.

Derive values from existing attributes or translate them from other files whenever possible.

Often, attributes contain information that is readily retrieved from other items, such as an invoice containing a part number description (which should be available from the part number file), or an invoice total (which can be calculated from the list of invoice prices and quantities). The goal here is to create a dictionary attribute-defining item that refers to the data that exists elsewhere and returns it in the required format. The actual value need not reside in this attribute.

Keep attribute data homogeneous.

Use an attribute for one and only one type of data. Never combine two or more data types into a single attribute. If two pieces of information go together, create another attribute to keep track of the second part. The controlling/dependent structure can be used to link them together. When storing numbers with an embedded decimal position, use the Mrxy processing code to normalize the stored data. If text is always displayed as uppercase, use the mcu processing code on input to convert to uppercase before storing the data.

Store data in the internal format.

Attributes such as dates and times should be stored in the internal format. The internal format is an integer number that represents the number of days before or after December 31, 1967. Dates plus or minus 200 years only require storage of up to five digits. Internal format values can be sorted more quickly or compared against other values than their external companions.

- Store numeric values in the normalized internal integer format. This is especially important when using AQL, because numbers that are stored in a noninternal format do not sort, select, or total correctly.
- Store text attributes in the format in which they are usually displayed. When necessary, use processing codes to normalize data prior to sorting.
- Telephone numbers, social security numbers, zip codes, and other numbers that contain specific patterns can be stored without the pattern delimiters; a simple processing code restores the pattern before output.

In short, using internal formats compresses data and maintains consistent data form.

Keep attributes and items small.

To reduce record size even further, keep the size and number of attributes to a minimum. Use processing codes to reduce the size of the data stored. Consider creating a new file to place rarely used attribute values and use the same item-ID as in the original file for speedy access. Store historical data in another file (another data section of the same file, such as sales,sales and sales,history). D3 performs best if the item size is kept below 1720 bytes.

Arrange attributes in order of importance and relative size.

With D3, attributes do not have to be in any particular order. However, planning of the attribute order makes files more efficient and programs (AQL as well as FlashBASIC) run faster. Generally, the rule is to organize attributes in descending order of importance, reading often used attribute values quickly, and in ascending order of average attribute size so time is not wasted skipping over long attribute values to get to the desired attributes.

By efficiently using D3 file structures, applications are written in less time, data occupies less disk space, programs run faster, and changes are made more easily.

Related concepts

[Details](#)

[Attribute name considerations](#)

[Item-ID considerations](#)

Attribute name considerations

The name of the attribute-defining item is used when specifying this attribute in AQL reports. It also is known as the attribute-defining item, item-ID or a0.

In naming attribute-defining items, keep in mind the following:

Attribute names, since they are item-IDs, can be up to 101 characters long but only the first 24 are significant. Do not use any of the following characters as part of an attribute-defining item attribute name:

- AQL delimiters (characters 252 through 255)
- The following five System string delimiters: ' " \ ()
- Control characters (CTRL+A through CTRL+Z)

The use of AQL delimiters in a file is not permitted. The rest (system string delimiters, control characters, spaces) can be accepted as characters in attribute names or in item-IDs, but are not advised.

The following characters are valid within attribute names: ! # \$ % & + - . / : ? @ _

As examples, PART#, PART-NO, PART_NO, and PART.NO: are all valid and different attribute names.

Optionally, the file may be created so that it is case sensitive. For example, B and b are treated as different characters. A caveat is that some names, like PART#, are valid in AQL but are invalid FlashBASIC variable names.

Make attribute names as descriptive as possible.

The attribute name should adequately describe the data and how it is handled without the need to look at the actual definition. Consider some typical names:

PART#

NAME

ADDR-1

ADDR-2

COST

PHONE

Make attribute names short.

Try to use shorter names rather than longer ones because:

- Attribute-defining items are often typed in AQL sentences and are too long to type `customer.address.line.one` rather than `addr.1`.
- Attribute names are the default column headings and override the column width if the name is longer than the column width specification.
- Attribute-defining items are inexpensive.

It takes very little space to store an attribute-defining item. Create additional attribute-defining items as needed for specialized views of data.

Avoid:

- Numeric names for attribute-defining items.
- An attribute name of `1099` can later be confusing whether `1099` stands for an item-ID or the attribute name in the sentence, `LIST CUSTOMER 1099`.
- Names of words already used as AQL commands or modifiers, for example `description`.

Related concepts

[Details](#)

[Database rules](#)

[Item-ID considerations](#)

Item-ID considerations

Another decision facing the designer is the makeup of the item-ID.

Here are a few guidelines for choosing an item-ID:

AQL delimiters in a file is not permitted.

The rest (system delimiters, control characters, spaces) can be accepted as characters in your item-IDs but are not advisable.

It is permitted to use the following characters within item-IDs: ! # \$ % & + - . / : ? @ _

For example, BOB.SMITH, BOB-SMITH, BOB_SMITH, and BOB/SMITH are all valid (and different) item-IDs. Also, keep in mind that the file may be case sensitive.

Each item-ID in a file must be unique.

- An item-ID must be assigned to only one item in a file. Unique also means that the item-ID cannot be MultiValued, for example, the item cannot have more than one item-ID to reference the data in this item.
- D3 has the ability to create item-IDs by default. D3 can generate an item-ID based on the date, time, and an internal counter. This creates a numeric item-ID nine digits long.
- By using the ID processing code in the file-defining item, D3 creates item-IDs in numerical order, starting with the number provided.
- Calling a FlashBASIC program subroutine from an FDI correlative.

Choose item-IDs that hash well.

Hashing describes how items are distributed among the groups in the file. An ideal situation would be that every group holds an equal number of items. In reality, the distribution of items may not be so even. D3 handles these inconsistencies well, so the system performance degradation they cause is negligible. The actual hashing scheme is described elsewhere, but because of the way the algorithm works, the right characters of the item-ID have the greatest influence on the group to which an item is assigned. This means that sequential alphabetic or numeric item-IDs hash better. Item-IDs should not be assigned in even multiples (411000, 412000, 413000, and so on) as this may cause an uneven distribution of the items in the groups. The **istat** command shows how items are placed within groups of existing files.

Limit the size of the item-ID to as few bytes as possible.

- Access allows item-IDs as large as 101 characters, but only the first 24 are significant.
- In general, smaller is better. Item-IDs less than ten characters long are encouraged. This keeps disk usage to a minimum and it improves the speed of AQL commands, although the increase may be imperceptible in small files.

Choose item-IDs that sort well if the items are sorted by the item-ID often.

If the item-ID is strictly alphabetic or strictly numeric, the results are straightforward. But if the item-ID is partially alphabetic and partially numeric, a sorted list shows the following item-IDs in this order: w2, w3, w399, w4, and w401.

To sort the item-IDs numerically, (for example, w2, w3, w399, w4, and w401) a FlashBASIC program called from the file-defining item correlative is needed to create an index, or use a synonym attribute-defining item with right alignment for attribute zero.

As a reminder, D3 can use the ID correlative code in the file-defining item to automatically assign sequential item-IDs.

Related concepts

[Details](#)

[Database rules](#)

[Attribute name considerations](#)

Creating a database in D3

To create a database, the following steps are necessary:

- Decide which files are needed and for each file:
 - Creating the Data File (dictionary section and data section)
 - Creating attribute-defining items
 - Modifying the file-defining item, if applicable
 - Creating indexes, if any
 - Writing any FlashBASIC subroutines that are called in dictionaries
- Write any FlashBASIC programs required for batch processes
- Write reports

Creating the data file

Creating a data file is easy. D3 files are created by specifying size in terms of frames, or groups. This file size is known as the primary file space, and it is a contiguous section of data on the disk. The size of the file is known as the modulo, and it should always be a prime number.

To create a data file, the command syntax is:

```
create-file filename dict-modulo data-modulo
```

Parameters	Description
<i>filename</i>	Is the (combination dictionary and data sections) name of the data file and should easily identify the contents of the file, such as customers, purchase order codes, inventory, and so on. Filenames can be up to 48 characters long.
<i>dict-modulo</i>	Is the size of the dictionary section in frames, as a prime number integer and is the number of frames to use in creating the dictionary section of the file. Since each frame is 2000 bytes, it is necessary to know how many bytes are used to store all the attribute-defining items for this file. Typical attribute-defining items use from 50 to 200 characters apiece. In this case, each frame holds about ten to 40 attribute-defining items.
<i>data-modulo</i>	Is the size of the data section in frames, as a prime number integer and is the number of frames to use in creating the data section of the file. Since each frame is 2000 bytes, it is necessary to know how many attributes are stored per item, the average number of characters of each attribute, and how many items are stored.

Parameters	Description
	 Note: The <code>prime {number}</code> command can be used to calculate the nearest prime number to your estimated figure.

To create the most efficiently-sized file:

- When a file is created, the size of the dictionary modulo and data modulo must be provided. These frames are set aside for the file, even though it is empty. No other file may access this space.
- The D3 file system is structured such that if the file is too small, or if the file grows beyond its original estimated size, the file automatically enlarges the file by adding overflow frames to it. The system still operates normally, but it takes more time to read overflow frames than it does to read primary file space frames, so the idea is to keep as much data in the primary file space as possible. Do this by specifying a properly sized data modulo. An undersized file uses more overflow frames and performs more slowly. An oversized file wastes disk space.
- Some files are structured so the average size of an item is larger than 1720 bytes. When an item is created that is larger than 1720 bytes, an automatic override takes place, and the item is not written to the primary file space. Instead, the item is placed entirely in overflow frames, and a pointer item is created in the primary file space, and it points to the overflow frames. This scheme improves ordinary search and sort operations performed on the file.

Specifying the proper file size exactly is not as critical as it seems. D3 has a routine called **resize** that examines a file, determines proper file size, and automatically resizes the file.

Create attribute-defining items that define the contents of each attribute.

- Determine the kinds of data that needs to be stored
- Consider how data is stored in the system
- Consider how data is displayed on output
- Name each attribute

Once all the attribute-defining items are created, modify the file-defining item:

- Modify the macro attribute to list all the attribute names in the desired order.
- Modify the correlative attribute to add any bridges, item-ID assignments or called FlashBASIC subroutines.

Now that the file is created and the attributes are defined, create the indexes for the attributes with:

```
create-index filename a-code
```

The setup is now ready for data entry. Invoke the Update Processor by entering:

```
U filename { item-ID}
```

Related concepts

[Sample database design and creation](#)

Sample database design and creation

A sample database of names and addresses is created to show the usual steps involved. Call it the address file.

Related concepts

[Creating the data file](#)

Deciding on attributes and definitions

To create a database, it is necessary to define the data.

Name	The name of the person on file. This example uses First name and Last name. Names can be separated as:				
	Title	First	Middle	Last	Extension
	Senator	Battson		Belfrie	
	Dr.	Sarah	D.	Bellam	Jr.
	Mr.	Noah	O.	Pinyon	Ph.D.
Address	The mailing address. U.S. addresses can be separated as: Care-of name Corporation name Street number, Street name, Apartment number, Additional directions City State Zip Code This example uses Street Address, City, State, Zip code and Country.				
Telephone Number	U.S. telephone numbers can be separated as:				
	Area Code	Exchange	Number	Extension	
	(714)	261	7425	100	

	Telephone numbers can also be differentiated as: Home Work Cellular Pager Fax Modem
International telephone numbers are more difficult, because in addition to the two or three digit country code, each country has its own city and exchange codes, and a different way of displaying the groupings. This example uses phone number category and phone number.	

Next, decide how each of these attributes are stored.

Attribute Number	Attribute Name	Description
1	First Name	Person's first name is a text string, stored as is.
2	Last Name	Person's last name is a text string, stored as is.
3	Street Address	Street portion of the address is not limited to the length of any attribute, but for practical reasons assume that mailing labels have a limit of 30 characters across. If necessary, this can be MultiValued to have several address lines.
4	City	City portion of the address is a text string.
5	State	State portion of the address is the two digit state code for U.S. addresses.
6	Zip Code	Zip code portion of the address is the Zip + 4 code for U.S. addresses.
7	Country	Country portion of the address is assumed to be U.S.A. if null.
8	Phone Category	This describes the type of telephone associated with the phone number (for example, home phone, work phone, fax, cellular, auto cellular, pager, modem).
9	Phone Number	Telephone number is a seven digit number for numbers within the same area code, or a ten digit number for numbers outside the area code.
10	Item-ID	File-defining item correlative is set up to assign new item-IDs as sequential numbers.

The database attributes are further defined, and the size of our database is approximated as follows:

0	Item-ID	4
1	First name	8
2	Last name	8

0	Item-ID	4
3	Street address	15
4	City	15
5	State	2
6	Zip Code	5
7	Country	0
8	Phone category	2
9	Phone number	10
	Total	65

The phone category attribute may reference a phone category file to keep the categories homogeneous. Also, the phone category attribute can be a controlling attribute of the phone number attribute, and keep the category and number on a one to one correlation.

Related concepts

[Creating the file](#)

[Creating attribute-defining items](#)

[Editing the file-defining item](#)

[Creating indexes](#)

[Connecting to the database](#)

Creating the file

In this example, the average item size is approximately 63 bytes. The address book has roughly 500 entries. Therefore, the file size should be about 31,500 bytes.

Each frame is about 2,000 bytes, so the number of frames used is approximately 16 ($31,500/2,000 = 16$). Round this up to the next prime number, 17.

When a file is created, the size for the dictionary section must also be specified. Experience shows that files like this use only a few attribute-defining items. A prime number size for the dictionary section of three, seven or 11 is a good estimate.

The file creation command is:

```
create-file address 3 17
```

Related concepts

[Deciding on attributes and definitions](#)

[Creating attribute-defining items](#)

[Editing the file-defining item](#)

[Creating indexes](#)

[Connecting to the database](#)

Creating attribute-defining items

Now that the file is created, the attribute-defining items can be created.

```
ud addresses f.name

01 dictionary-code      : a
02 attribute-count      : 1
03 substitute-header    :
04 structure            :
07 output-conversion    :
08 correlative          :
09 attribute-type       : T
10 column-width         : 10
14 input-conversion     :
15 macro                :
16 output-macro         :
17 description          : First Name
18 hotkey.all           :
19 hotkey1              :
:
28 hotkey0              :
```

```
ud addresses 1.name

01 dictionary-code      : a
02 attribute-count      : 2
03 substitute-header    :
04 structure            :
07 output-conversion    :
08 correlative          :
09 attribute-type       : T
10 column-width         : 15
14 input-conversion     :
15 macro                :
16 output-macro         :
17 description          : Last Name
18 hotkey.all           :
19 hotkey1              :
:
28 hotkey0              :
```

```
ud addresses addr

01 dictionary-code      : a
02 attribute-count      : 3
03 substitute-header    :
04 structure            :
07 output-conversion    :
08 correlative          :
09 attribute-type       : T
10 column-width         : 30
```

```

14 input-conversion      :
15 macro                 :
16 output-macro          :
17 description            : Address line-1 (street address)
18 hotkey.all             :
19 hotkey1                :
:
28 hotkey0               :

```

ud addresses city

```

01 dictionary-code       : a
02 attribute-count       : 4
03 substitute-header     :
04 structure             :
07 output-conversion     :
08 correlative           :
09 attribute-type        : T
10 column-width          : 30
14 input-conversion      :
15 macro                 :
16 output-macro          :
17 description            : City
18 hotkey.all             :
19 hotkey1                :
:
28 hotkey0               :

```

ud addresses st

```

01 dictionary-code       : a
02 attribute-count       : 5
03 substitute-header     :
04 structure             :
07 output-conversion     :
08 correlative           :
09 attribute-type        : T
10 column-width          : 2
14 input-conversion      :
15 macro                 :
16 output-macro          :
17 description            : State
18 hotkey.all             :
19 hotkey1                :
:
28 hotkey0               :

```

ud addresses zip

```

01 dictionary-code       : a
02 attribute-count       : 6
03 substitute-header     :

```

```

04 structure           :
07 output-conversion   :
08 correlative         :
09 attribute-type      : N
10 column-width        : 10
14 input-conversion    :
15 macro               :
16 output-macro        :
17 description          : Zip code
                        may be : 00000
                        or : 00000-0000

18 hotkey.all          :
19 hotkey1              :
:
28 hotkey0             :

```

ud addresses phone.category

```

01 dictionary-code     : a
02 attribute-count     : 7
03 substitute-header   : category
04 structure           :
07 output-conversion   :
08 correlative         : a7(tPHONE.CATEGORIES;c;;1)["1","5"]
09 attribute-type      : L
10 column-width        : 5
14 input-conversion    : iPHONE.CATEGORIES;a1
15 macro               :
16 output-macro        :
17 description          : Telephone number "category"
                        (show only the 1st 5 characters)
                        Translate to another file to keep the
                        categories.

18 hotkey.all          :
19 hotkey1              :
:
28 hotkey0             :

```

This requires a separate file to be built for phone.categories. Zero is the category code and one is the category description.

The description is entered in the phone.category attribute-defining item. It gets the item-ID of that item, and the attribute-defining item correlative translates the item-ID into the description stored in attribute 1.

The **phone.categories** file needs an index built on attribute 1 to work with the Update Processor.

create-index PHONE.CATEGORIES a1 ud addresses phone.no

```

01 dictionary-code     : a
02 attribute-count     : 8
03 substitute-header   :
04 structure           :
07 output-conversion   :
08 correlative         : aif 8(mcn) = 8 then if 8(L7) = "1" then 8(MR###-

```

```

#####) else if 8(L10) = "1" then 8(MR###-###-
#####) else 8
09 attribute-type           : R
10 column-width             : 12
14 input-conversion         :
15 macro                    :
16 output-macro             :
17 description               : Telephone number
                             If nonnumeric, then "special". Show it as-is.
                             If 7 digits, then "local".Format as: '000-0000'
                             If 10 digits, then "area code".Format as:'000-
                             000-0000'
18 hotkey.all               :
19 hotkey1                  :
:
28 hotkey0                  :

```

In the processing codes above, a telephone number is defined to be either seven or ten digits. If anything else is entered, it is regarded as special and is stored as is. In this way, extensions may be entered as 261-7425 ex. 100.

That defines all of the attributes but a few additions make this even better:

ud addresses lf.name

```

01 dictionary-code         : s
02 attribute-count         : 0
03 substitute-header       :
04 structure               :
07 output-conversion       :
08 correlative             : a2:", ":1
09 attribute-type          : T
10 column-width            : 27
14 input-conversion        :
15 macro                   :
16 output-macro            :
17 description              : Last Name, comma, First Name
18 hotkey.all              :
19 hotkey1                 :
:
28 hotkey0                 :

```

ud addresses address

```

01 dictionary-code         : s
02 attribute-count         : 0
03 substitute-header       :
04 structure               :
07 output-conversion       :
08 correlative             : a3:" ["1","30"] : 4:", ":5:" ":6
09 attribute-type          : T
10 column-width            : 30
14 input-conversion        :
15 macro                   :
16 output-macro            :

```

```

17 description          : Show:
                        street.addr
                        city, st zip
18 hotkey.all           :
19 hotkey1              :
:
28 hotkey0              :

```

The additional attribute names `lf.name`, `f1.name`, and `address` allow several attributes to be combined.

Related concepts

[Deciding on attributes and definitions](#)

[Creating the file](#)

[Editing the file-defining item](#)

[Creating indexes](#)

[Connecting to the database](#)

Editing the file-defining item

With all the attribute-defining items built, the file-defining item can be modified to finalize the definition.

```

ud addresses

01 dictionary-code      : d
02 base                 : 39305
03 modulo               : 17
04 structure            :
05 retrieval-lock       :
06 update-lock          :
07 password             :
08 correlative          : id1001
09 attribute-type       : L
10 column-width         : 10
14 input-conversion     :
15 macro                : f.name lf.name addr city st zip phone.category
                        phone.no
16 output-macro         : lfname address phone.category phone.no
17 description          : The Addresses file
                        Contains names, addresses and phone numbers.
18 hotkey.all           :
19 hotkey1              :
:
28 hotkey0              :

```

Related concepts

[Deciding on attributes and definitions](#)

[Creating the file](#)

[Creating attribute-defining items](#)

[Creating indexes](#)

[Connecting to the database](#)

Creating indexes

Create the indexes for the attributes in lastname-firstname order with:

```
create-index addresses a2:1
```

Creating any Support Files

In this example, the support file **phone.categories** must be created with:

```
create-file phone.categories 1 1
```

```
ud phone.categories desc
```

```
01 dictionary-code           : a
02 attribute-count           : 1
03 substitute-header         :
04 structure                  :
05 retrieval-lock            :
06 update-lock               :
07 password                  :
08 correlative                : id0
09 attribute-type            : L
10 column-width              : 10
14 input-conversion          :
15 macro                     : desc
16 output-macro              :
17 description                : The Telephone Category file
                             : Used by the "Addresses" file.
18 hotkey.all                :
19 hotkey1                   :
:
28 hotkey0                   :
```

```
ud phone.categories
```

```
01 dictionary-code           : d
02 attribute-count           : 39611
03 modulo                     :
04 structure                  :
07 output-conversion          :
08 correlative                :
09 attribute-type            : T
10 column-width              : 20
14 input-conversion          :
15 macro                     :
16 output-macro              :
17 description                : Description
18 hotkey.all                :
19 hotkey1                   :
:
28 hotkey0                   :
```

```

phone.categories desc.....
1                               work
2                               home
3                               cellular
4                               auto cellular
5                               pager
6                               fax
7                               modem

```

With the definitions complete, the data can be entered:

```

u addresses
f.name
l.name
addr
city
st
zip
phone.category
phone.no

```

When the item is filed, the indexes are automatically updated.

Points to Remember

- The attribute-defining item length (column width) in no way restricts how many characters can be entered or displayed. It only specifies a column width on output.
- Only during file creation can the case sensitivity be specified.
- If more attributes need to be added later, a new attribute-defining item can be added to the database, specifying a new attribute number. The existing data does not need to be modified.

Related concepts

[Deciding on attributes and definitions](#)

[Creating the file](#)

[Creating attribute-defining items](#)

[Editing the file-defining item](#)

[Connecting to the database](#)

Connecting to the database

Connect to the D3 Database via the VME, D3 File Manager, or the D3 ODBC Client.

Connecting via the VME

Use telnet to connect to the Virtual Machine Environment (VME) and run green screen applications. Select telnet localhost from the D3 Windows menu.

Connecting via the D3 File Manager

The D3 File Manager provides a GUI application for viewing D3 files and starts automatically if you select **Launch the**

Application at the end of the installation.

Connecting via the D3 ODBC Client

To use ODBC connectivity, install the D3 ODBC Client driver on the client system. Go to the ODBC documentation page on the Rocket Software Web Site and navigate to the ODBC driver on the FTP site:

<http://www.rocketsoftware.com>

Related concepts

[Deciding on attributes and definitions](#)

[Creating the file](#)

[Creating attribute-defining items](#)

[Editing the file-defining item](#)

[Creating indexes](#)

Virtual machine environment in Windows

The Virtual Machine Environment (VME) is a subsystem of D3 Windows. It provides an ASCII mode that is backwards compatible with previous Pick applications. See the *D3 Reference Manual* for more information on the VME.

Accessing the VME

The D3 VME service is a multithreaded server.

This server supports:

- [Serial terminals](#)
Connected directly to a Windows Server, either directly or through modems. A maximum of 255 workstations can be connected to a Windows Server.
- [Telnet emulation](#)
- C++ and Visual Basic objects using Remote Procedure Calls (RPC) remote execution

The VME accesses files and accounts that are managed by the D3 File System.

Although client workstations can be anywhere on the network, the VME runs on a centralized system. For example, the clients may be connected through Telnet. There can be any number of VME servers on a domain. They do not cooperate closely, even though they can share D3 files.

Once logged on to the VME, the TCL prompt displays. Run any ASCII application using the workstation emulation provided with the Telnet client.

Related concepts

[File system access](#)

[Serial terminals](#)

[Telnet emulation](#)

[VME tapes](#)

[VME printers](#)

[Form queue](#)

[Print file](#)

[VME supported OSFI drivers](#)

Starting D3 VME

Run **D3Vme.exe** to start the Service Control Manager on Windows. On Windows, the VME is a background service that starts automatically when the server boots.

During installation the setup procedure runs **D3Vme /install** and installs the service in the Service Database. The **D3Vme** command, runs automatically from the Service Control Manager and starts the server in background. Users can then connect to it using local workstations or Telnet.

When started in the background, no message is displayed. If the service is started successfully, the Service Control Manager (Services in the control panel), displays the D3 Virtual Machine Environment as Started. Otherwise, Windows displays a message box at boot time indicating that one service was not started successfully. The boot result or any error message is logged in the Windows event viewer under Application.

The VME service must be set up as detailed in Service Setup in the Administrators Guide.

Complete the instructions below to start the VME in the background from the Control Panel.

1. Double-click **Services** from the **Control Panel**.
2. Select the **D3 Virtual Machine Environment** service, then click **Start**.

 **Note:** You can also type `net start d3vme` from the MS-DOS command line.

 **Tip:** To start the VME in the foreground, type `d3vme /debug` from the MS-DOS command line. The VME Server starts in the foreground and the user is able to log on to line 0. This mode is a degraded mode with limited console capability.

- When started in the foreground, closing the console or logging off the desktop shuts down the D3 VME.
- When attempting to close the console window, a dialog box displays. Click **OK** to continue the shutdown of D3 or **CANCEL** to abort.
- When logging off the desktop, it is not possible to cancel the logoff and shutdown of D3. A warning message displays. Click **OK** to shutdown D3.

Related tasks

[Stopping D3 VME](#)

Stopping D3 VME

You have several options to stop the VME.

Do one of the following:

- Type the `shutdown` command at the TCL prompt.
- Double-click **Services** from the Control Panel (Windows only), and stop the Virtual Machine Environment service.
- Type `net stop d3vme` at the MS-DOS command line.

 **Note:**

- Shutting down Windows while D3 Windows is still running causes a loss of overflow.



- When started in the foreground, closing the console or logging off the desktop shuts down D3 VME.
- When attempting to close the console window, a dialog box displays. Select **OK** to continue the shutdown of D3 or **CANCEL** to abort.
- When logging off the desktop, it is not possible to cancel the logoff and shutdown of D3. A warning message displays. Select **OK** to shutdown D3. At this point, the only option is to not select **OK**.

Related tasks

[Starting D3 VME](#)

File system access

The VME does not contain application files. User data is located outside the VME, stored on the D3 Windows file system distributed across the network. Data is accessed through the Open Systems File Interface (OSFI), which provides transparent access to D3 applications, and allows sharing data with Visual Basic applications or other VME environments on the network.

The driver name in the `dm,hosts,` file is FSI.

Files located outside the VME are managed by the D3 File Manager. There are several advantages to locating files outside of the VME:

- Data is shared with other applications or with other VME environments, allowing VME users access to several systems.
- Data is distributed. Accounts and files are read from many servers, improving performance and reliability.
- Better multiuser performance. Each file object manages its own locking and disk space, limiting the bottlenecks on shared resources.
- Disk space is used more efficiently. The VME can be left to a very small size, since it needs to handle only spool jobs, user workspace, and some small required files in the dm account.
- The File system is more reliable. Because files are distinct objects on the network, locks and disk space are handled independently of other files, limiting risks.
- Unlimited file size (2 terabytes per file), as opposed to the current 1 GB for the entire VME.

Note that accessing files from the network is slower than with files located inside the VME.

Once an account has been restored into the external file system, log to the external account by creating a Q-pointer

in the MDS. For example, to create a Q-pointer to the external account accounting, create the following item in the MDS:

```
accounting
001 Q{S}
002 fsi:accounting
```

The QS.UPD utility creates all Q-pointers automatically.

To access databases or tables defined in another Network Domain, use Table Naming Conventions as described in the Administrators Guide.

Consider, for example, a large multidomain environment where there is a Personnel database in both the Manufacturing and the Sales domains. The domains `fsi:[Manufacturing]Personnel` and `fsi:[Sales]Personnel` allow accessing the two databases.

 **Note:** Once an external file has been opened by a D3 process running in the VME, the file remains opened, even if the process drops back to TCL. External files are closed only when the user logs off.

Related concepts

[Accessing the VME](#)

[Serial terminals](#)

[Telnet emulation](#)

[VME tapes](#)

[VME printers](#)

[Form queue](#)

[Print file](#)

[VME supported OSFI drivers](#)

Frame size

The VME maps the disk space into 4 KB frames. Store data in FSI, not in VME.

Related concepts

[Restoring accounts](#)

[Saving accounts](#)

Restoring accounts

The VME supports Advanced Pick and most R83 compatible tape formats with some restrictions on the devices (see VME Tapes).

An easy way to load a D3 database is to use a pseudo tape created for example, on a D3 UNIX implementation, and share this file across the network using a PC-NFS package.

Restore data in the new file system, using this syntax:

```
account-restore fsi: name{(o}
from: name on tape
```

-or-

```
account-restore name{(o}
from: name on tape
```

where *name* is the name of the destination account in the new D3 File System. Note that *name on tape* is required if it is different from the name of the new account.

The (o option must be used to overload the account if it exists. Note this *overloading* is not the same as *overwriting* the account. If an item exists both in an existing file and on tape, it is overwritten by the item on tape. If an item exists in an existing file but not on tape, it is left in the file.

By default, the files coming from the tape are restored on the server where the account was created. To force the restore to load important files on specific servers, first create the files on the servers using the D3 File Manager (see [Creating a table](#)).

If a restored file does not exist, it is created with a modulo equal to the original modulo and a frame size equal to 4 KB. To resize the file, either set the *resize* attribute before saving, or pre-create the account and file using the D3 File Manager to the desired modulo.

 **Note:** A full restore, using the `/restore` option in `D3Vme.exe`, also restores data to the FSI space. The `/fileload` option deletes the FSI space and recreates a blank file system before the full restore.

Related concepts

[Frame size](#)

[Saving accounts](#)

Saving accounts

The FSI is capable of producing an Advanced Pick or R83 compatible account save.

Complete the instructions below to save an FSI account:

1. Create a Q-pointer in the FSI MDS file:

```
account name
001 QS
002 fsi: account name
```

2. Do one of the following:

- Save an account in an Advanced Pick backward compatible format, using the (a option.
- Save an account in a R83 backward compatible format, using the (c option.

Without using any options, the account save is restorable on D3 Windows.

Related concepts

[Frame size](#)

[Restoring accounts](#)

Serial terminals

Asynchronous ASCII terminals connect directly to the VME and either built-in ports (COM1, COM2) or extension ports (COM1 to COM255 on Windows) are used.

The ports start automatically when VME is initialized by configuring the ports using the **D3 Connect** tab of the D3 Device Manager (see the *Administrators Guide*) or by using the **dev-make** TCL command (see the *D3 Reference Manual*).

The port number is associated to the PIB number that is using it.

Ports cannot be used by any other Windows application, even if they are not logged on.

When VME initializes, select <return> to display the logon banner.



Note:

- The **dev-remov**, **exit**, and **disconnect** TCL commands (see the *D3 Reference Manual*) free the terminal so it can be used by another Windows application. Use the **dev-make** command to reinitialize it.
- VME does not use the Windows TAPI interface to access the device. Therefore, it is not aware that a non-active application has reserved the device through TAPI.

Related concepts

[Accessing the VME](#)

[File system access](#)

[Telnet emulation](#)

[VME tapes](#)

[VME printers](#)

[Form queue](#)

[Print file](#)

[VME supported OSFI drivers](#)

Telnet emulation

The VME acts as a simplified Telnet server or client. Access the VME from any Telnet client or use the VME to call any Telnet server.

The VME server uses TCP port number 23 (the default Telnet port). If there is another Telnet server on the system, the port number can be changed using the D3 Connect tab in the D3 Device Manager (see the *D3 System Administration Guide* for more information).

The Telnet server works in three optional, nonexclusive modes:

Mode	Description
Standard Server	When an incoming call is accepted, a PIB is allocated dynamically to the new process from a range defined in using the Number of PIBs value in the PIB Configurations tab of the D3 Device Manager (default 1 to maximum number of PIBs -1). This mode of operation is

Mode	Description
	suitable for applications that do not depend on the PIB number they are running on. To disable the standard mode, set the Number of PIBs value to -1.
Nailed Server	A predefined, nonstandard TCP port number is associated, or <i>nailed</i>, to a given PIB. When a client requires a connection to a predetermined PIB, it must specify the TCP port number, from 1024 to 32767, during the connection. Most Telnet clients allow specifying an option. For example, UNIX Telnet clients allow the form <code>telnet host port</code> and the Windows Telnet client allows specifying a port in the Connect > Remote dialog box. To associate PIBs and Ports, see the dev-make TCL command or the <i>D3 Device Manager</i> section of the <i>D3 System Administration Guide</i> for more information.
Nailed Client	In this mode, the VME acts as a client to a Telnet server running on another system. A host name and TCP port are associated, or nailed, to a given PIB. When such a process is started, it establishes a Telnet connection to the specified server. This mechanism can be used on some terminal servers, allowing automatic connection of a serial terminal through a network (see dev-make TCL command).



Note:

- The Telnet server does not recognize urgent data, meaning that a Telnet client that sends such data when trying to send a Break does not work. To send a Break, specify a BreakChar entry in the D3 Settings tab of the D3 Device Manager (see the *D3 System Administration Guide* for more information).
- A Telnet client disconnects using the **exit** or **disconnect** TCL command by closing the Telnet session or by issuing from another port the **dev-remov** TCL command.

Related concepts

[Accessing the VME](#)

[File system access](#)

[Serial terminals](#)

[VME tapes](#)

[VME printers](#)

[Form queue](#)

[Print file](#)

[VME supported OSFI drivers](#)

VME tapes

The VME supports a set of devices that can be used as tapes to save and restore data from previous Pick implementations. The devices supported depend on the underlying host system.

Tapes must be defined in the registry (see VME Server Configuration in the *System Administration Guide*) before they can be used.

Once the VME server is started, the tape configuration can be changed with the **dev-make** and **dev-remov** TCL commands.

Up to 16 tape devices, numbered from 0 through 15, can be used simultaneously.

Note that when the VME is started, all tape devices configured in the registry are installed automatically. They show in the device list when **list-device** is executed. When D3 is up and running, the user can use **dev-make** to add more devices without having to reboot.

All systems support pseudo tapes. There is no (practical) limitation to the size of a pseudo tape. The name of a pseudo tape is a full explicit path name (for example, c:\tmp\backup) or partial name, in which case it is located in the directory specified in the registry by the Root value. If the host file does not exist, it is created the first time the tape is opened (read or write).

 **Note:** Once a tape is opened by the VME, the device is locked and cannot be accessed by any other application. The open takes place at the first access, not when the device is attached. In other words, doing a **set-device** does not lock the device. To lock the device, either read or write it or do a **t-rew**. The device gets unlocked when a **t-det** is issued.

The label size of the sequential device may need to be changed, depending where the tape is coming from. By default, the system will assume the label was written inside a 512-byte tape block. Some tape devices will require 50 bytes, 80 bytes, or 16 KB.

For Windows: Windows supports, in addition to pseudo tapes, up to two floppies (A and B) and any tape device that can be used from Windows:

- Floppy names are **\\.\A:** and **\\.\B:**
- Tape devices are named **\\.\TAPE0**, **\\.\TAPE1**, and so on

The exact nature of the device is fairly transparent to the VME. The device type code in the registry is for informational purposes only.

Related concepts

[Accessing the VME](#)

[File system access](#)

[Serial terminals](#)

[Telnet emulation](#)

[VME printers](#)

[Form queue](#)

[Print file](#)

[VME supported OSFI drivers](#)

VME printers

The VME supports printer devices through the underlying host system. There are three types of printer interfaces: Direct Printers, Shared Printers, and Dedicated Printers.

Direct Printers

To use direct printers, the printer interface must be defined in the registry (see *VME Server Configuration* in the *Administrators Guide*) before printers can be used. Once the VME Server is started, printers can be added or removed using **dev-make -t Printer** or **dev-remov** TCL commands.

The direct printer VME interface does not actually use printers. Printers are not started or stopped from D3. The spooling process sends the output directly to the host system using the printer associated with the assigned Form Queue. The only user or application interfaces are the [Form queue](#) and the [Print file](#). The maximum number of Form Queues and the maximum number of Print Files are defined in the registry.

When using this interface, the printing process is managed completely by the Windows Print Spooler. Therefore, most Spooler commands do not function as expected. Note, however, that the D3 Spooler must be started as usual to allow managing hold files and some other functions. There are no D3 printer processes to activate.

There are a few differences that must be noted:

- No Job entry number is displayed.
- Print jobs are going directly in the Windows Spooler and must be managed from the Windows Printer Manager. For example, the **sp-kill** command cannot kill a job being printed in Windows. The **listpeqs** command shows only job entries spooled to form queues other than the ones described in the registry. Jobs spooled to the form queues mapped to a Windows printer are never stored in the VME Spooler, even if the user used an (h option to the **sp-assign** command.
- Some functionality is not available (alignment, multiple copies) or must be used from the Windows Printer Manager.

Shared Printers

The output of the printer process is routed to a Windows printer.

Printers can be activated or removed using **dev-make -t NTPrinter** or **dev-remov** associated with **Startprinter** or **Stopprinter**.

Note that managing print jobs on a shared printer may require intervention in the Windows Print Manager. Because the output of the D3 printer process is routed to the Windows printer, printing from D3 appears almost instantaneous. Killing a job, for example, requires using the Windows Print Manager. The job name in the printer gives some information about the origin of the print job (see Print File Job ID).

Dedicated Printers

In this mode, a printer is connected to a serial port or a Telnet nailed client or server. The **startptr** command allows starting a printer on the device. The printer is managed completely by the VME.

The printer cannot be accessed by any other Windows application, including other VME on the network.



Note: This mode of operation should be restricted to very special usage, because it is not possible to share the printer device.

Related concepts

[Accessing the VME](#)
[File system access](#)
[Serial terminals](#)
[Telnet emulation](#)
[VME tapes](#)
[Form queue](#)
[Print file](#)
[VME supported OSFI drivers](#)

Form queue

When used as a direct printer, a form queue is assigned one-to-one with the associated printer device name (not the printer path) defined in the registry. For example, form queue 0 can be assigned to HP LaserJet 4L, which is the printer \\prod\\hp4. In this example, the registry would designate the printer as HP LaserJet 4L. The printer name can be found in the Windows Printer Manager.

The form queue number can be any number between 0 and 125.

The registry contains the maximum number of form queues for the entire VME. To avoid wasting memory, this number should be set to slightly larger than the number of form queues actually required, allowing some safety margin. For example, if using the form queues 0, 5, and 99, a value of 4 as the maximum number of form queues would be suitable (one would be reserved).

When not used as a direct printer registry, form queues retain their classic meaning.

Related concepts

[Accessing the VME](#)
[File system access](#)
[Serial terminals](#)
[Telnet emulation](#)
[VME tapes](#)
[VME printers](#)
[Print file](#)
[VME supported OSFI drivers](#)

Print file

The maximum number of print files simultaneously opened for each process as used (for example, by the BASIC print on statement) is defined in the registry. This number should be relatively small since memory is allocated for each print file whether it is opened or not. For example, setting this number to 32 would allow up to 32 print files to be opened simultaneously per process.

The table which describes the opened print files is allocated the first time a process prints.

Job ID

The D3 VME print file job ID in the Windows spooler has this syntax:

```
ProductName_ ComputerName_ VMENAME_ PibNumber_F{ FormQueue}
```

Parameters

Parameter	Description
<i>ProductName</i>	Product name (for example, D3).
<i>ComputerName</i>	Computer name (for example, ntdev).
<i>VMENAME</i>	VME machine name (for example, pick0).
<i>PibNumber</i>	PIB number of the D3 printer process or the process issuing the print command using direct printer.
<i>FormQueue</i>	Form queue associated with the direct printer (not available when using D3 Printer).

Example

```
D3_ntdev_pick0_3_F0
```

Related concepts

[Accessing the VME](#)

[File system access](#)

[Serial terminals](#)

[Telnet emulation](#)

[VME tapes](#)

[VME printers](#)

[Form queue](#)

[VME supported OSFI drivers](#)

VME supported OSFI drivers

In addition to the standard drivers, the VME supports the following OSFI drivers defined in the **dm,hosts,** file.

Driver	Description
OSFI Windows Driver	Access to the underlying host file system. This driver and some derivatives allow access to Windows.
OSFI Remote Pick Files	Access to remote D3 files through TCP/IP. This driver allows access to a remote D3 system. See the <i>D3 Reference Manual</i> for more information.
FSI Driver	Access to the distributed File System Interface.

OSFI Windows Driver

This driver allows access to the underlying Windows file system with the following conventions:

- D3 file is represented by a directory, and the directory must exist.
- D3 item is a Windows file.

- Dictionary of a D3 file is represented by a subdirectory **\$DICT**.

Examples of Windows driver names are:

Driver Name	Description
NT	Used for text records in which a line is terminated by a line feed. When copied into D3, the line feed is replaced by an attribute mark.
DOS	Used for text records in which a line is terminated by a carriage return / line feed. When copied into D3, the sequence carriage return / line feed is replaced by an attribute mark. Tabulation are also replaced by 4 spaces, and vice versa.
NT_BIN	Used for binary records or data which must not be translated in any way.  Warning: This driver should not be used to read items using the BASIC READ statement.

Examples of valid Windows OSFI file names are:

OSFI File Name	Directory	Description
NT	/temp	Directory /temp on the current drive.
DOS	D:/temp	Directory /temp on the drive D:.

 **Note:** The use of slash marks (/) instead of the usual MS-DOS backslash (\).

The options in the host item are identical to the options of the UNIX driver with the following addition:

Lx

where x is a one letter code representing an MS-DOS drive letter. This option eliminates the need to specify an MS-DOS letter.

The entries A, B, C, and D in the host file are shortcuts to the MS-DOS drives A:, B:, C:, and D:.. For example, it is possible to refer to the file A:/Docs, as a D3 file.

Related concepts

[Accessing the VME](#)

[File system access](#)

[Serial terminals](#)

[Telnet emulation](#)

[VME tapes](#)

[VME printers](#)

[Form queue](#)

[Print file](#)

File System Interface

The File System Interface (FSI) is a system to organize and track files and provide a parallel access to multiple disks.

 **Note:** FSI is a somewhat ambiguous term as it refers to both a file system and the interface to that file system. Be aware that the FSI file system is a standalone file system that does not require the D3 VME. The FSI file system does not provide access to VME files if the D3 VME is present.

Some of the differences between the Virtual Machine Environment (VME) and File System Interface (FSI) are:

Environment/System	Description
VME	• Set of resources obtained from Windows or UNIX at initialization that display the functions and characteristics of a D3 computer system, and are shared by several D3 processes. For example, a real machine (the Windows or UNIX-based operating system) emulates a virtual machine (the D3 computer system), which then has disk space, tape drives, printers, terminals, users, and the D3 file system and language. • Only one virtual machine can exist on any given single hardware platform. • Subsystem of D3 Windows that provides an ASCII mode that is backwards compatible with previous Rocket applications. • D3 Windows VME is required in order to start and run the FSI. It acts as the front-end to the FSI. • idn correlative in VME can start from 0 (for example, id0). • The <code>list-lockq</code> command is supported for VME locks.
FSI	• Server is a multithreaded process that can service several clients requests. • Process is more reliable because the files are distinct objects on the network, locks and disk space are handled independently of other files which limits risks. • Server also centralizes the record locks associated with the tables it manages. • idn correlative must start from 1 (for example, id1). id0 in the FSI is ignored.

Environment/System	Description
	The <code>list-lockq</code> command is not supported for FSI locks.

FSI on D3 Windows

Unlike on D3 UNIX platforms, the FSI and VME on D3 Windows can be installed either together or separately. It is recommended that you have both the VME and the FSI installed on your system.

The VME, which acts as the front-end to the FSI, is required to log on to the FSI. Additionally, the FSI Server is required, because each D3 file server must have an FSI process started to access the D3 tables.

 **Note:** All D3 ACCESS commands are supported.

Before you begin working with the FSI, verify that:

- Both the VME and FSI Server are installed on your system. See the *D3 Windows Installation Guide*.
- Both VME and FSI Servers are configured during installation.

For example, if you selected a Typical installation, verify that your D3 VME is started in Windows Services. See [Starting the FSI server](#).

 **Note:**

- If you selected the Typical D3 Windows installation, start the D3 Virtual Machine in Windows Services to start both the VME Server and the FSI Server.
- If you selected the Custom D3 Windows installation, start both the D3 Virtual Machine and the D3 FSI. The D3 VME must be started to log on to or use the FSI.

- All D3 Windows installation procedures for first time installation and upgrading are complete.
- D3 Windows is activated on your system.

 **Note:** For detailed installation instructions, see the *D3 Windows Installation Guide*.

Related concepts

[Using the FSI tools](#)

Configuring the FSI server

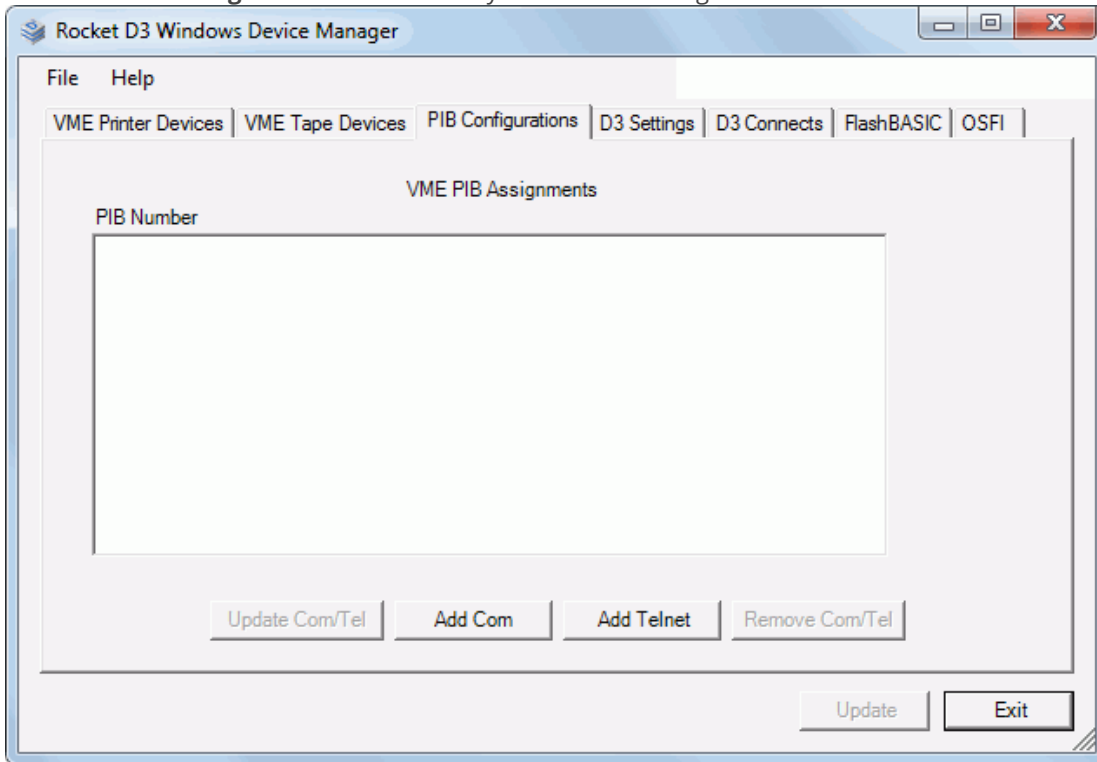
Follow these instructions to make changes to the default FSI Server configuration settings.

 **Note:** If no modifications need to be made, proceed to the topic [Starting the FSI server](#).

1. Open the **D3 Device Manager** from the **D3 Program** menu.

The Device Manager displays.

2. Select the **PIB Configurations** tab to modify the default settings.



3. Click **Add Com** to add the new modifications, then **Exit**.

Related concepts

[Users file](#)

Related tasks

[Starting the FSI server](#)

[Stopping the FSI server](#)

[Logging on to the FSI](#)

Starting the FSI server

When installing D3 Windows for the first or upgrading, you are asked to select between two types of installations: Typical or Custom.

Type	Description
Typical	Installs the VME and FSI Server. When installed, the D3 VME is set to start manually or automatically when Windows is restarted.  Note: Typical installation installs both the VME and FSI and is recommended if using the FSI Server.

Type	Description
Custom	Installs either the VME Server or FSI Server. If the FSI Server is installed, the D3 FSI is set to start manually or automatically when Windows is restarted.

Depending on the type of installation (Typical or Custom), the D3 VME and/or D3 FSI is either started automatically or manually when Windows is restarted.

For Typical and Custom installations, follow these instructions to start the FSI Server.

1. Go to **Windows Services**.

The Services dialog box displays.

2. Do one of the following:

- For Typical installation (recommended):

Select **D3 VME**, then click the **Start Service** icon. Both the VME and FSI Servers are now started.

- For Custom installation:

- a. Select **D3 FSI**, then click **Start Service** icon.

This starts the D3 FSI Server.

- b. Select **D3 VME**, then click the **Start Service** icon.

This starts the D3 VME Server.

3. (Optional) Change **Startup Type** options from the Service dialog box as displayed below. See *D3 Windows Installation Guide*.

Related concepts

[Users file](#)

Related tasks

[Configuring the FSI server](#)

[Stopping the FSI server](#)

[Logging on to the FSI](#)

Stopping the FSI server

For Typical and Custom installations, follow these instructions to stop the FSI Server.

1. Go to **Windows Services**.

2. Do one of the following:

- For Typical installation (recommended):

- Select **D3 VME**, then click the **Stop Service** icon. This stops the VME and FSI Servers.

- For Custom installation:

- a. Select **D3 FSI**, then click the **Stop Service** icon.

This stops the D3 FSI Server.

- b. Select **D3 Virtual Machine**, then click the **Stop Service** icon.

This stops the D3 Virtual Machine.

Related concepts

[Users file](#)

Related tasks

[Configuring the FSI server](#)

[Starting the FSI server](#)

[Logging on to the FSI](#)

Logging on to the FSI

Before logging on to the FSI dm account, you must first log on to the D3 dm account.

1. Start a telnet session.
2. Select **Start > D3 > telnet localhost** from the program menu.

A telnet localhost session displays.

3. Press any key.

This prompt displays:

```
Enter your user id:
```

4. Type **dm** to log on to the dm account.

This prompt displays:

```
Master dictionary:
```

5. Type **dm** to log on to the Master Dictionary.
6. Type **to fsi:dm** to log on to the FSI dm account.

```
master dictionary: dm
                    3
<<< 09:50:04  --      D      --   13 Nov 2009
```

```
...  
  
05 Aug 2009 nt  
:to fsi:dm  
  
< Connect time= 0 Mins.; CPU= 0 Units; LPTR pages= 0      >  
:
```

You are now logged on to the FSI.

Related concepts

[Users file](#)

Related tasks

[Configuring the FSI server](#)

[Starting the FSI server](#)

[Stopping the FSI server](#)

Users file

The users file must be a Q-pointer to the **FSI:dm,users** file to be able to use Visual Basic or compile from the D3 File Manager. This is required because these tools update the users file automatically to create a user with a name identical to the Windows user name.

The `qs.upd` utility, which is run automatically when the VME is booted, transfers the users file to the FSI (unless it is a Q-pointer to another file, specifically set by the System Administrator). Note that the original users file is renamed to **users.<datestamp>**, where *datestamp* is the actual date.

To disable this mechanism, rename the file, for example, users to **users.vme**, then set the users item to be a Q-pointer to **dm,users.vme**. This prevents Visual Basic from logging on to the VME.

The system also creates internal users as follows:

- *name*\$x\$

where *x* is a decimal number. This user-ID is used by triggers when doing an execute for example. *name* is the user-ID as found in the users file.

- *name_*

This user-ID is used by client applications, like the file manager or Visual Basic, when logging on to the VME. *name* is the Windows user name the application is currently logged on as.

The password to these users is generated automatically and can be different from session to session.

Related tasks

[Configuring the FSI server](#)

[Starting the FSI server](#)

[Stopping the FSI server](#)

[Logging on to the FSI](#)

Using the FSI tools

The FSI allows you to define files to resize automatically or manually.

Related concepts

[FSI on D3 Windows](#)

Using automatic file resizing

This topic describes the FSI native automatic background file resizing process.

The main functions are:

- User-specified fill ratio that triggers the file resizing.
- The ratio is global for the whole domain.
- Exclusion list of accounts and/or files on which resizing is never performed.
- Scheduled background process resizing files.
- Controlled by items in a resizing control file.
- Activity logging.

The file resize operates only on the files residing on the current FSI Server. There is one resizer per FSI Server.

Follow these steps to resize files automatically using the Task Scheduler or manually using the Resize Menu.

1. Restore the D3 accounts and complete a full save to tape to the dm account. Type:

```
to dm
file-save
```

2. Define the file to be resized in the FSI dm account (see [Resizing control files](#)).

For example, type:

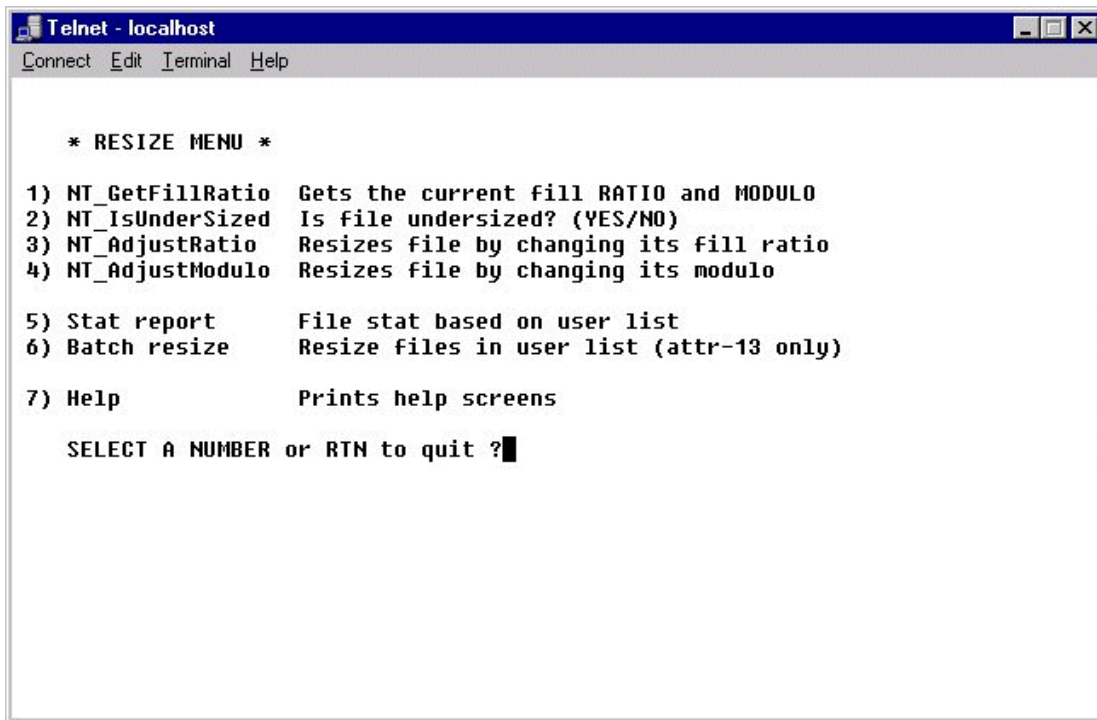
```
u resize include account, file, file
```

3. Type `nt_resize-menu` , then press ENTER.

The message **Press any key to Continue** displays.

4. Press any key to continue.

The Resize Menu displays.



5. Select one of the Resize Menu options to manually resize your files.

See the `nt_resize-menu` command in the *D3 Reference Manual* for more information on these options.

Related concepts

[Resizing control files](#)

[Setting inactivity timeouts on a remote server connection](#)

Resizing control files

The file resize in the FSI dm account controls the automatic resize process. The dictionary contains these items:

Item	Description	Format
Fill	Describes the target fill ratio	001 Target fill ratio (in %), where % is the space allocated for the files. To create a fill ratio, type <code>u dict resize fill</code> .
Include	List of accounts/files to include	Multi-attribute list of files to include in the resizing process. One file per attribute coded as follows: • Account,, Entire account • Account,file,

Item	Description	Format
		Dictionary and data levels Account,file,file Data level only To list the files to include, type: <pre>u dict resize include</pre> Note: Lines starting with an asterisk are comments and are ignored.
Exclude		

If the Exclude list is empty, resize only the files in the Include list. If the Exclude list is not empty, resize only files not in the Exclude list. If both lists are empty, resize *all* files.

The data level `resize,resize` contains the currently active or queued records of the resizing. Each item has this format:

Attribute	Description	
0	<i>file.no '*' unique.ID</i>	
	where	
	<i>file.no</i>	File number of the file being resized
	<i>unique.ID</i>	Unique-ID (system19)
1	State includes: - R)unning - C)omplete - E)rror - Q)ueued - S)uspended	
2	Server	

Attribute	Description
3	Account
4	Dictionary (field is blank if resizing the MD of the account)
5	Data level (field is blank if resizing the MD of the account or the dictionary of the file)
6	Old modulo in decimal
7	New modulo in decimal
8	Start date in internal format
9	Start time in internal format
10	Stop or suspended date in internal format
11	Stop or suspended time in internal format
12	Error/exception, if an error record
13	Optional information

The data level **resize,log** contains the records of the completed resizing operation. The format is identical.

Related concepts

[Setting inactivity timeouts on a remote server connection](#)

Related tasks

[Using automatic file resizing](#)

Setting inactivity timeouts on a remote server connection

The D3 Windows File System Interface allows setting inactivity timeouts on remote server connections.

When a timeout is set, the client disconnects after being inactive for the designated period of time. Timeouts are set from the D3 Connect tab of the D3 Device Manager (see the *D3 System Administration Guide* for more information). To suppress the timeout feature, deselect the RPC Inactivity Timeout check box.



Note:

- The disconnection caused by the timeout is transparent to the application. On the client side, the files remain opened. If the file is used after the timeout, the connection is automatically re-established.
- When the timeout expires, the file on the remote server is actually closed. This invokes the close trigger if it exists and aborts any pending transactions (on systems that support transactions). Furthermore, if a **SELECT** is in process on a file when a timeout occurs, the **SELECT** restarts at the beginning of the file when re-invoked. Therefore, we suggest that you don't use timeouts in situations where transactions or **SELECT** operations are occurring over extended periods of time.



- If your connection is over TCP/IP, the **NETSTAT** Windows command displays the state as **TIME_WAIT** for a few minutes after the timeout has expired. This is incidental and should not cause any problems.

Related concepts

[Resizing control files](#)

Related tasks

[Using automatic file resizing](#)

D3 Windows utilities

The D3 File Manager is a Windows application that enables you to manage a D3 distributed database. It displays a tree representation of the databases and the tables each database contains.

The File Manager applies output conversion to values in the File Manager and supports empty item-ID's.

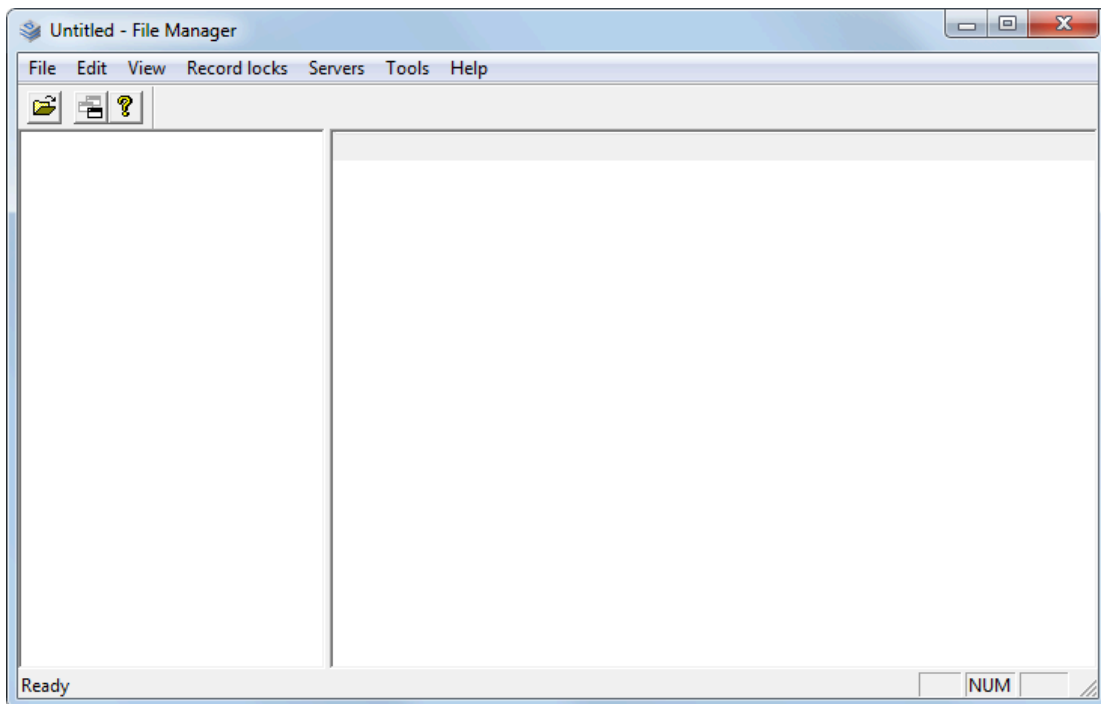
The Performance Monitoring Tools describe the tools and guidelines for evaluating the performance of a system while it is running.

D3 File Manager

The D3 File Manager is a Windows application that enables you to manage a D3 distributed database. It displays a tree representation of the databases and the tables each database contains.

To open the File Manager, select **All > Programs > Rocket D3 > File Manager** from the Windows **Start** menu.

The File Manager displays.



Related concepts

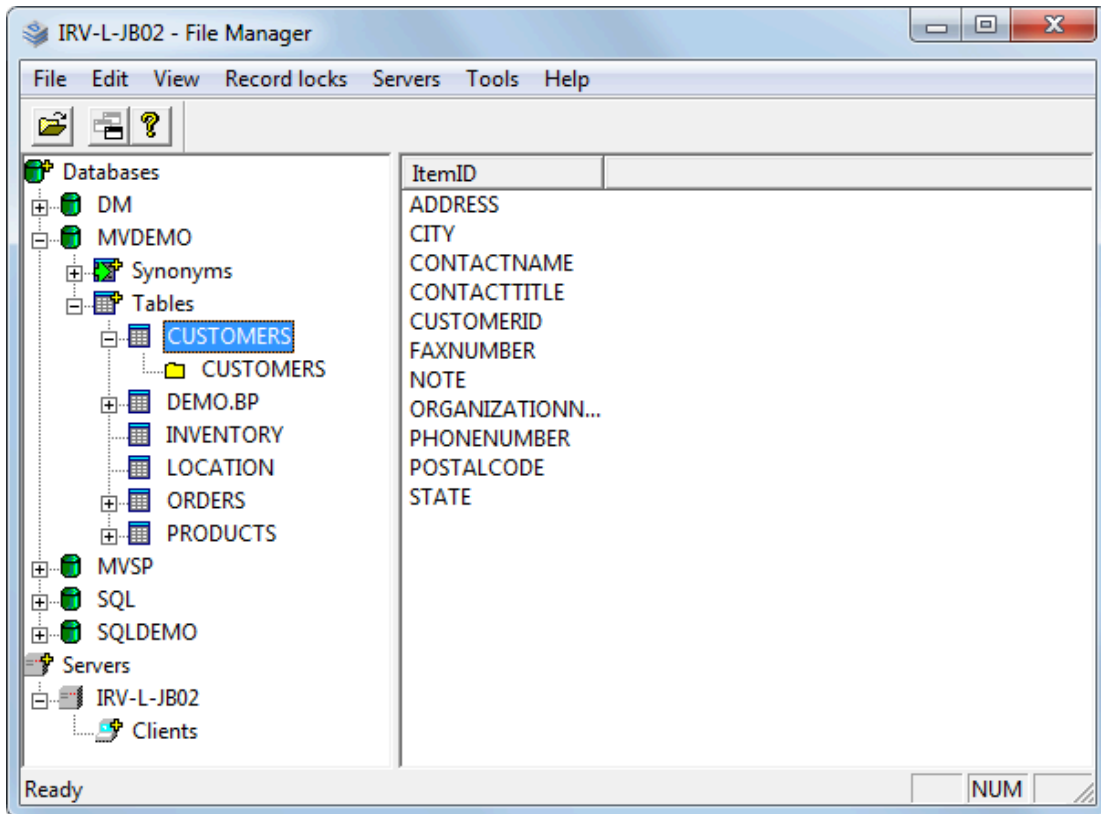
[Performance monitoring tools](#)

File Manager layout

The File Manager application comprises the Menu Bar, Tool Bar and two panels: the Domain Explorer and the Workspace.

- The panel on the left is the **Domain Explorer** which displays all of the databases, tables and servers defined on a specified domain.

- The panel on the right is the **Workspace** which displays various information pertinent to the options selected from the **Menu Bar** or the **Domain Explorer**.



The table below describes the features available from the File Manager's **Menu Bar**, **Tool Bar**, **Domain Explorer** and **Workspace**.

Feature	Description	
Menu Bar	The Menu Bar includes the following menus:	
	File	Operations for opening the default Domain, and creating, renaming and deleting Databases. See Domains and Databases for more information.
	Edit	Operations for editing Records and D-Pointers as well as standard Cut, Copy, Paste, Undo and Redo functionality. See Editing a record and Editing a D-pointer for more information.
	View	Operations for Refreshing the current display, displaying and hiding the Tool Bar and Status Bar , and opening the File Manger - Options dialog box which contains some general File Manager options. See Editing File Manager options for more information.
	Record Locks	Operations for displaying and clearing record locks. See Listing or clearing record locks for more information.
	Servers	Operations for displaying the list of available servers and changing the MDS server name. See Listing servers and Selecting the MDS server name for more information.
	Tools	Operations for importing records and setting the Flash Shared Code memory usage. See Importing records and Displaying Flash shared code for more information.

Feature	Description	
	Help	Operations for displaying the File Manager help documentation and the current D3 version.
Tool Bar	The Tool Bar includes the following icons:	
		Open the default domain in the Domain Explorer .
		Opens additional instances of the File Manager application. Additional windows are useful when using the drag-and-drop method to copy or move data, especially when it is difficult to have both the source and the destination in view.
		Displays the File Manager help documentation.
Domain Explorer	The Domain Explorer displays a tree-view of the databases, tables and synonyms of the current domain as well as the servers and clients associated to the current domain. Right-clicking nodes in the databases, tables and synonyms tree-view displays the various operation that can be performed on that database, table or synonym.	
		Root database node which holds all of the databases on the current domain. Right-clicking this node displays operations for creating a new database and editing the D-pointer. See Creating a database and Editing a D-pointer for more information.
		Database node which defines a single database in the current domain, including its Dictionary tables, Data Level tables and Synonyms. Right-clicking this node displays operations for renaming and deleting the database. See Renaming a database and Deleting a database for more information.
		Root table node which holds all of the tables defined on the current database. Right-clicking this node displays operations for displaying the table's Master Dictionary and properties, counting the records in the table, creating a new table, checking table overflow, and editing the table's D-pointer and records. See Listing the contents of a table , Counting a table , Creating a table , Editing a record , Editing a D-pointer , Checking overflow and Editing General table properties for more information.
		Table node which holds all of the tables defined on the current table. Right-clicking this node displays operations for displaying the table's Dictionary and properties, counting the records in the table, copying, renaming and deleting the table, creating a new Data Level for the table, clearing the table's Dictionary, and editing the table's D-pointer and records. See Listing the contents of a table , Counting a table , Copying a table , Renaming a table , Deleting a table , Creating a new data level for a table , Clearing a table , Editing a record , Editing a D-pointer , and Editing General table properties for more information.
		Data Level node which holds all of the data defined on the current table. Right-clicking this node displays operations for displaying the Data Level's data and properties, counting the records in the Data Level, copying, renaming, deleting and clearing the Data Level, managing indexes, and editing records in the Data Level. See Listing the contents of a table , Counting a table , Copying a table , Renaming a table , Deleting a table , Clearing a table , Creating indexes , Editing a record , and Editing General table properties for more information.
		Root synonym node which holds all of the table synonyms (Q-pointers) on the current database. Right-clicking this node displays the operation for editing (creating) a new synonym. See Creating and editing synonyms for more information.
		Synonym node which defines a single table synonym (Q-pointer) on the current

Feature	Description
	database. Right-clicking this node displays the operation for editing the synonym. See Creating and editing synonyms for more information.
	Root server node which displays the server on which the current domain is defined and all of the clients associated to that server. Clicking this node displays server licensing information in the Workspace .
	Server node which defines the MDS server on which the current domain is defined.
	Root client node which displays all of the clients associated to the server. Clicking this node displays a listing of clients and Client-Id's in the Workspace .
	Client node which defines one of the clients associated to the server.
	Client-Id node which displays an active threads on a client.
Workspace	Displays various information depending upon which operation is selected from the Menu Bar or Domain Explorer .

Related concepts

[Domains](#)
[Databases](#)
[Tables](#)
[Records](#)
[Additional File Manger operations](#)
[Environment variables](#)

Related tasks

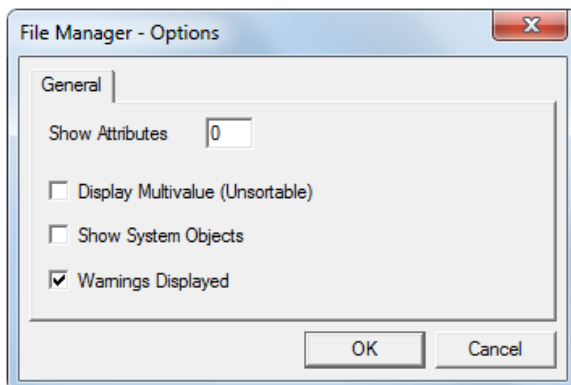
[Editing File Manager options](#)

Editing File Manager options

The File Manger - Options dialog box enables you to configure various general File Manager parameters.

1. Select **Options** from the **View** menu.

The File Manger - Options dialog box displays.



Below are the available **File Manager** options:

Option	Description
Show Attributes	Number of attributes or fields to display when listing a table. 0 lists only the record-IDs. The maximum value is 128.
Display MultiValue (Unsortable)	Displays output in MultiValue format. Each value in the attribute displays in a separate row.
Show System Objects	
Warnings Displayed	Select this box to require user confirmation when an operation is cleared or deleted.

2. Make your changes and click the **OK** button.

Your specified changes are saved.

Related concepts

[File Manager layout](#)

[Domains](#)

[Databases](#)

[Tables](#)

[Records](#)

[Additional File Manger operations](#)

[Environment variables](#)

Domains

A Windows concept that defines a set of systems, clients, and servers that are administered as a unit. A domain controller is designated to handle the security.

Related concepts

[File Manager layout](#)

[Databases](#)

[Tables](#)

[Records](#)

[Additional File Manger operations](#)

[Environment variables](#)

Related tasks

[Editing File Manager options](#)

Opening a domain

Before accessing databases, the domain must be opened.

By default, the domain named MDS is opened. If the MDS server cannot be opened, a message displays indicating that the specified server name is invalid or not accessible.

To select an alternate domain, select **MDS Server Name** from the **Servers** menu and follow the instructions in [Selecting the MDS server name](#).

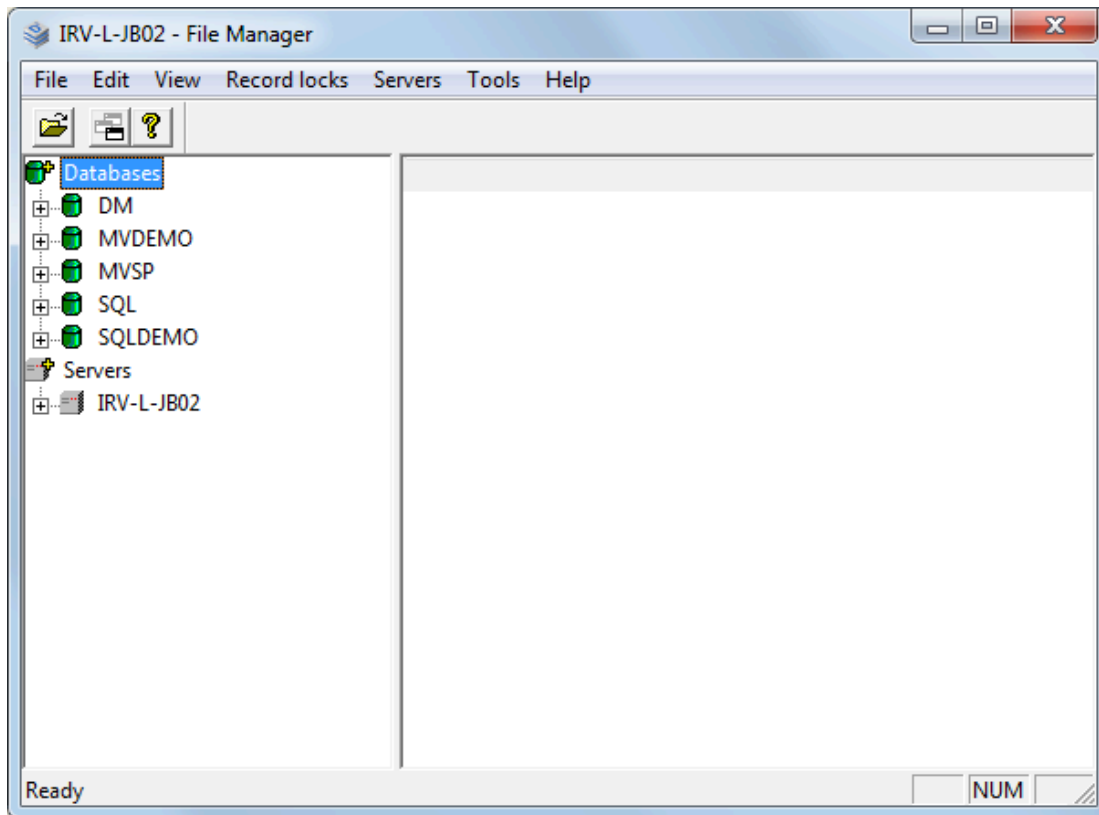
When a domain is opened, two main nodes display in the **Domain Explorer**.

- The **Databases** node includes all of the databases defined on the domain.
- The **Servers** node includes all of the Servers and Clients associated to the domain.

Do one of the following:

- Select **Open Domain** from the **File** menu.
- Click the **Open Folder** button  from the **Tool Bar**.

The domain displays in the **Domain Explorer**.



Related tasks

[Selecting the MDS server name](#)

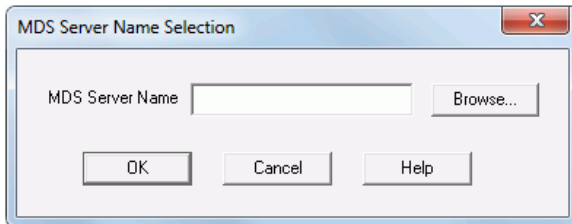
Selecting the MDS server name

By default, the File Manager uses the MDS server name called MDS, which holds the default file system. If there is an alternative D3 file system on the network, or if you want to reach a file system in another domain, the MDS Server Name must be changed.

The current MDS Server name is displayed in the title bar of the File Manager window.

1. Select **MDS Server Name** from the **Servers** menu.

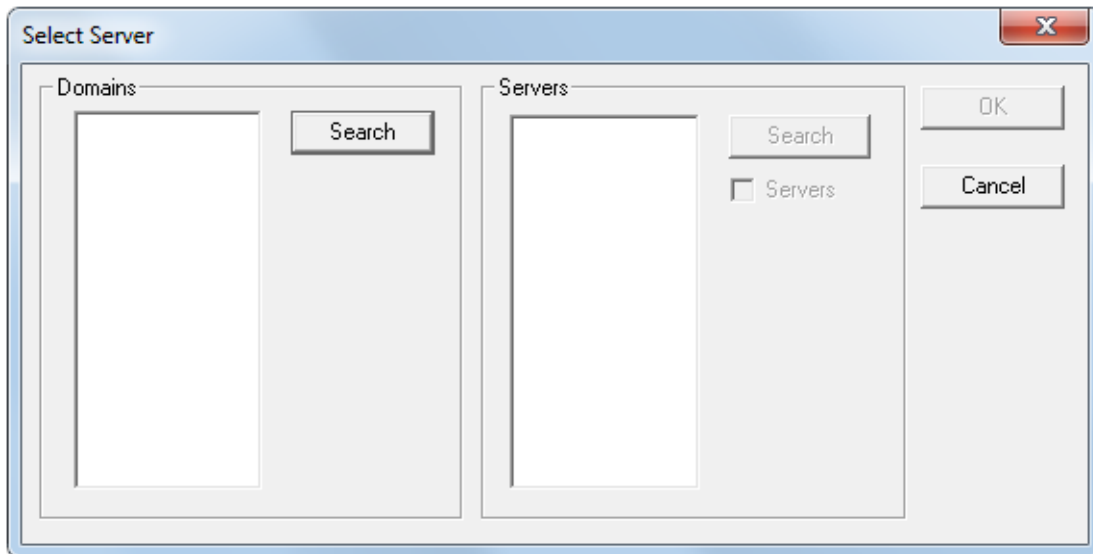
The MDS Server Name Selection dialog box displays.



2. Click **Browse** to search for all available servers on the network.

Tip: You can also type the name you assigned to the MDS Server in text box, then click **OK**. Note that this is not the network name of the computer where the root of the file system resides. It is the name displayed in the **MDS Server Name** text box of the **D3 Settings** tab (in the D3 Device Manager).

The Select Server dialog box displays.



3. Click the **Search** button from the **Domains** section and select the appropriate domain from the list of domains that displays.
4. Click the **Search** button from the **Server** section and select the appropriate server from the list of servers that displays. Select the Servers check box to display only those servers where D3 is installed and currently active.
5. Click **OK** and then click **OK** again.

The MDS Server Name is selected.

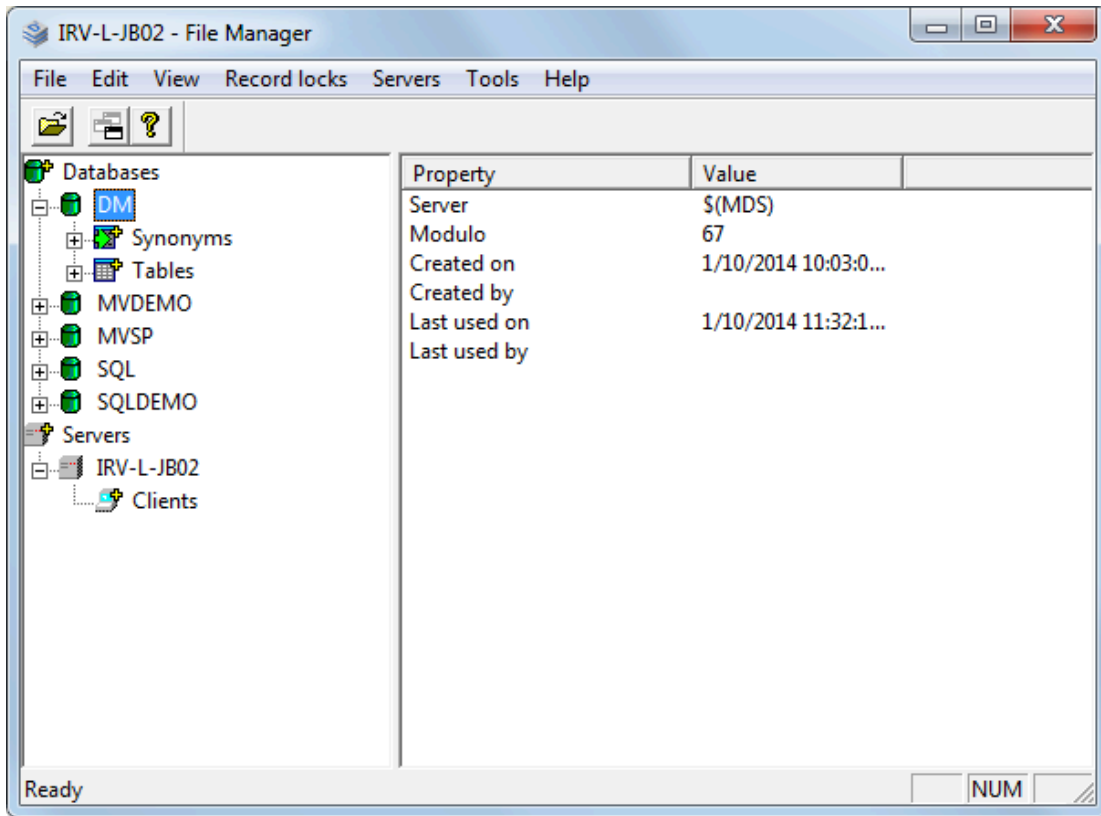
Related tasks

[Opening a domain](#)

Databases

The topics in this section describe the utilities available for opening, creating, renaming and deleting databases. Additionally, the utilities for creating and editing synonyms are described.

Selecting a database node displays information about the database in the File Manager **Workspace**, including the server on which it resides, the modulo for the database's Master Dictionary and database creation and use information.



Under the database node, these nodes display:

Node	Description
Synonyms	This expandable node contains a list of all the table synonyms (Q-pointers) in the database. Click this node to list all the synonyms.
Tables	This expandable node contains a list of all the tables in the database. Double-click this node to list all the tables.

Related concepts

- [File Manager layout](#)
- [Domains](#)
- [Tables](#)
- [Records](#)
- [Additional File Manger operations](#)
- [Environment variables](#)

Related tasks

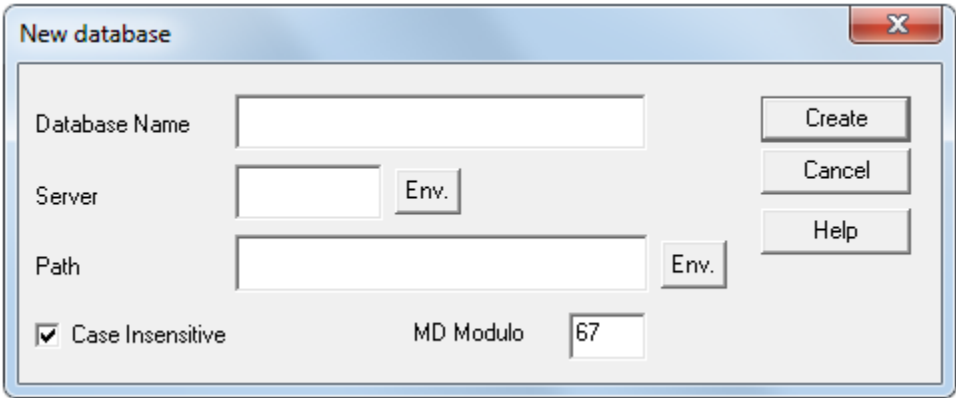
- [Editing File Manager options](#)

Creating a database

The **New Database** option enables you to create a new database in the D3 environment on the network. Complete these instructions to create a new database.

- 1. Select **New Database** from the **File** menu or right-click the **Databases** node from the **Domain Explorer** and select the **New Database** option.

The New database dialog box displays.



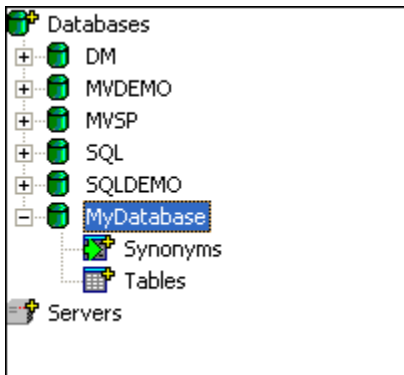
- 2. Specify the configuration for the new database as described below:

Value	Description
Database Name	Specifies the name for the new database. Note: The maximum database name length is 50 characters and it is case insensitive.
Server	Specifies the server that will store the database. Either type the server name or click the Env. button to select the server from a list of available servers. This server will hold the Master Dictionary and, optionally, tables in the account. Tables can then be created (See Creating a table). If not specified, the database is created on the MDS server. The server name can be either the network name of the computer, an alias defined in the registry of this server, or a Server environment variables. Note: The D3 File System Interface service must be started on the specified server.
Path	(Optional) Specifies the path where the database supporting host files is created. Either type the path or click the Env. button to select the server from a list of available paths. Note: It is usually better to leave this field blank and let the system locate the file. The registry on the specified server contains the name of the root of the

Value	Description
	 D3 file system, and all files are located relative to the path. If specified, the name is any valid host file directory. If it does not exist, it is created automatically. The path can be any valid host file system path, or a Path environment variables.
Case Insensitive	(Optional) Specifies that any Dictionary file name that is created in the database will be case insensitive.
MD Modulo	Specifies the modulo of the Master Dictionary. The default should be sufficient for most cases. If the account will contain many tables, the modulo should be increased. Any prime number (not a multiple of 2 or 5) can be selected.

3. Click **Create**.

The new database is created and displays under the **Databases** node in the **Domain Explorer**.



 **Note:** To be able to use a database from the VME, it is easier to create remote Q-pointers to the FSI database. Such a Q-pointer is created in the MDS file of the VME, and is formatted as follows:

```
mds database
001 QS
002 fsi:database
```

With the Q-pointer created, users can now log to this account. Make sure the Master Dictionary has been updated (use the TCL command `update-md database`).

Related tasks

[Deleting a database](#)

[Renaming a database](#)

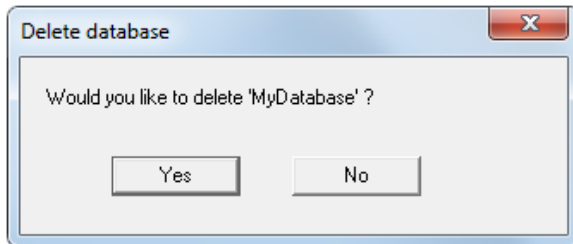
[Creating and editing synonyms](#)

Deleting a database

Deleting a database deletes all the tables in the database.

1. Select the database to delete from the **Domain Explorer** and then select **Delete Database** from the **File** Menu. You can also right-click the database to delete from the **Domain Explorer** and then select the **Delete Database** option.

If the **Warnings Displayed** option is enabled from the File Manager - Options dialog box, the Delete database dialog box displays and confirmation is required. See [Editing File Manager options](#) for more information.



2. Click **Yes**.

The specified database is deleted and is removed from the **Databases** node in the **Domain Explorer**.

Related tasks

[Creating a database](#)

[Renaming a database](#)

[Creating and editing synonyms](#)

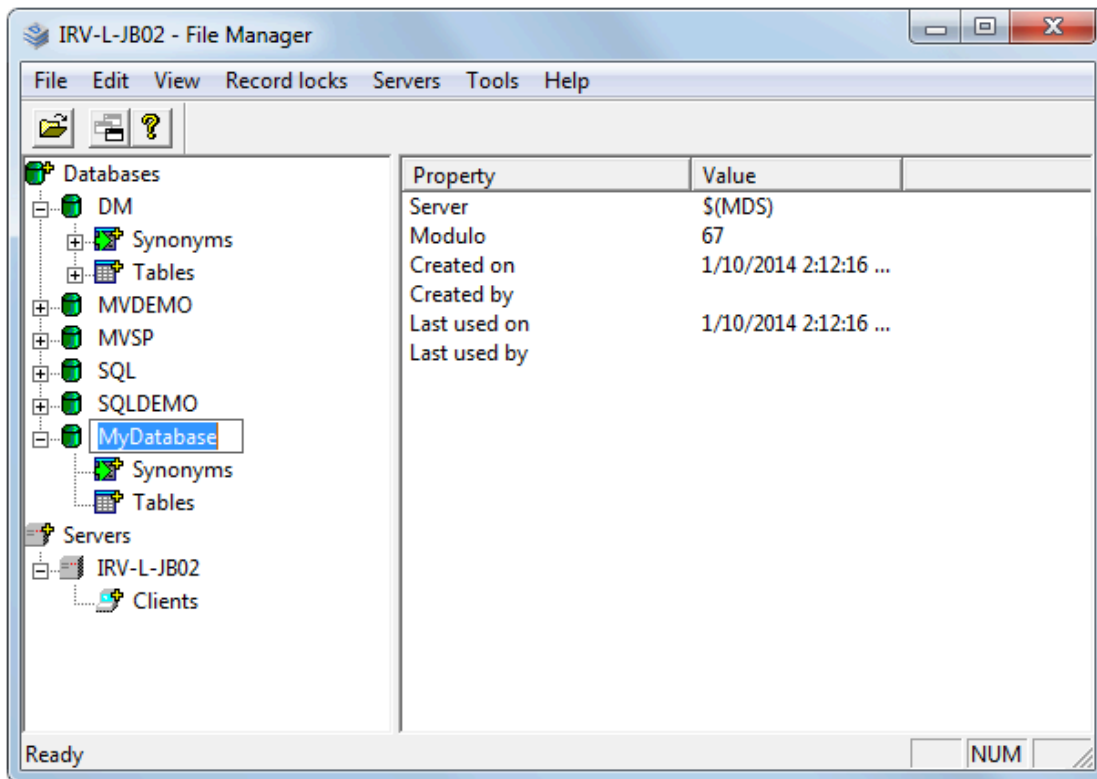
Renaming a database

You can rename any database in your domain.

Complete these instructions to rename a database.

1. Select the database to rename from the **Domain Explorer** and then select **Rename Database** from the **File** Menu. You can also right-click the database to rename from the **Domain Explorer** and select the **Rename Database** option.

The name for the specified database is highlighted in the **Domain Explorer**.



2. Type the new database name and then press **ENTER** .

The specified database is renamed.

Related tasks

[Creating a database](#)

[Deleting a database](#)

[Creating and editing synonyms](#)

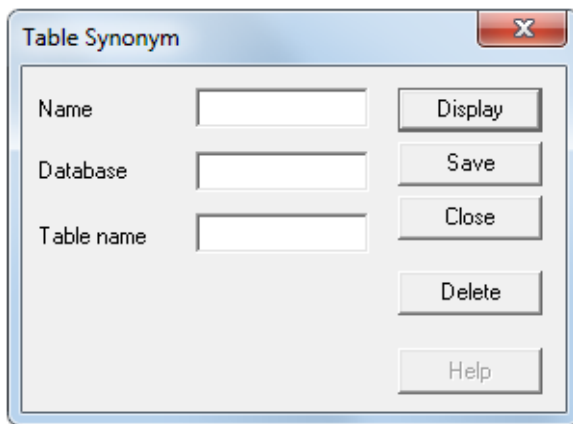
Creating and editing synonyms

Synonyms (or Q-pointers) are used in account Master Dictionaries to point to other files.

Complete the instructions below to create a synonym.

1. Right-click the **Synonyms** node in which you want to create a new synonym from the **Domain Explorer** and select the **Edit Synonym** option.

The Table Synonym dialog box displays.



2. Provide the following values:

Value	Description
Name	Synonym name.
Database	Name of the Master Dictionary (account) where the destination table resides. If the Database name is not specified, the current account name is assumed.
Table name	Name of the destination table to which this item points. If the table name is not specified, it is assumed that it has the same name as the Database name.

3. Click **Save**.

The synonym is created and displays under the **Synonyms** node in the **Domain Explorer**.

- Tip:** To edit an existing synonym, right-click the synonym to edit from the **Domain Explorer** and select the **Edit Synonym** option.
 - To edit the synonym, change the appropriate values set in the **Name**, **Database** and **Table name** text boxes and click **Save**.
 - To delete the synonym, click the **Delete** button. Upon deletion, the synonym is removed from the **Domain Explorer**.

Related tasks

- [Creating a database](#)
- [Deleting a database](#)
- [Renaming a database](#)

Tables

This section describes the various operations that can be performed on Dictionary and Data Level tables.

Related concepts

- [File Manager layout](#)
- [Domains](#)

- [Databases](#)
- [Records](#)
- [Additional File Manger operations](#)
- [Environment variables](#)

Related tasks

[Editing File Manager options](#)

Creating a table

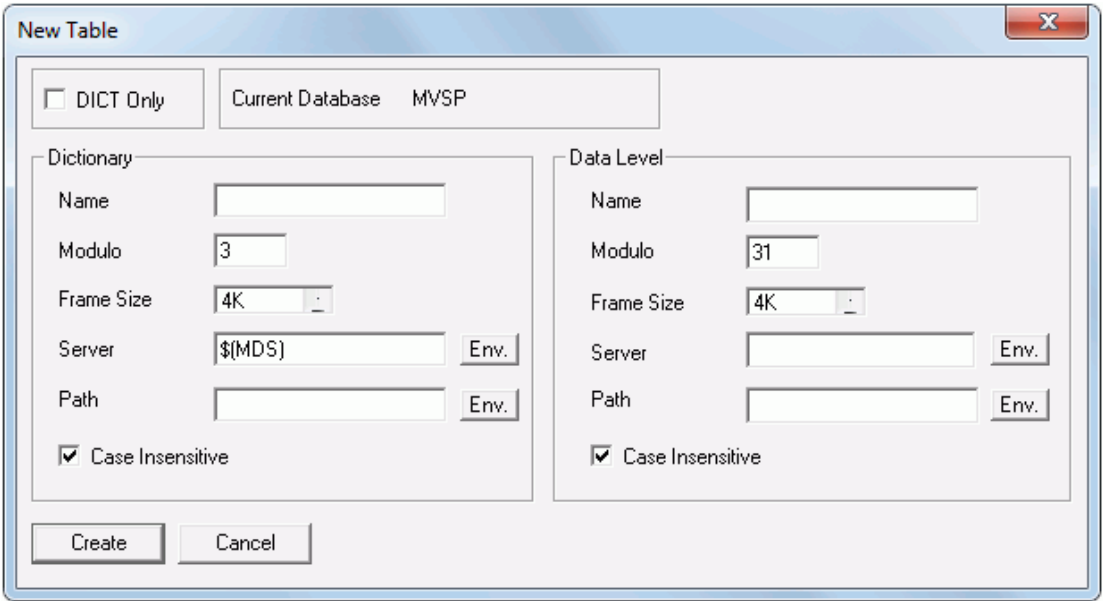
Use the **New Table** option to create a new Dictionary table (and optionally a new Data Level table) in a database on the D3 server.

Complete the instructions below to create a new table.

To create a new Data Level on an already existing Dictionary table, see [Creating a new data level for a table](#).

1. Right-click the **Tables** node from the **Domain Explorer** and select the **New Table** option.

The New Table dialog box displays.

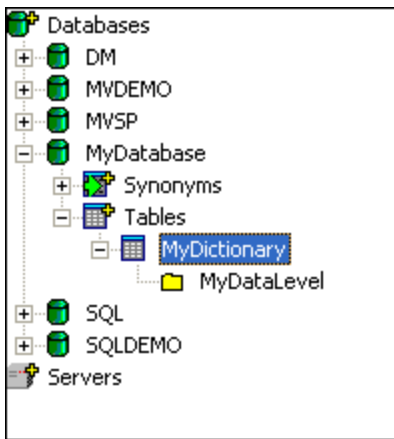


2. Complete these fields:

Value	Description
DICT Only	◦ When checked, the Data Level section of the dialog is disabled. Only the Dictionary table will be created. ◦ When unchecked, the Data Level section of the dialog is enabled, allowing you to define both a new Dictionary table and a new Data Level table. The new Data Level table will be listed as a sub-item of the new Dictionary table in the Domain Explorer. Note that the parameter descriptions below apply to both Dictionary and Data Level tables.

Value	Description
Name	Table Name. Any string of up to 50 characters, excluding the colon (:) character. If the database was created using the case insensitive option, the Dictionary file name is case insensitive. If the Dictionary is case insensitive, the Data Level name is case insensitive, whether the database is case sensitive or not.
Modulo	Enter the table modulo. The modulo should be selected in conjunction with the frame size so that the items are properly distributed across the table, without excessive group overflow.
Frame Size	Select a frame size. By default, both the Dictionary and Data Level frame sizes are 4 KB. Small frame sizes are suited to files that contain a few small items.
Server	Network name of the server where the table will reside. By default, the dialog will create the table on the server where the database resides, but there is no requirement. The Dictionary and the Data Levels do not need to be located on the same systems. Selecting the server is essentially a question of disk space, performance, and proximity to the users. The server name can be either the network name of the computer, an alias defined in the registry of this server, or a server environment variable. The list of Server Environment Variables is displayed by clicking the Env. button. The D3 File System Interface service must be started on this server. See Server environment variables for more information.
Path	Full path name where the table is to reside. Use forward slashes (/) to separate the directory names. If the path starts with a / or a drive letter (for example, C:/mypath), the path is absolute and the table will be created on that location. If the path does not start with a /, the path is relative and is added at the end of the root directory specified in the server configuration used to create the table. When using a drive, it is important to specify an absolute path (start with a /), otherwise the directory will be appended to whatever current directory the drive is set to. Note that UNC names (\\server\path) are not supported. The Path can be any valid host file system path, or a path environment variable. The list of Path Environment Variables is displayed by clicking the Env. button. See Path environment variables for more information.  Note: It is advised to leave the path empty and to let the system create the table under the file system server root.
Case Insensitive	Specifies that all items in the table, including table names for a Dictionary, will be case insensitive.

- The new table is created and displays under the specified **Tables** node in the **Domain Explorer**.



Related concepts

[Table correlatives](#)

Related tasks

[Listing the contents of a table](#)

[Copying a table](#)

[Counting a table](#)

[Deleting a table](#)

[Creating a new data level for a table](#)

[Clearing a table](#)

[Renaming a table](#)

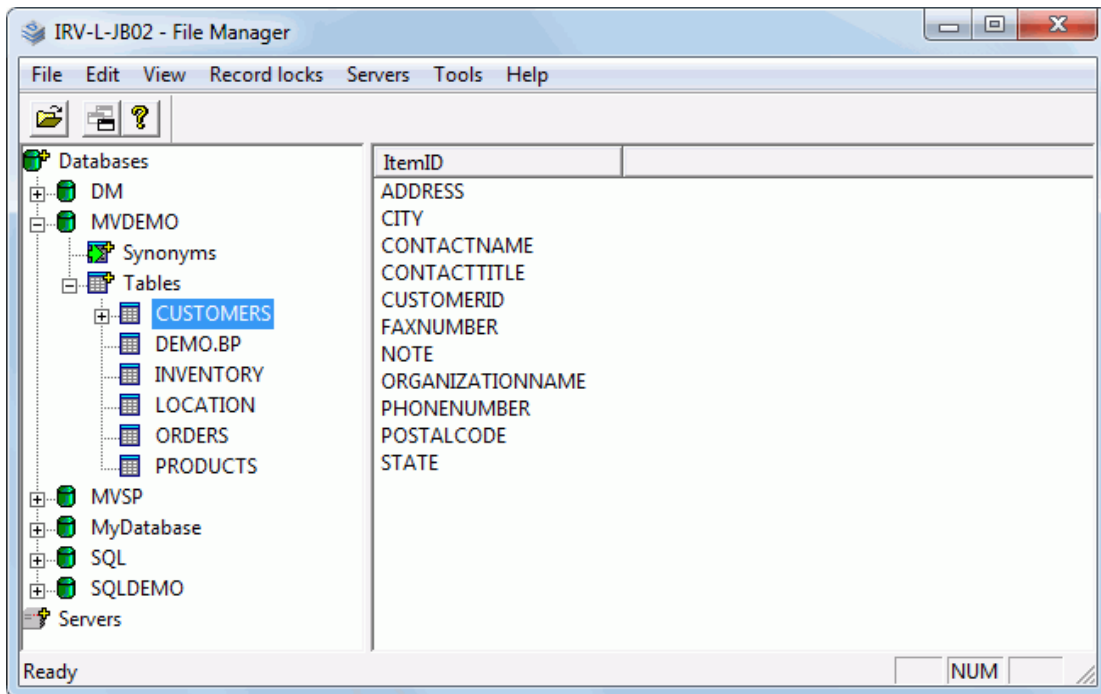
[Moving a table](#)

Listing the contents of a table

Complete the instructions below to list the contents of a table.

Right-click the table node associated to the table whose data you want to display from the **Domain Explorer** and select the **Show Data** option. Note that you can also double-click a table node to display the data.

The data for the specified table displays in the **Workspace**.



By default, only the record-IDs (the keys) are displayed. To show the content of the records, change the **Show Attributes** option from the File Manager - Options dialog. See [Editing File Manager options](#) for more information.

Related concepts

[Table correlatives](#)

Related tasks

[Creating a table](#)

[Copying a table](#)

[Counting a table](#)

[Deleting a table](#)

[Creating a new data level for a table](#)

[Clearing a table](#)

[Renaming a table](#)

[Moving a table](#)

Copying a table

Copying a table copies all items from one table to another. The table can be copied using a right-click menu option or by dragging-and-dropping a table to a new destination.



Note:

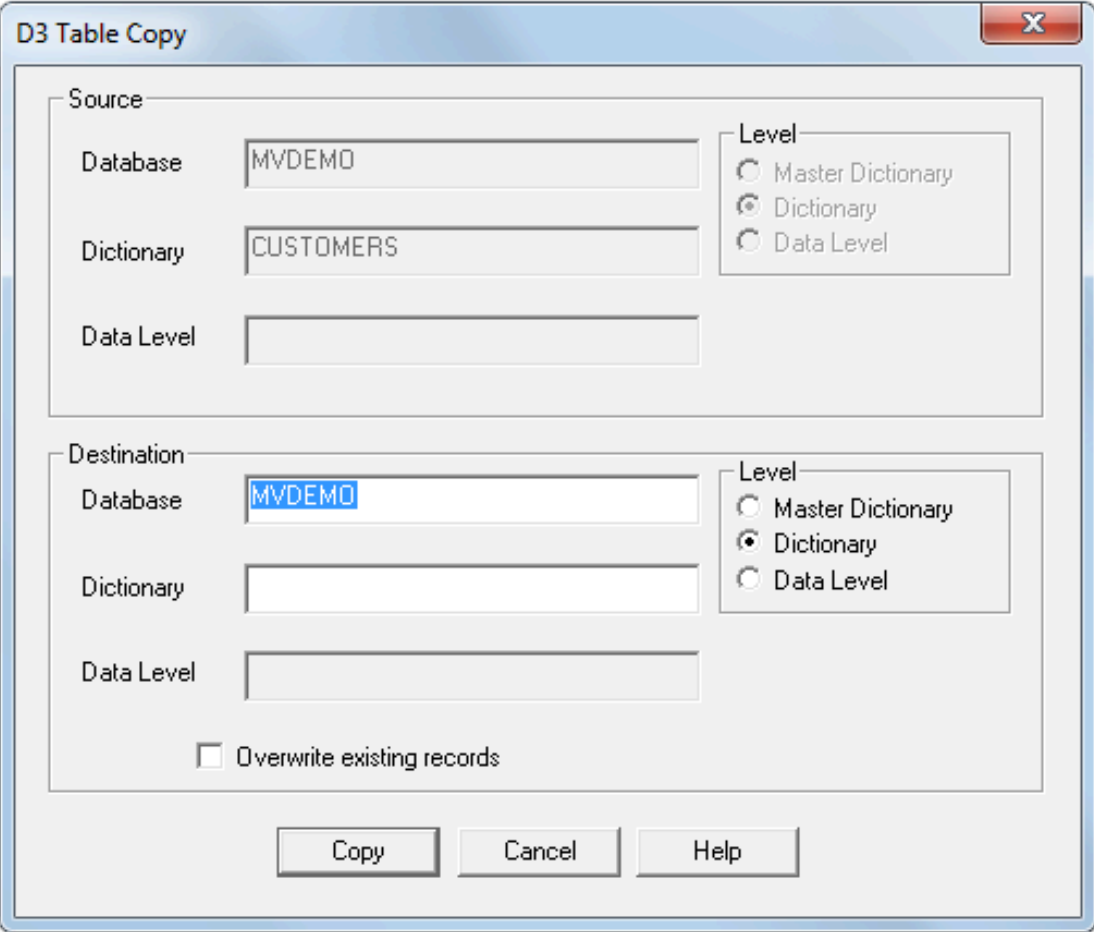
- If the table being copied is a Dictionary, the D-pointers are not copied.
- Copying by using the menu option is more efficient when copying files on the same server.



- When using the drag-and-drop method, if the table being copied already exists in the destination table, the copy operation will fail.

1. Right-click the table to copy from the **Domain Explorer** and select the **Copy** option.

The D3 Table Copy dialog box displays. Note that the characteristics of the table being copied are displayed in the **Source** section of the dialog box.



The D3 Table Copy dialog box is shown with the following fields and options:

- Source Section:**
 - Database: MVDEMO
 - Dictionary: CUSTOMERS
 - Data Level: (empty)
 - Level: Radio buttons for Master Dictionary, Dictionary (selected), and Data Level.
- Destination Section:**
 - Database: MVDEMO
 - Dictionary: (empty)
 - Data Level: (empty)
 - Level: Radio buttons for Master Dictionary, Dictionary (selected), and Data Level.
- Overwrite existing records:** A checkbox that is currently unchecked.
- Buttons:** Copy, Cancel, and Help.

2. Complete these fields:

Value	Description
Database	Database name for the destination table. By default, the destination table is assumed to be on the same database as the source.
Dictionary	File Dictionary.
Data Level	Data Level.
Level	Either Master Dictionary, Dictionary, or Data Level. By default, the destination level is assumed to be identical to the source level. As appropriate, selecting one of the buttons disables some other fields.

Value	Description
Overwrite Existing Records	When checked, records with the same record-IDs will be overwritten. If not checked, conflicting records will not be overwritten.

3. Click **Copy**.

The copy operation is performed and the number of records copied displays.

If both the source and destination table are on the same server, the copy operation takes place entirely on the remote server. If they are not on the same server, the copy operation is performed by the client, which reads the items from one server and copies them to the other.

 **Tip:** To copy using drag-and-drop, select the Dictionary or Data Level table node to copy from the **Domain Explorer**, press and hold the CTRL key and then drag-and-drop the selected node to the destination Data Level or Dictionary node.

Related concepts

[Table correlatives](#)

Related tasks

[Creating a table](#)

[Listing the contents of a table](#)

[Counting a table](#)

[Deleting a table](#)

[Creating a new data level for a table](#)

[Clearing a table](#)

[Renaming a table](#)

[Moving a table](#)

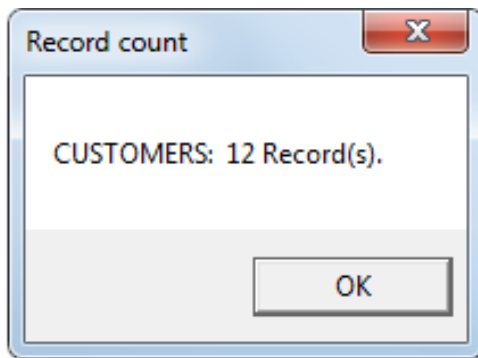
Counting a table

Complete these instructions to display the number of records contained within a specified table (whether Master Dictionary, Dictionary or Data Level).

1. Right-click the table to count from the **Domain Explorer** and select the **Count Records** option.

 **Note:** This command is executed on the server that holds the selected table and may take a significant time.

A message displays the number of records in the table.



2. Click **OK**.

Related concepts

[Table correlatives](#)

Related tasks

[Creating a table](#)

[Listing the contents of a table](#)

[Copying a table](#)

[Deleting a table](#)

[Creating a new data level for a table](#)

[Clearing a table](#)

[Renaming a table](#)

[Moving a table](#)

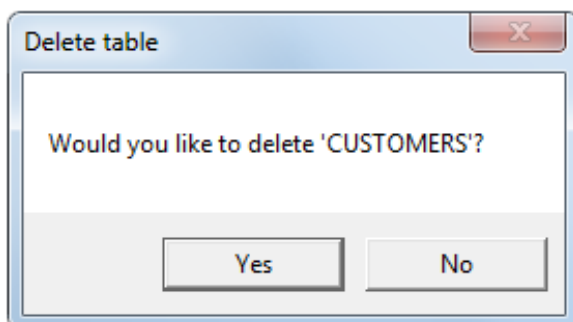
Deleting a table

You can delete individual Dictionary or Data Level tables. If deleting a Dictionary table, all Data Level tables residing within that Dictionary table are recursively deleted.

Complete these instructions to delete a table from a database.

1. Right-click the table to delete from the **Domain Explorer** and select the **Delete Table** option. Note that you can also select the table and press the DELETE key or drag the table to the **Recycle Bin**.

If the **Warnings Displayed** option is enabled from the File Manger - Options dialog, the Delete table dialog box displays and confirmation is required. See [Editing File Manager options](#) for more information.



2. Click **Yes**.

The specified table is deleted and is removed from the **Tables** node in the **Domain Explorer**.

Related concepts

[Table correlatives](#)

Related tasks

- [Creating a table](#)
- [Listing the contents of a table](#)
- [Copying a table](#)
- [Counting a table](#)
- [Creating a new data level for a table](#)
- [Clearing a table](#)
- [Renaming a table](#)
- [Moving a table](#)

Creating a new data level for a table

You can add new Data Levels to any existing Dictionary table.

To add both a new Dictionary and Data Level to a database, see [Creating a table](#).

1. Right-click the Dictionary table node to which you want to add the new Data Level table from the **Domain Explorer** and select the **New Data Level** option.

The New Table dialog box displays. Although this dialog box contains sections for both Dictionary and Data Levels, only the Data Level section is enabled and can be edited.

The 'New Table' dialog box is shown. It has a title bar with a close button. Inside, there's a 'Current Database' field set to 'MVDEMO'. Below this are two main sections: 'Dictionary' and 'Data Level'. The 'Dictionary' section is disabled (greyed out) and contains fields for Name (CUSTOMERS), Modulo (0), Frame Size, Server (\$mds), Path (\$mds:\$MD)/customers/c, and a 'Case Insensitive' checkbox (unchecked). The 'Data Level' section is active and contains fields for Name, Modulo (31), Frame Size (4K), Server (\$mds), Path, and a 'Case Insensitive' checkbox (checked). At the bottom are 'Create' and 'Cancel' buttons.

2. Complete the following fields:

Value	Description
Name	Table Name. Any string of up to 50 characters, excluding the colon (:) character. If the database was created using the case insensitive option, the Dictionary file name is case

Value	Description
	insensitive. If the Dictionary is case insensitive, the Data Level name is case insensitive, whether the database is case sensitive or not.
Modulo	Enter the table modulo. The modulo should be selected in conjunction with the frame size so that the items are properly distributed across the table, without excessive group overflow.
Frame Size	Select a frame size. By default, both the Dictionary and Data Level frame sizes are 4 KB. Small frame sizes are suited to files that contain a few small items.
Server	Network name of the server where the table will reside. By default, the dialog will create the table on the server where the database resides, but there is no requirement. The Dictionary and the Data Levels do not need to be located on the same systems. Selecting the server is essentially a question of disk space, performance, and proximity to the users. The server name can be either the network name of the computer, an alias defined in the registry of this server, or a server environment variable. The list of Server Environment Variables is displayed by clicking the Env. button. The D3 File System Interface service must be started on this server. See Server environment variables for more information.
Path	Full path name where the table is to reside. Use forward slashes (/) to separate the directory names. If the path starts with a / or a drive letter (for example, C:/mypath), the path is absolute and the table will be created on that location. If the path does not start with a /, the path is relative and is added at the end of the root directory specified in the server configuration used to create the table. When using a drive, it is important to specify an absolute path (start with a /), otherwise the directory will be appended to whatever current directory the drive is set to. Note that UNC names (\\server\path) are not supported. The Path can be any valid host file system path, or a path environment variable. The list of Path Environment Variables is displayed by clicking the Env. button. See Path environment variables for more information.  Note: It is advised to leave the path empty and to let the system create the table under the file system server root.
Case Insensitive	Specifies that all items in the table, including table names for a Dictionary, will be case insensitive.

- Click **Create**.

The new Data Level is created and displays under the specified Dictionary table node in the **Domain Explorer**.

Related concepts

[Table correlatives](#)

Related tasks

[Creating a table](#)

[Listing the contents of a table](#)

[Copying a table](#)

[Counting a table](#)

[Deleting a table](#)

[Clearing a table](#)

[Renaming a table](#)

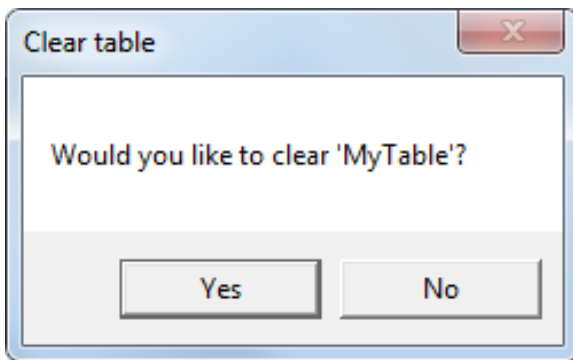
[Moving a table](#)

Clearing a table

Clearing a table deletes all records in the table. Both Dictionary tables and Data Level tables can be separately cleared. If clearing a Dictionary table, the Data Level tables residing within that Dictionary table are not cleared. Additionally, Dictionary table D-pointers that point to tables are not deleted.

1. Right-click the Dictionary or Data Level table to clear from the **Domain Explorer** and select the **Clear Table Dictionary** option (for Dictionary tables) or the **Clear Data Level** option (for Data Level tables).

If the **Warnings Displayed** option is enabled from the File Manger - Options dialog, the **Clear table** dialog box displays and confirmation is required. See [Editing File Manager options](#) for more information.



2. Click **Yes**.

The records in the specified table are cleared.

Related concepts

[Table correlatives](#)

Related tasks

[Creating a table](#)

[Listing the contents of a table](#)

[Copying a table](#)

[Counting a table](#)

[Deleting a table](#)

[Creating a new data level for a table](#)

[Renaming a table](#)

[Moving a table](#)

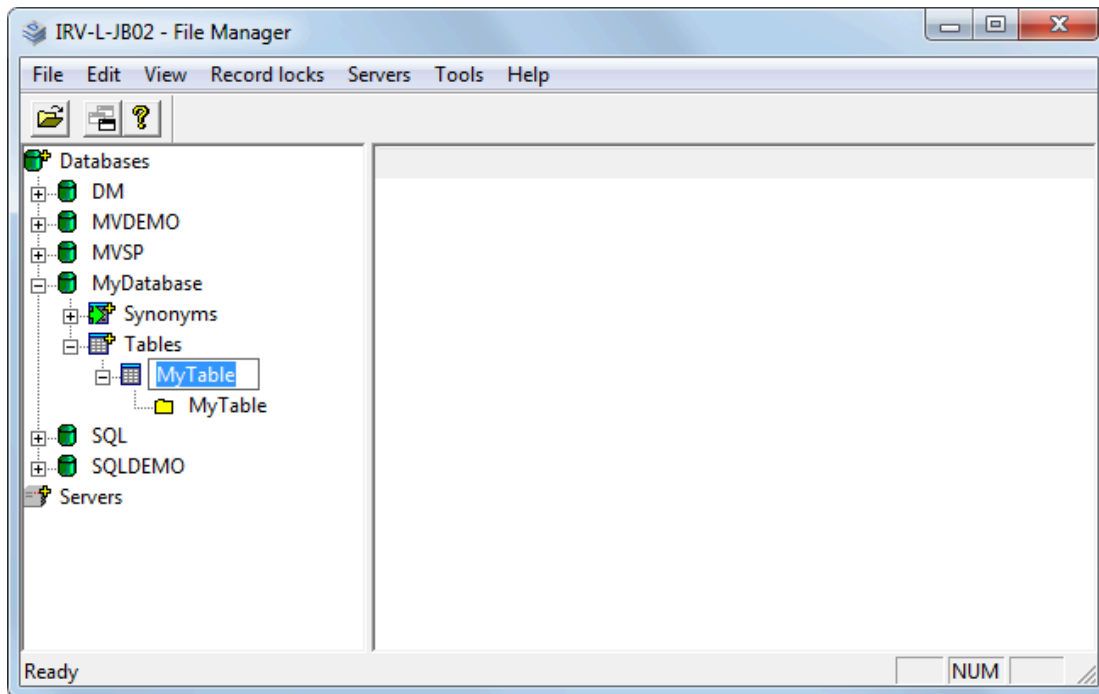
Renaming a table

You can rename any table in your database.

Complete these instructions to rename a table.

1. Right-click the table to rename from the **Domain Explorer** and select the **Rename** option.

The name for the specified table is highlighted in the **Domain Explorer**.



2. Type the new table name and then press **ENTER** .

The specified table is renamed.

Related concepts

[Table correlatives](#)

Related tasks

[Creating a table](#)

[Listing the contents of a table](#)

[Copying a table](#)

[Counting a table](#)

[Deleting a table](#)

[Creating a new data level for a table](#)

[Clearing a table](#)

[Moving a table](#)

Moving a table

Complete these instructions to move a table.

Do one of the following:

- To move a Dictionary table, select the Dictionary table node and drag-and-drop it into the destination **Tables** node.
- To move a Data Level table, select the Data Level table node and drag-and-drop it into the destination Dictionary table node.

Related concepts

[Table correlatives](#)

Related tasks

- [Creating a table](#)
- [Listing the contents of a table](#)
- [Copying a table](#)
- [Counting a table](#)
- [Deleting a table](#)
- [Creating a new data level for a table](#)
- [Clearing a table](#)
- [Renaming a table](#)

Table correlatives

Table Correlatives define optional operations to be performed on the table when it is accessed.

Tables correlatives include:

- Indexes

```
I{n} acorrelative
```

<i>n</i>	Index number (may be missing). Generated by the system.
<i>acorrelative</i>	Index description.

- Write triggers

```
CALLX {file.spec} item
```

<i>file.spec</i>	Optional file name where the Flash subroutine is.
<i>item</i>	Flash subroutine name.

Related tasks

- [Creating a table](#)
- [Listing the contents of a table](#)
- [Copying a table](#)
- [Counting a table](#)
- [Deleting a table](#)
- [Creating a new data level for a table](#)
- [Clearing a table](#)
- [Renaming a table](#)
- [Moving a table](#)

Records

The following section describes copying, editing, and deleting records. Additionally, information on editing D-Pointers

is presented.

Related concepts

[File Manager layout](#)

[Domains](#)

[Databases](#)

[Tables](#)

[Additional File Manger operations](#)

[Environment variables](#)

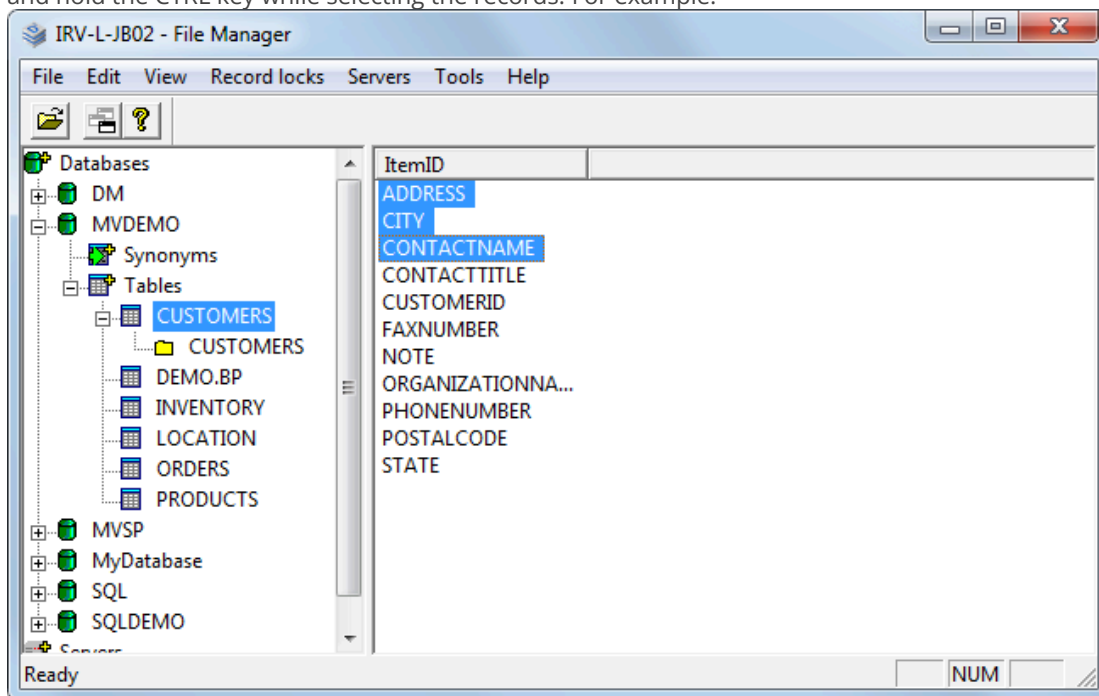
Related tasks

[Editing File Manager options](#)

Copying records

Complete these instructions to copy records from one Dictionary or Data Level table of a database to a different Dictionary or Data Level table.

1. Right-click the Dictionary or Data Level table of a database from the **Domain Explorer** and select the **Show Data** option to display the table's contents in the **Workspace**.
2. Select one or more records from the source Dictionary or Data Level table. To select multiple records, press and hold the CTRL key while selecting the records. For example:



3. Press and hold the CTRL key while dragging the selected records to the destination Dictionary or Data Level table.
4. Right-click the new destination Dictionary or Data Level table from the **Domain Explorer** and select the **Show Data** option to confirm the records have been copied.

Related tasks

[Creating a record](#)

[Importing records](#)

[Editing a record](#)

[Deleting records](#)

[Editing a D-pointer](#)

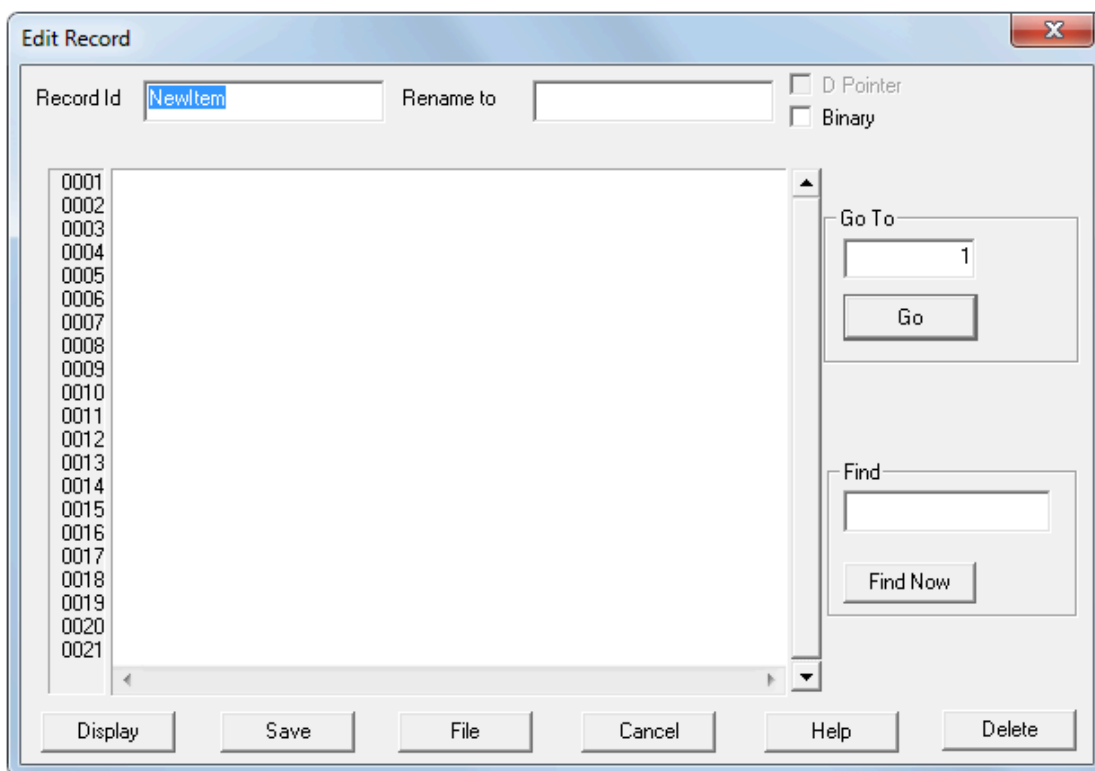
Creating a record

New records can be created in the Master Dictionary table, the Dictionary table or the Data Level table.

Complete the instructions below to create a new record. To edit an existing record, see [Editing a record](#) for more information.

1. Right-click the Master Dictionary table, Dictionary table or Data Level table on which to create the new record from the **Domain Explorer** and select the **Edit Record** option.

The Edit Record dialog box displays.



2. Type a name for the new record in the Record Id text box (replacing the default NewItem text).
3. Click your mouse inside the record editor (the white space to the right of the numbered list) and begin entering the attributes that will comprise your record. Press **ENTER** to move to the next line in the record editor.

Tip: You can use the **Go To** text box to return the cursor to an attribute number that has already been entered. Additionally, you can use the **Find** text box to find specific text within your record.

4. Select the **Binary** check box if the new record is a binary record.
5. Click **Save**.

The new record is saved. To verify the new record, right-click the table in which the new record was created

and select the **Show Data** option.

Related tasks

[Copying records](#)

[Importing records](#)

[Editing a record](#)

[Deleting records](#)

[Editing a D-pointer](#)

Importing records

Complete these instructions to import records into a Master Dictionary, Dictionary or Data Level table.

1. Select the Master Dictionary table, Dictionary table or Data Level table into which you want to import the new record from the **Domain Explorer**.
2. Select the **Import** option from the **Tools** menu.

The **Import** dialog box displays.

3. Select the record to import.
4. Type the **Record Id** to assign to the new record in the **Into** text box.
5. Click **Import**.

The record is imported. To verify the imported record, right-click the table into which the record was imported and select the **Show Data** option.

Related tasks

[Copying records](#)

[Creating a record](#)

[Editing a record](#)

[Deleting records](#)

[Editing a D-pointer](#)

Editing a record

Records can be edited using the Edit Record dialog box. Complete the instructions below to edit a record.

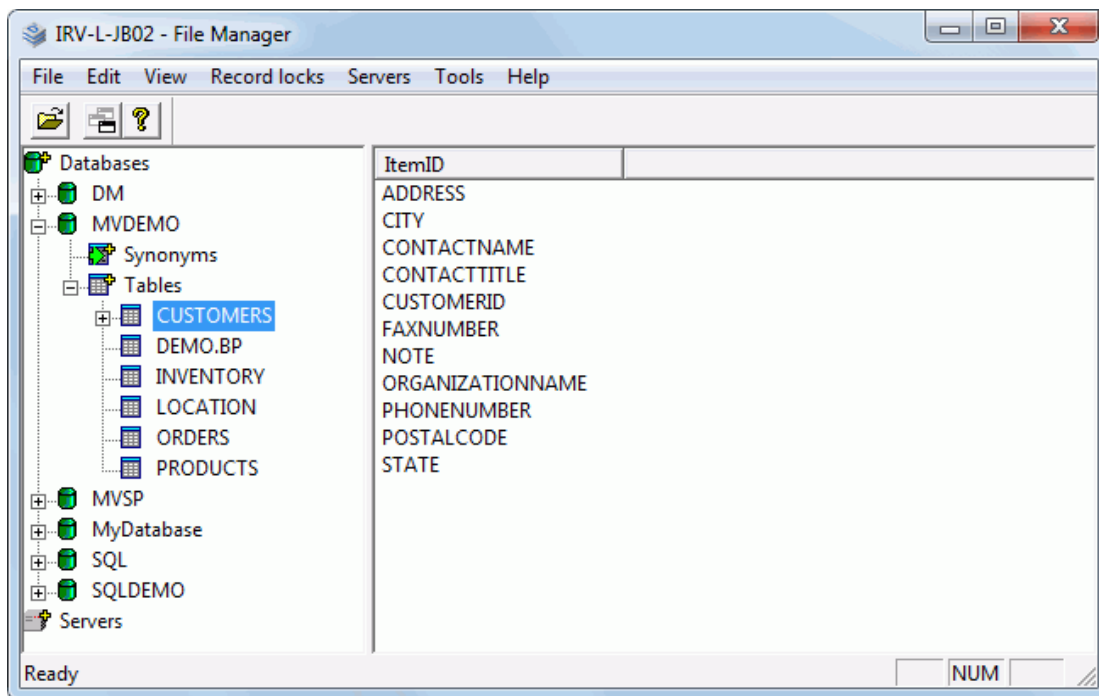
To create anew record, see [Creating a record](#) for more information.



Note: When editing a record, a record lock is applied. If the record has already been locked by another application, the record is edited as read only. Click the **Display** button to check whether the record lock has been released. If the record lock has been released by the other application, it is now edited with write permissions.

1. Right-click the table that contains the record to edit and select the **Show Data** option.

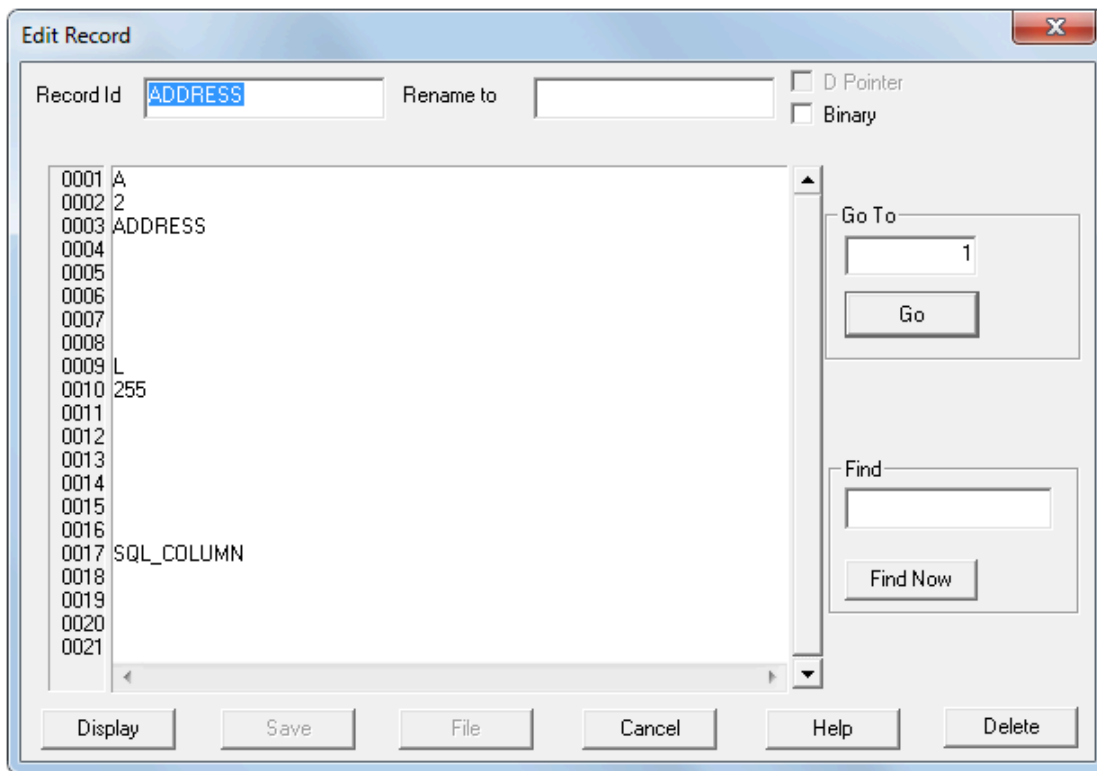
The data for the specified record displays in the **Workspace**.



2. Right-click the record to edit from the **Workspace** and select the **Edit Record** option. You can also select the record to edit from the **Workspace** and select **Edit Record** from the **Edit** menu.

The specified record displays in the Edit Record dialog box.

Tip: Multiple records can be selected for editing by holding down the CTRL key while selecting records. If multiple records are selected, each subsequent record displays after a record has been saved and closed or the record has been filed with the **File** button.



3. Do any of the following:

- To rename the **Record Id**, type a new name for the record in **Rename to** text box.
- Select or deselect the **Binary** check box to change the record to binary or non-binary.
- Add, edit or remove any of the attributes in the record editor section.

Note: You can use the **Go To** text box to return the cursor to an attribute number that has already been entered. Additionally, you can use the **Find** text box to find specific text within your record.

4. Click **Save**.

The edited record is saved. To verify the edited record, right-click the table in which the record was edited and select the **Show Data** option.

Related tasks

- [Copying records](#)
- [Creating a record](#)
- [Importing records](#)
- [Deleting records](#)
- [Editing a D-pointer](#)

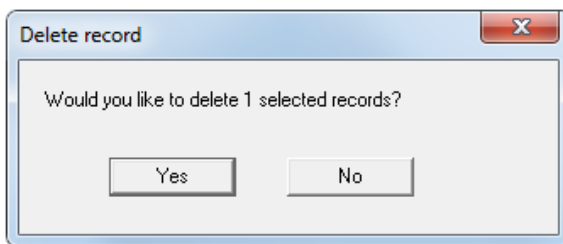
Deleting records

Complete these instructions to delete a record from a Master Dictionary, Dictionary or Data Level table.

1. Right-click the Master Dictionary, Dictionary or Data Level table of a database from the **Domain Explorer** and select the **Show Data** option to display the table's records in the **Workspace**.
2. Select one or more records to delete. To select multiple records, press and hold the CTRL key while selecting the records.
3. Do one of the following:
 - Press the **Delete** key.
 - Drag the select records to the **Recycle Bin**.

 **Warning:** A record cannot be recovered once it has been placed in the **Recycle Bin**.

If the **Warnings Displayed** option is enabled from the File Manger - Options dialog box, the Delete record dialog box displays and confirmation is required. See [Editing File Manager options](#) for more information.



4. Click **Yes**.

The specified records are deleted.

Related tasks

[Copying records](#)
[Creating a record](#)
[Importing records](#)
[Editing a record](#)
[Editing a D-pointer](#)

Editing a D-pointer

A D-pointer is a special record in a Master Dictionary or a Table Dictionary that contains information about a table such as its location, type, and other correlatives. These records are critical to the table organization because they enable you to easily find the tables across the network. For safety reasons, the Master Dictionaries of the accounts should be created on a replicated directory.

It is normally not necessary to edit D-pointers because the File System Interface servers maintain them. However, at times it may be necessary to recreate them. For example, you need to recreate a D-pointer following a disk error on a single server to re-establish the links to the various data tables in the database or to manually add [Table correlatives](#).

1. Select the Domain, Master Dictionary, or Dictionary table where the D-pointer needs to be created or modified from the **Domain Manager**, and then select **Edit D-pointer** from the **Edit** menu. You can also right-click the

Domain, Master Dictionary, or Dictionary table and select the **Edit D-pointer** option.

The Edit D Pointer dialog box displays.

2. Complete these fields:

Value	Description
Record ID	The name of the D-pointer. This name is the normal table name, as seen by the application. Be careful about the case sensitivity of the Dictionary. To determine whether the Dictionary is case sensitive, see Editing General table properties .
Table Name	Actual table name. If the table name is simple (for example, it does not contain reserved characters) the table name is identical to the record ID.
Server	Name of the server where the table is located. See Server environment variables for more information.
Path	Path name. See Path environment variables for more information.
Modulo	The actual value of the modulo is not important. Enter a nonzero value for a hashed table (normal data) or 0 for a direct table. Once the D-pointer has been rebuilt, the

Value	Description
	actual modulo can be used by examining the General Table Properties .
Table Correlatives	Edit the table correlatives. There must one correlative per line, upper or lowercase.
Table Type	Reserved. Leave this field set to Hashed FSI Table .
Save State	Specifies a save state (as either a DX or DY pointer).

3. Click **Update**.

Related tasks

[Copying records](#)
[Creating a record](#)
[Importing records](#)
[Editing a record](#)
[Deleting records](#)

Additional File Manger operations

The following section describes the operations available for listing and manipulating information.

Related concepts

[File Manager layout](#)
[Domains](#)
[Databases](#)
[Tables](#)
[Records](#)
[Environment variables](#)

Related tasks

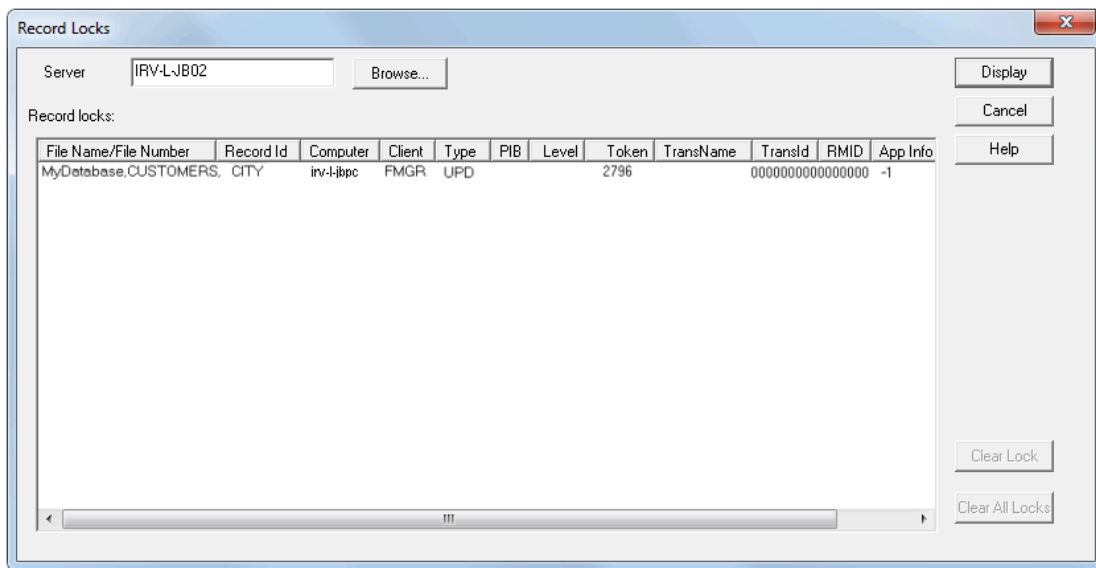
[Editing File Manager options](#)

Listing or clearing record locks

The Record Locks dialog box enables you to list and clear record locks on a particular server.

1. Select **Record Locks** from the **Record Locks** menu.

The Record Locks dialog box displays.



2. Enter the server name in the **Server** text box or click **Browse** to select a different server. Note that the current server is selected by default.
3. Click the **Display** button.

Below is a description of the information that displays in the **Record Locks** dialog box.

Value	Description
File Name/File Number	Database, Dictionary table and Data Level table in which the locked file resides.
Record Id	Record Id of the locked item.
Computer	Computer name where the locked record is located.
Client	Computer name of the client.
Type	Lock Type.
(0xtttttttt) or PIB	Thread-ID, in hexadecimal, of the thread that owns the lock. If the client thread is a VME thread, the PIB number replaces the thread-ID.
Level	Lock owner's push-level number.
Token	Unique identifier for the locked item.
TransName	Transaction name.
TransId	Transaction ID.
RMID	
App Info	

4. To clear locks, do one of the following:
 - To clear a specific lock, select the lock to clear from the **Record locks** list and click the **Clear Lock** button.
 - To clear all locks, click the **Clear All Locks** button.

Warning: Clearing record locks may create database inconsistencies and should be done only with extreme care.

Related tasks

[Editing General table properties](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Listing servers](#)

[Displaying Flash shared code](#)

[Creating indexes](#)

[Checking overflow](#)

[Running a hash test](#)

[Resizing files](#)

Editing General table properties

The Properties dialog box allows you to view and change table properties.

Complete these instructions to display and change the General Table properties.

1. Right-click the table to inspect from the **Domain Explorer** and select the **Properties** option.

The Properties dialog box displays.

2. Click the **General** tab.

The **General** tab displays.

The screenshot shows the 'ORDERS' Properties dialog box with the 'General' tab selected. The dialog box has a title bar with 'ORDERS' and a close button. Inside, there are three tabs: 'General', 'Usage', and 'Statistics'. The 'General' tab contains the following fields and controls:

- Server:** A text box with a blue cursor.
- Path:** A text box containing '..\mydemo/orders/orders/orders'.
- Modulo:** A text box containing '3'.
- Frame size (K):** A text box containing '4'.
- Created on:** A text box containing '1/10/2014 11:32'.
- By:** A text box.
- Table Id:** A text box containing '106'.
- Case Insensitive:** A checked checkbox.
- Help:** A button.
- Hash:** A button.
- Resize:** A button.
- Compression Trigger:** A text box containing '0'.
- K:** A text box.
- Compress:** A button.

At the bottom of the dialog box are three buttons: 'OK', 'Cancel', and 'Apply'.

The table below describes the information and functionality provided in the **General** tab.

Value	Description
Server	Displays the network server name where the table is located.
Case Insensitive	Checked if the table is case insensitive. This property cannot be modified.
Path	Displays the path and full name where the table is located, without the table extension.
Modulo	Displays the current modulo for the table. The modulo can be increased by typing a new value and clicking the Resize button. See Resizing files for more information.
Compression Trigger	Displays frame size in kilobytes. This property cannot be modified.
Hash	Produces a graphic representation of item-ID distribution within a given file, according to the <i>test</i> modulo provided. The command is useful when performing file reallocation, providing insight into how the item-IDs hash into the file, as well as providing a suggested modulo. See Running a hash test for more information.
Resize	Increases the size of the modulo. A new modulo must be specified from the Modulo text box to enable this button. See Resizing files for more information.

Related tasks

[Listing or clearing record locks](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Listing servers](#)

[Displaying Flash shared code](#)

[Creating indexes](#)

[Checking overflow](#)

[Running a hash test](#)

[Resizing files](#)

Viewing Usage table properties

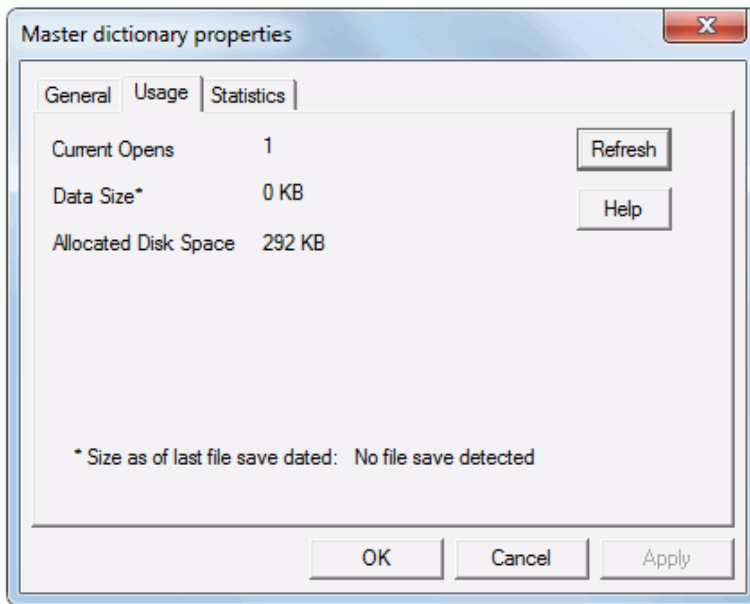
Complete these instructions to display the Usage Table properties.

1. Right-click the table to inspect from the **Domain Explorer** and select the **Properties** option.

The Properties dialog box displays.

2. Click the **Usage** tab.

The **Usage** tab displays.



The table below describes the information and functionality provided in the **Usage** tab.

Value	Description
Current Opens	Number of clients currently having the table open. The File Manager itself has the table opened once (twice if the table is the Master Dictionary of a database).
Data Size	Data size as of the last file save.
Allocated Disk Space	Amount of disk space allocated for the table.
Refresh	Click the Refresh button to update the display.

Related tasks

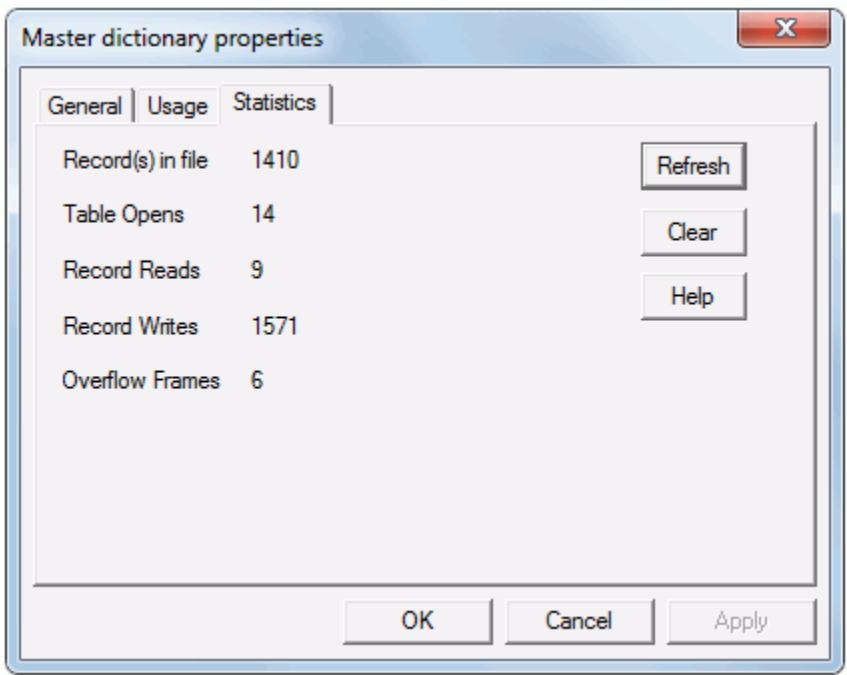
- [Listing or clearing record locks](#)
- [Editing General table properties](#)
- [Displaying Statistics table properties](#)
- [Listing servers](#)
- [Displaying Flash shared code](#)
- [Creating indexes](#)
- [Checking overflow](#)
- [Running a hash test](#)
- [Resizing files](#)

Displaying Statistics table properties

Complete these instructions to display the Statistics Table properties.

1. Right-click the table to inspect from the **Domain Explorer** and select the **Properties** option.
The Properties dialog box displays.
2. Click the **Statistics** tab.

The **Statistics** tab displays.



The table below describes the information and functionality provided in the **Statistics** tab.

Value	Description
Records in file	Current approximate count of records in table. This field cannot be cleared.
Table Opens	Number of table open operations.
Record Reads	Number of read record operations.
Record Write	Number of write record operations.
Overflow Frames	Number of overflow frames.
Refresh	Updates the display.
Clear	Resets the statistic counters.

Related tasks

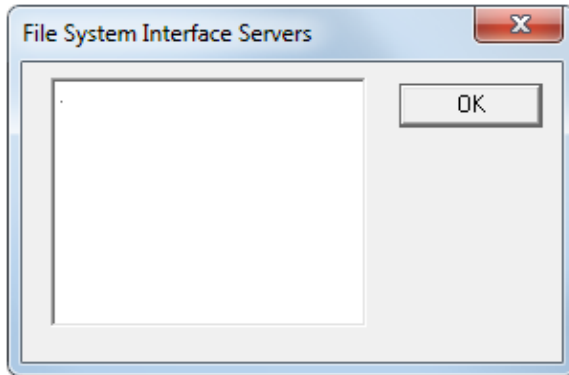
- [Listing or clearing record locks](#)
- [Editing General table properties](#)
- [Viewing Usage table properties](#)
- [Listing servers](#)
- [Displaying Flash shared code](#)
- [Creating indexes](#)
- [Checking overflow](#)
- [Running a hash test](#)
- [Resizing files](#)

Listing servers

The File System Interface Servers dialog box lists the FSI servers being accessed by the currently open domain.

1. Select **Server List** from the **Servers** menu.

The File System Interface Servers dialog box displays.



2. Click **OK** to close the dialog box.

Related tasks

[Listing or clearing record locks](#)

[Editing General table properties](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Displaying Flash shared code](#)

[Creating indexes](#)

[Checking overflow](#)

[Running a hash test](#)

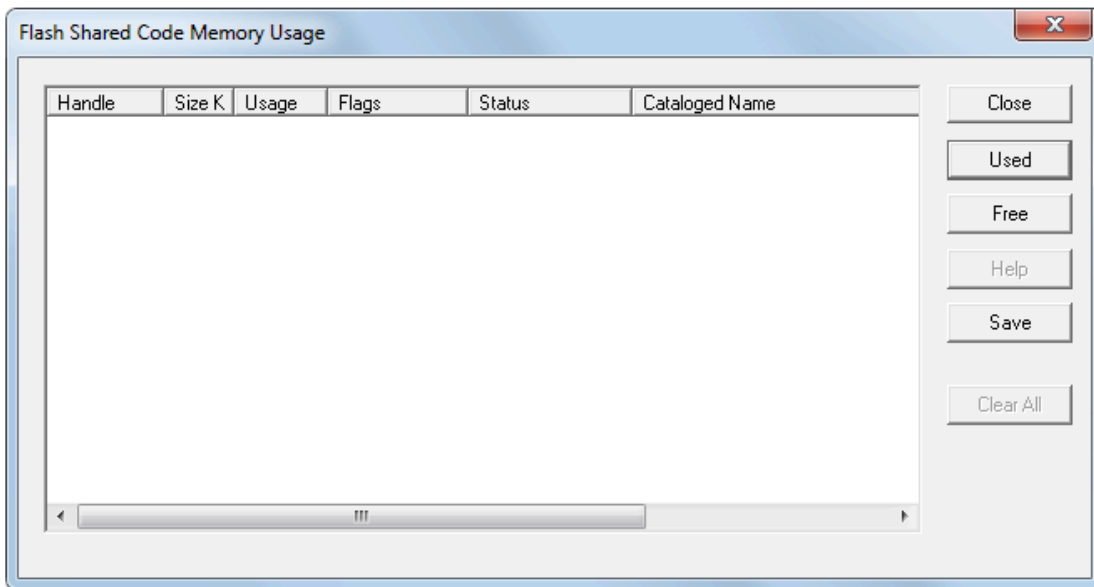
[Resizing files](#)

Displaying Flash shared code

The File Manager enables you to display the current usage of the shared memory segment that contains the shared FlashBASIC object code.

1. Select **Flash Shared Code** from the **Tools** menu.

The Flash Shared Code Memory Usage dialog box opens.



2. Review button functionality and field descriptions in the following table:

Button	Description	
Close	Click Close to exit this dialog box.	
Used	Click Used to display a list of the used blocks. This list shows the following information:	
	Handle	Shared memory address.
	Size K	Size in KB of the segment.
	Usage	Number of times this program is referenced by users and files (in the case of triggers).
	Flags	Shows the type followed by a decimal value of the program's flags. Set to 1 if the module stays loaded in memory, even if it is currently unused (Usage=0). The following lists the possible values.
		Bad Corrupted header
		Lost Abandoned block
		Old Obsolete block
		Sticky Not implemented
		Local Not shared
		Free Not used
		Shared Normal
		Bit values display in decimal and are:
	1	Unused

Button	Description		
		2	Main
		4	Sticky
		8	Local
		16	Case
		32	No Text
		64	Interp
		128	Obsolete
		256	Abandoned
		512	Trigger
	Status	Possible values are:	
		Bad	Invalid signature. Displays with the signature
		OK	Normal
		Err	Corrupted object. Displays with the header lock
		Loading	Block allocated but the object has not been loaded and relocated
	Cataloged Name	File and record name of the module.	
	Unique Name	File number followed by the program's Item-ID.	
Free	Click Free to display the free blocks. The size in kilobytes (KB) is displayed.		
Help	Click Help to view help topics related to this dialog box.		
Save	Click Save to save a copy of the current Flash shared code to a file. This is useful for debugging purposes for Rocket support.		
Clear All	Click Clear All to clear (release) all used blocks. This button is only available when the used blocks are displayed, and if there is at least one used block.		
	 Warning: This option can cause severe aborts if used while an application is currently running a module and should be used with extreme caution.  Note: This segment is self-extensible, by chunks of 512 KB, and is created on demand. Therefore, it is possible that no free segment and no used segment display in the list if no FlashBASIC program was ever run on the system.		

For more information, see:

- *D3 Reference Manual:*
 - Recompiling a trigger

- nt_inv-flash command
- *D3 Windows System Administration Guide:*

D3 Device Manager, FlashBASIC settings, Load Trigger at Each File Open description

Related tasks

[Listing or clearing record locks](#)

[Editing General table properties](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Listing servers](#)

[Creating indexes](#)

[Checking overflow](#)

[Running a hash test](#)

[Resizing files](#)

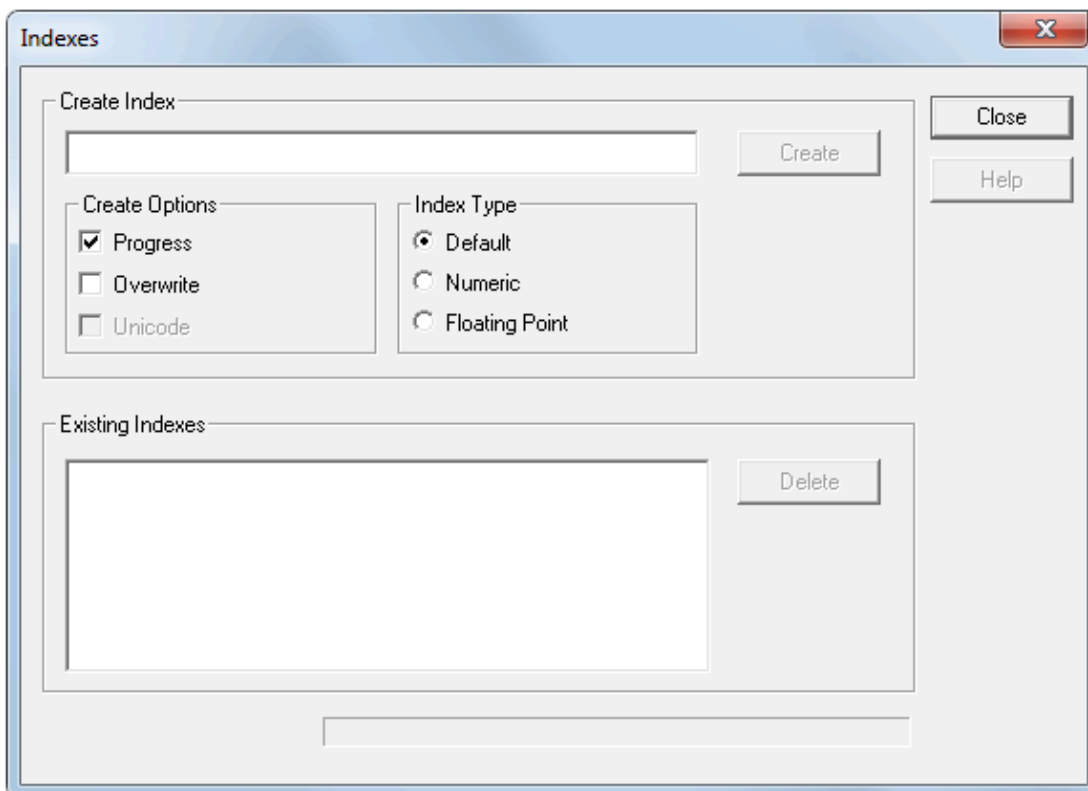
Creating indexes

Use the Indexes option to view, create and delete B-tree indexes on the specified table.

See the *create-index Command* topic in the *D3 Reference Manual* for more information on index creation.

1. Right-click the Data Level table on which to create the new index from the **Domain Explorer** and select the **Indexes** option.

The Indexes dialog box displays.



2. Provide the following values. See the *create-index Command* topic in the *D3 Reference Manual* for more information on index creation options.

Value	Description	
Create Index	Specifies the index A-correlative.	
Create Options	Select any of the available options:	
	Progress	Displays a progress bar indicating the progress of the index build. This option may slow down the build because the server returns periodically to indicate its progress. Without this option, the build proceeds silently on the server.
	Overwrite	If the index already exists, this option specifies to overwrite the index without prompting for confirmation.
	Unicode	This check box will be checked if the associated data is Unicode data. Note that this value cannot be changed (unchecked).
Index Type	Select the appropriate option:	
	Default	Creates standard indexes.
	Numeric	Creates numeric indexes.
	Floating Point	Creates floating-point indexes.

3. Click **Create**.

The index is created and displays in the **Existing Indexes** list.

-  **Note:** Indexes listed in the in the **Existing Indexes** list can be rebuilt or deleted:
- To rebuild an index, double-click the index to rebuild from the **Existing Indexes** list to display the index name in the **Create Index** text box. Then select the **Overwrite** option and click the **Create** button. See the *indexer Command* in the *D3 Reference Manual* for more information on rebuilding indexes.
 - To delete an index, select the index to delete from the **Existing Indexes** list and click the **Delete** button.

Related tasks

[Listing or clearing record locks](#)

[Editing General table properties](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Listing servers](#)

[Displaying Flash shared code](#)

[Checking overflow](#)

[Running a hash test](#)

[Resizing files](#)

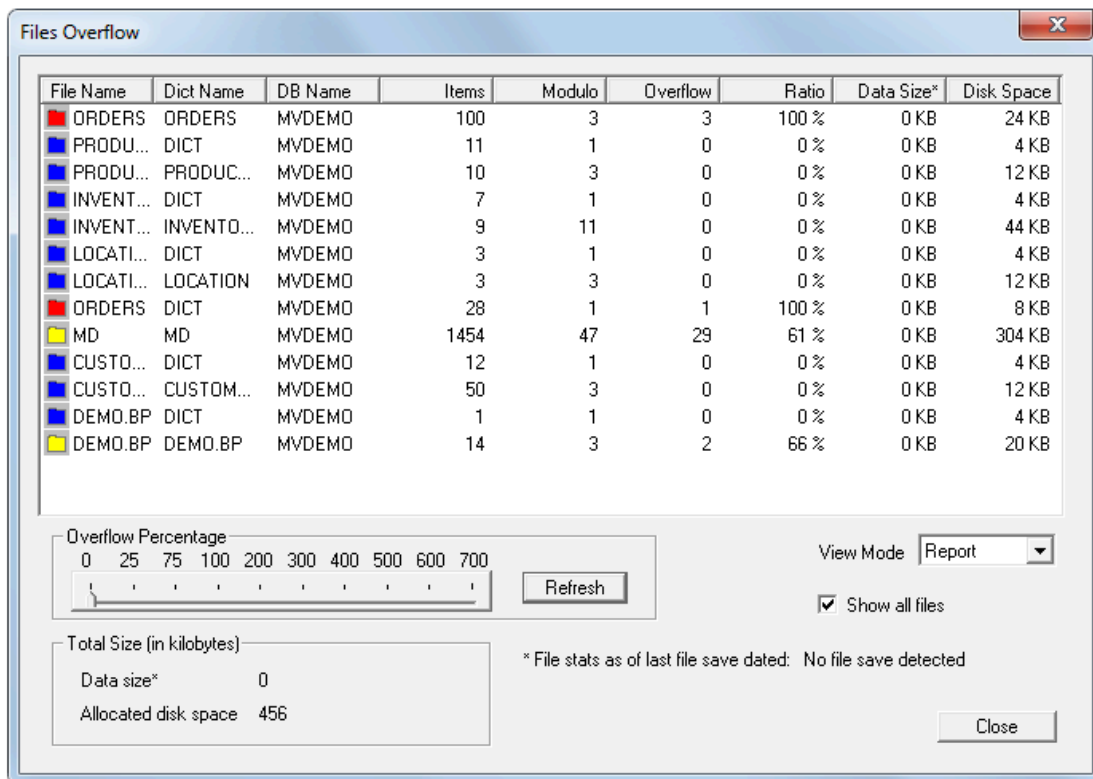
Checking overflow

The Check Overflow Files option displays an overflow report for the specified files.

Complete the instructions below to display the overflow report.

1. Right click the **Tables** node for the database you want to check from the **Domain Explorer** and select the **Check Overflow Files** option.

The Files Overflow dialog box displays.



2. Do any of the following:

- Select the **Show all files** check box and click the **Refresh** button to displays statistics for all tables in the database. If the **Show all files** check box is not selected, only those tables that are currently in an overflow condition are displayed.

The files icons representing each table in the database are color-coded to indicate the overflow state:

Icon	Description
	Indicates a file with an overflow condition whose ratio is less than 100%.
	Indicates a file with an overflow condition whose ratio is 100% or more.
	Indicates a file without an overflow condition.

- Select the **Small Icons** option from the **View Mode** drop-down to suppress the statistic report and only display icons representing each table in the database.

- Change the **Overflow Percentage** slider and click the **Refresh** button to limit the files displayed in the report to those that have an overflow percentage (ratio) of the selected value or greater. For example, if 200 is selected, only those files with an overflow ratio of 200% or greater are displayed.
- Click any of the files in the report to open the **General** tab of the Properties dialog box where you can test and resize the modulo for the file. See [Editing General table properties](#), [Running a hash test](#) and [Resizing files](#) for more information.

Related tasks

[Listing or clearing record locks](#)

[Editing General table properties](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Listing servers](#)

[Displaying Flash shared code](#)

[Creating indexes](#)

[Running a hash test](#)

[Resizing files](#)

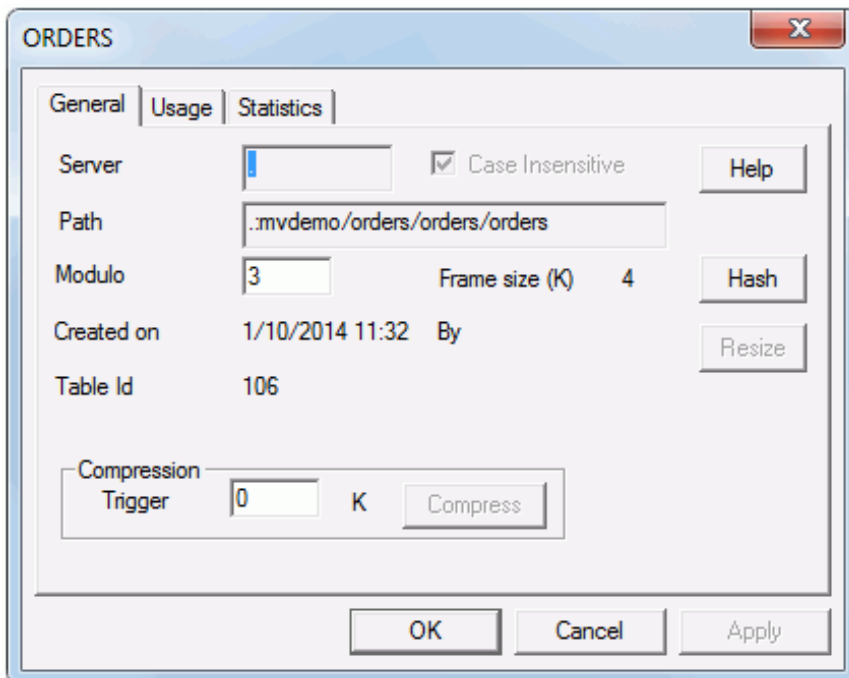
Running a hash test

The hash test feature displays the current state of the file and allows you to test various hash values.

Complete the instructions below to run a hash test. See the *hash-test Command* topic in the *D3 Reference Manual* for more information.

1. Right-click the table to resize from the **Domain Explorer** and select the **Properties** option.

The **General** tab of the Properties dialog box displays.



- Click the **Hash** button.

The Hash Test dialog box displays the current hash statistics for the current modulo.

Modulo

☒ Modulo

☐ Max fill ratio %

☒ Prime only

File Sample Size

%

File sample size : 100%
Total item count : 100, Byte Count : 9834
Avg bytes/item : 98, Avg items/group : 33

- Do one of the following:

- Select the **Modulo** option and type a modulo to test.

Tip:

- If the Prime only check box is selected, the number you type for the modulo will automatically adjust to the highest and closest prime number.
- When examining large files, only a sample (10% if the file has a modulo larger than 10,000) is taken. This causes the hash test to return approximate results. After experimenting with various modulos, it may be necessary to set the **File Sample Size** to 100% to get a more exact result of the hash test.
- Click the Reset button to return all of your testing values to their original values.

- Select the **Max fill ratio** option and type and maximum fill ratio percentage to test.

- Click the **Test** button.

The hashing statistic display is updated based on the entered **Modulo/Max fill ratio**.

- Do one of the following:

- If the tested **Modulo/Max fill ratio** is not sufficient, continue to test different values until a desired result is reached.

- If the tested **Modulo/Max fill ratio** is sufficient, click the **Resize** button to update the modulo. See [Resizing files](#) for more information.

6. Click **Close** when the testing process is complete.

Related tasks

[Listing or clearing record locks](#)

[Editing General table properties](#)

[Viewing Usage table properties](#)

[Displaying Statistics table properties](#)

[Listing servers](#)

[Displaying Flash shared code](#)

[Creating indexes](#)

[Checking overflow](#)

[Resizing files](#)

Resizing files

The resizing process works by extending the primary space of the file and rehashing all items. While this process is running, access to the file may be slower, but the file is otherwise available as normal. If the system is rebooted while files are being resized, the resizing process is restarted automatically by the servers. The resizing is done in the background by system threads, running on the servers where the file is located.

The current version has the following limitations that will be addressed in future releases:

- The modulo of a file can only be increased.

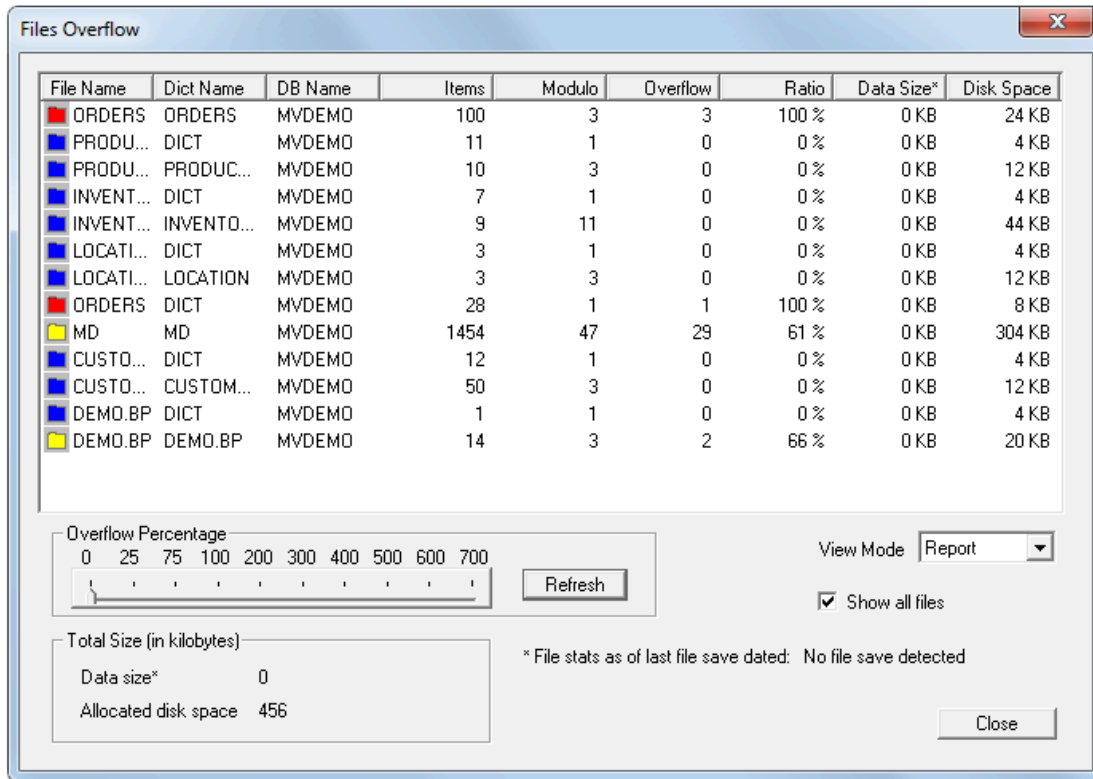


Note: If the modulo is unchanged, resizing compacts the file instead.

- After resizing, overflow space is not relinquished to the underlying host file system. The File Manager reports lower overflow usage, but Explorer still displays the .D3O overflow spaces.
- There is no command to stop the resizing. It is paused automatically when a sequential access is performed.
- The D-pointer does not reflect the new modulo. The **nt_file-info** command reports the new modulo correctly (see the *D3 Reference Manual* for more information).
- The number of overflow frames reported by the File Manager is actually the combination of the number of frames used by large items and the number of frames used by overflowing groups. It is therefore pessimistic. This version corrects this problem, but a full save of the FSI is required to correct these counters. It is not a critical problem, but it makes determining poorly sized files difficult. A new File Manager option allows finding poorly sized files more accurately.

1. Right-click the **Tables** node in the database to be examined from the **Domain Explorer** and select the **Check Overflow Files** option.

The Files Overflow dialog box displays. Files in the database are represented with a list of color coded icons. Files marked yellow and red may need resizing. See [Checking overflow](#) for more information.



2. Click the file to be resized.

The **General** tab of the of the Properties dialog box displays.

Tip: This property information can also be accessed for any table by right-clicking the **Tables** node from the **Domain Explorer** and selecting the **Properties** option.

ORDERS

General Usage Statistics

Server ☒ Case Insensitive

Path

Modulo Frame size (K)

Created on 1/10/2014 11:32 By

Table Id 106

Compression Trigger K

3. Type a new modulo in the **Modulo** text box and click the **Resize** button.



Note:

- Even though the Hash Test dialog box allows testing a non-prime modulo, the resizing process always adjusts to the highest and closest prime number before starting the resizing.
- The Resize Table dialog box can also be opened from the Hash Test dialog box. See [Running a hash test](#) for more information.

The Resize Table dialog box displays.

Resize Table

Database MVDEMO

Table ORDERS

Current Modulo 3

New Modulo

Table Check Options

☒ Check ☐ No Check

☐ Check and Fix

Ready

4. Select the appropriate **Table Check Option**:

Option	Description
Check	(Default) Automatically performs the file error check. If any file errors are encountered, the user is notified of the error and the file resize process is not initiated.
No Check	Disables the automatic file check process.
Check and Fix	Attempts to correct any file errors encountered. If errors are found and are successfully fixed, the user is notified and file resize process is started. If errors are found and <i>cannot</i> be successfully fixed, the user is notified and file resize process is not initiated.

- Click the **Start** button.
The file resize process is initiated. Once the process starts, the status bar displays the progress of the resize. When the status bar reaches the end of the status bar box and **Ready** once again displays above the status bar, the resize process is complete.
- Click **Close**.

Related tasks

[Listing or clearing record locks](#)
[Editing General table properties](#)
[Viewing Usage table properties](#)
[Displaying Statistics table properties](#)
[Listing servers](#)
[Displaying Flash shared code](#)
[Creating indexes](#)
[Checking overflow](#)
[Running a hash test](#)

Environment variables

Environment variables enable you to use symbolic names for elements of the file system structure. Server names and path specification can be encoded using environment variables. The advantage of using variables instead of hard coded names is that it enables you to move databases and tables to another system or location.

An environment variable is represented by:

name

where *name* is a case-insensitive symbolic name.

The content of the variable can be any string. In addition, an environment variable can use (refer to) another environment variable.

Reserved Variables

The following variables are reserved:

Variable	Description
MDS	Replaced by the computer name of the MDS server.
MD	Replaced by the path to the current database MD directory.

Variable	Description
ROOT	Replaced by an empty string to specify the D3 file system root.

Variable Locations

Environment variables are stored in the Catalogue,Environment table in the dm database on the MDS server.

Variable Types

Variables are global to the domain and are either path or server environment variables.

Related concepts

[File Manager layout](#)

[Domains](#)

[Databases](#)

[Tables](#)

[Records](#)

[Additional File Manger operations](#)

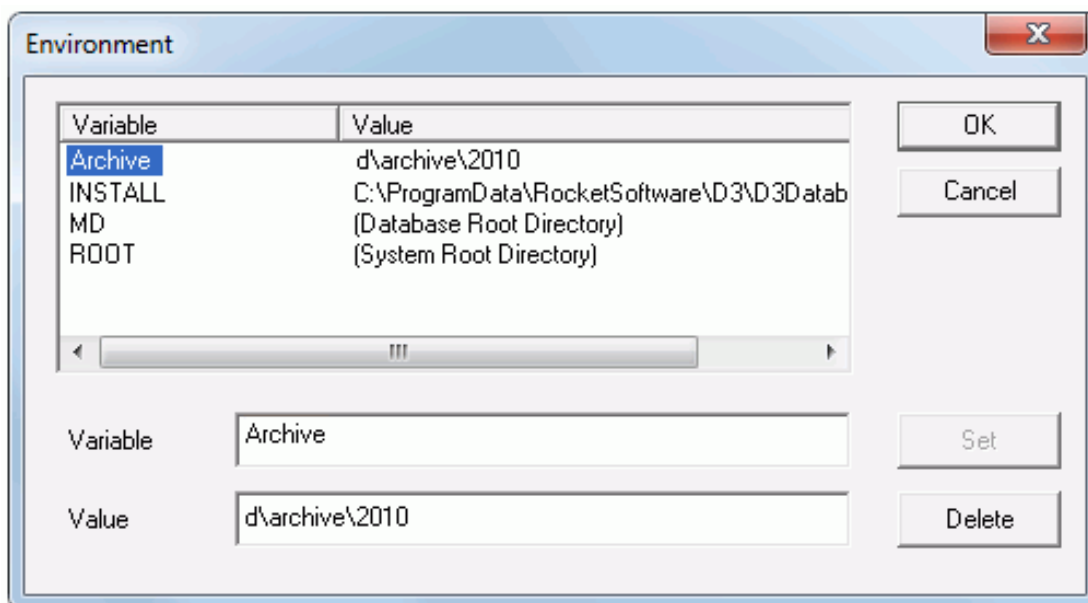
Related tasks

[Editing File Manager options](#)

Path environment variables

The server environment variables isolate the file system from the actual computer names. Wherever a server name is expected, the **Env.** button enables you to select, create, or delete server variables.

For example, in the dialog box below, Archive is a variable name that represents the server d:\archive\2010. Creating a database using this variable name enables you to move the database by simply changing the value of the variable.



See [Server environment variables](#) for information on setting up Server Environment Variables.

Functions of the Environment dialog box

The list box lists the variables and their values.

Function	Instruction
To select a path	Select the entry in the Variable list and click OK .
To add a new variable	Enter the variable in the Variable text box and the value in the Value text box, then click Set .
To modify a variable value	Enter the new value in the Value text box, then click Set .
To delete a variable	Select the variable from the Variable list , then click Delete .



Note:

- MD is a predefined variable that cannot be changed and refers to the database root directory. This value is often the default and normally does not need to be selected.
- ROOT is a predefined variable that cannot be changed and refers to the D3 File System root directory. At run time, it is replaced by the root directory specified at install time on the specified server.

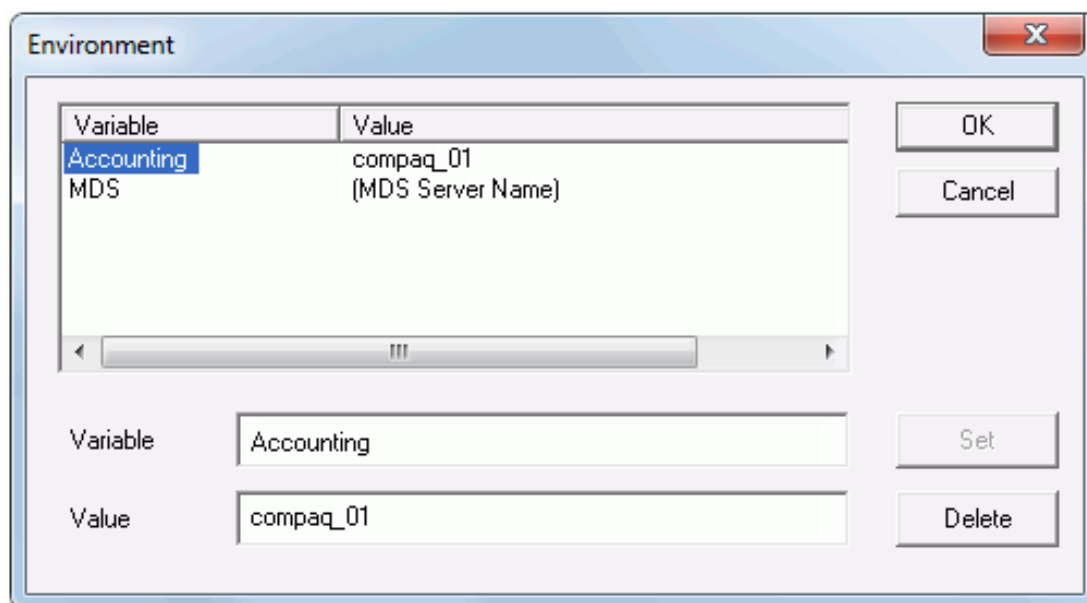
Related concepts

[Server environment variables](#)

Server environment variables

The server environment variables isolate the file system from the actual computer names. Wherever a server name is expected, the Env. button enables you to select, create, or delete server variables.

For example, in the dialog box below, Accounting is a variable name which represents the server Compaq_01. Creating a database using this variable name enables you to move the database by changing the value of the variable.



See [Path environment variables](#) for information on setting up Path Environment Variables.

Functions of the Environment dialog box

The list box lists the variables and their values.

Function	Instruction
To select a server	Select the entry in the Variable list, then click OK .
To add a new variable	Enter the variable in the Variable text box and enter the value in the Value text box, then click Set .
To modify a variable value	Select the variable from the Variable list and enter the new value in the Value text box, then click Set .
To delete a variable	Select the variable from the Variable list then click Delete .

 **Note:** MDS is a predefined variable, and cannot be changed.

Related concepts

[Path environment variables](#)

Performance monitoring tools

D3 is a Windows application and can be monitored by standard Windows Performance Monitoring Tools.

In addition to these tools, D3 Performance Monitoring Tools can be added to the Windows Performance Monitor to obtain statistics specific to D3.

Related concepts

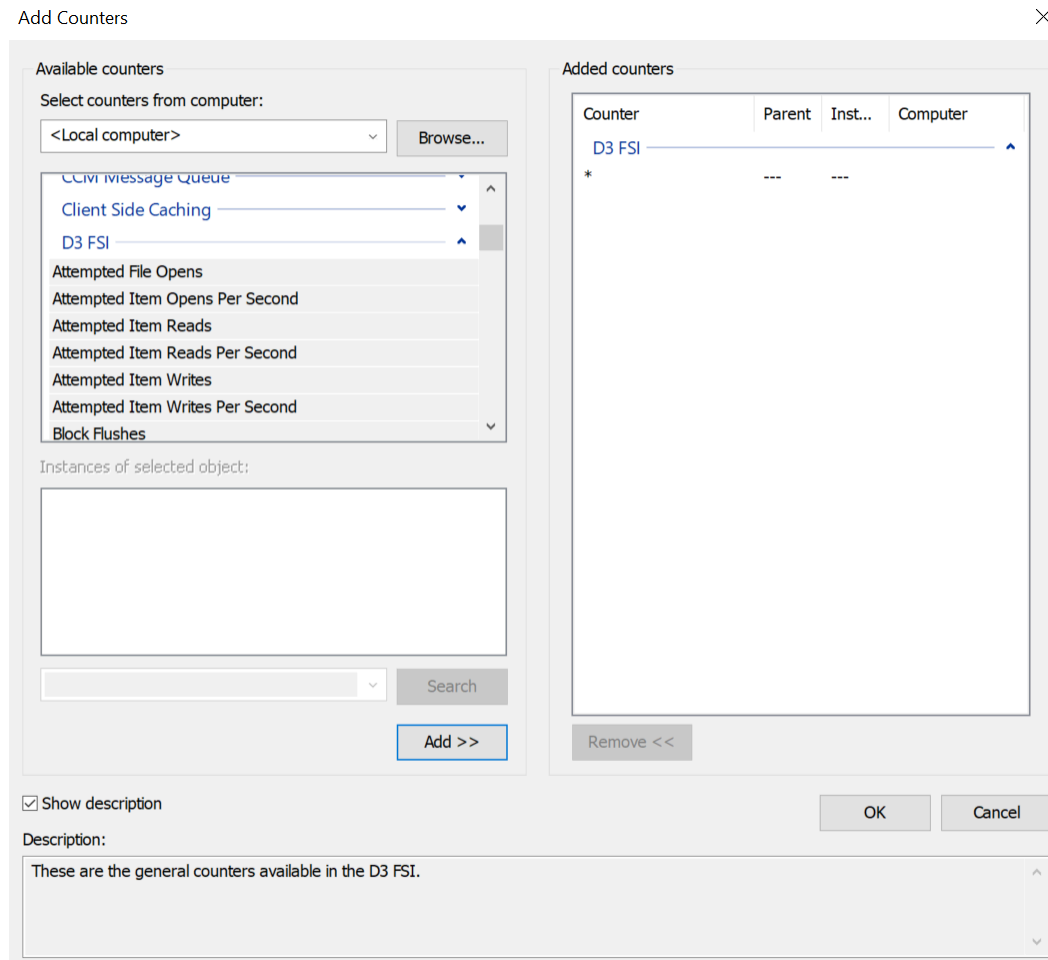
[D3 File Manager](#)

Windows performance monitoring tools

The Windows performance monitor enables you to control a variety of system behaviors to ensure the best performance and identify performance problems.

This topic assumes that the reader is familiar with the general operations of the Windows performance monitor `perfmon.exe`, which can be found in the administrative program group.

In the Add Counters dialog box the Available Counters list includes a D3 FSI entry. One or more of the D3 FSI counters can be added for monitoring. Checking the Show description check box will provide information about the currently selected counter.



This topic describes only some of the system parameters that can be useful on a server running D3. See the Microsoft Windows System Guide for a complete description of this tool.

Memory

Page Faults/sec	A high value (more than 10-50 times a second) indicates that the physical memory is not sufficient to satisfy the demand and that paging is occurring.
-----------------	--

Logical Disk

Disk Reads/Sec	Represent the total disk accesses.
Disk Writes/Sec	Total disk writes per second.

Process

% Privileged Time	This is the time spent performing system tasks. This time should always be as low as possible. A consistent level above 10% may indicate hardware problems (for example, a serial device generating interrupts because a cable is not connected) or abnormal locks (for example, a poorly sized D3 table accessed often by many threads).
% User Time	This is the time spent performing user tasks. It is normal for this counter to be always very high (unless the system is idle...). Note that some OLE applications (for example, Microsoft Visual SourceSafe) consume 100% of user CPU, even when idle, making this counter difficult to use. The Task Manager gives a per-task CPU usage and is more convenient.
Handle Count	For each D3 table, at least two Windows files are opened per section. If the table overflows or if there are indexes, more Windows files can be opened. On average, the number of file handle divided by 3 will give an approximate number of D3 opened tables. For a large application, it is not uncommon to have thousands of files opened. This number may therefore be high. A D3 opened table occupies roughly 1.5 KB in memory. This may help in diagnosing memory problems. For example, to open and access constantly 10,000 tables, up to 15 MB of server memory are required just for the file tables.
Page Faults/sec	This counter represents the page faults triggered by the specified process itself.
Thread Count	The Virtual Machine Environment has one thread per user. It is therefore normal to have many threads running for the D3VME.EXE applications. A FSI Server typically has fewer threads (less than 30). Windows is creating and deleting server threads as required.

Related concepts

[D3 performance monitoring tools](#)

D3 performance monitoring tools

You can use the `%getperfcount()` function from FlashBASIC to obtain these performance counters.

See *File System Memory Usage* in the *Administrators Guide* for a brief overview of host memory, which aids in understanding the following counters.

D3 File System

Cache Hits/s	Represents the number of times per second a given page was found in the mapped pages.  Note: A mapped page is not necessarily in physical memory since Windows can have flushed the file cache. This situation should only arise in extreme cases.
Cache Misses/s	Represents the number of times per second a given page was not found in the mapped pages. Each cache miss does not necessarily trigger a disk I/O, because the missing page can still be in

	the Windows disk cache.
Flush/s	Represents the number of times per second an explicit disk write was triggered. Disk writes are performed by a background thread controlled by the registry entry FlushPeriod.  Note: This does not include the cache flushes Windows may do on its own.
Group Locks/s	Represents the number of times per second a thread failed to acquire a group lock. Group locks are internal locks set when a table is modified. High group lock counts indicate that small tables are accessed by many threads. A solution might be to increase the modulo of these tables.
Item Locks/s	Represents the number of times per second a thread failed to acquire an item lock.
Mem Full/s	Represents the number of times per second the server failed to map a page in memory because all entries are in use. This counter should always be null. If this counter becomes non-null, it is an indication that the memory is too small for the application. If the physical memory is already above 2 GB of memory, splitting the database across two or more servers could help solve this problem.
Open/s	Represents the number of times per second a table was opened. If this number is consistently high, it may be an indication of an application problem that keeps opening files and never closes them.  Note: The VME caches file descriptors. As a result, it does not hurt to leave files open. Generally, Visual Basic OLE applications do not have access to such a cache. Therefore, a rogue Visual BASIC application that loops opening files can create this kind of problem.
Ovf Add/s	Represents the number of times per second a table was extended. If this number is consistently not null, it is an indication that one or more files are severely undersized and are in the process of being populated.
Ovf Read/s	Represents the number of times per second poorly-sized tables are accessed. If this number is consistently not null, it is an indication that one or more files are severely undersized and are accessed heavily. A high ratio (Ovf Read / (Read + Write) is an indication that there are one or more frequently used files that are poorly sized. It is important to locate these files and resize them. The D3 File Manager has a tool to scan a database for poorly sized files and, currently, they can be resized using the nt_resize utility from the VME (see the <i>D3 Reference Manual</i>). Determining that the ratio is high depends on the capability of the system. A 50% ratio would start to be problematic. Approaching 100% will likely degrade performance.
Read/s	Represents the number of times per second a record was read.  Note: This does not necessarily imply a disk I/O.
Writes/s	Represents the number of times per second a record was written.

	 Note: This does not necessarily imply a disk I/O.
--	---

Related concepts

[Windows performance monitoring tools](#)

System structure

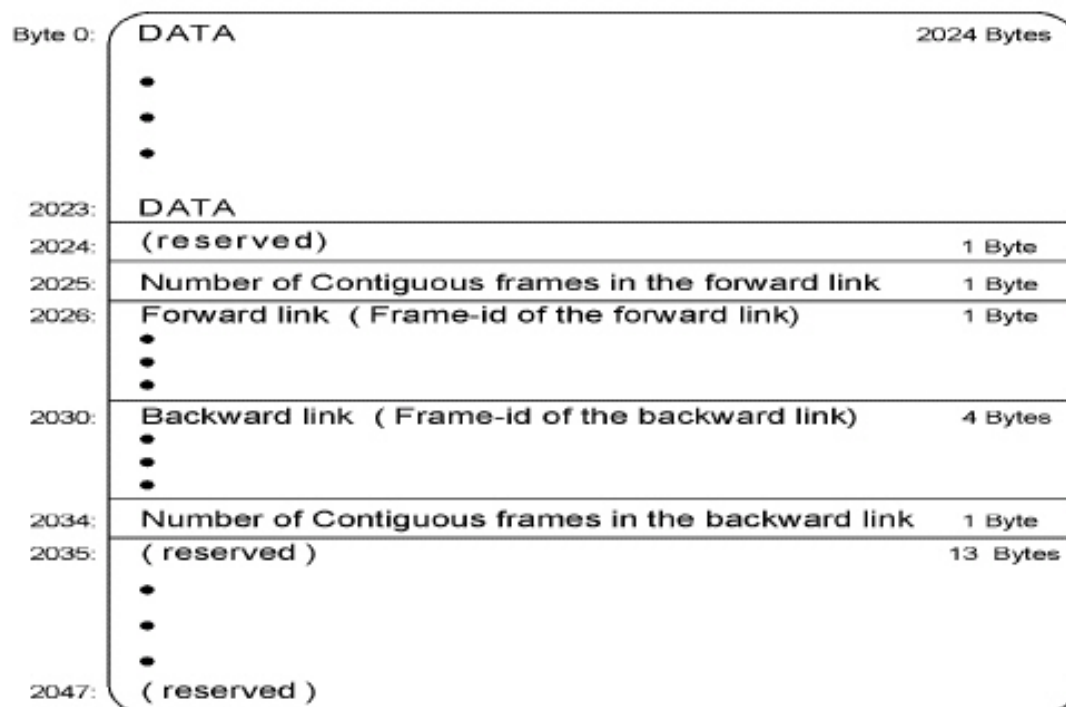
For the reader who is interested in how the D3 system sees the disk and how it arranges things, here is a look at the disk layout.

- Idea of a frame
- What it is
- How frames are used to store data
- Layout of the disk

Frames

A frame holds data and system overhead. Considering a frame size of 2048 bytes, 48 bytes are reserved for system use and 2000 are available for data.

Here is a 2 KB frame:



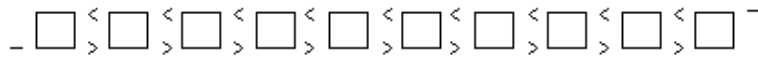
Each frame has a unique address or FID.

Related concepts

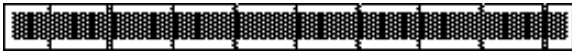
[Major parts of the system](#)

Linking frames

Forward and backward link bytes are used to maintain a data structure called a linked list, which can be envisioned as links in a chain. Each link is attached to two other links—one in front and one behind.



By linking frames together in this fashion, D3 can store data larger than the capacity of a single frame even though the linked frames are scattered around the disk drive or drives.



Note that it is faster to grab *nn* contiguous frames than it is to grab one frame, read the forward link, get the next frame, read it, and so on.

Related concepts

- [I/O devices](#)
- [Monitor](#)
- [Memory management](#)
- [Frame requests and frame faults](#)
- [Concept of a process](#)
- [Process identification blocks](#)
- [User pibs and phantom pibs](#)
- [Scheduler](#)
- [Scheduling algorithm](#)
- [Super high priority process](#)

I/O devices

The different hardware devices (serial ports, disk drives, tape drives, and so on) are referenced through the D3 Monitor that passes the I/O requests to the host device drivers.

Related concepts

- [Linking frames](#)
- [Monitor](#)
- [Memory management](#)
- [Frame requests and frame faults](#)
- [Concept of a process](#)
- [Process identification blocks](#)
- [User pibs and phantom pibs](#)
- [Scheduler](#)
- [Scheduling algorithm](#)
- [Super high priority process](#)

Monitor

The Monitor is the control portion of D3. It handles the hardware resources of the machine or passes requests to the host operating system and services requests for these resources from the D3 Virtual Machine.

The Monitor performs:

- Terminal I/O
- Printer I/O
- Tape I/O
- Disk I/O
- Memory Management /Disk Caching

The Monitor's controller merely passes these requests from the Virtual Machine directly to the host operating system to be serviced.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Memory management](#)

[Frame requests and frame faults](#)

[Concept of a process](#)

[Process identification blocks](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Scheduling algorithm](#)

[Super high priority process](#)

Memory management

The D3 Virtual Machine is based on a system that uses Virtual Memory. Virtual Memory is like a vast source of RAM. The Monitor manages the hardware RAM as a series of buffers or blocks of RAM that hold one frame each. Therefore, after the Monitor is loaded in RAM, any additional RAM is used as buffers.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Frame requests and frame faults](#)

[Concept of a process](#)

[Process identification blocks](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Scheduling algorithm](#)

[Super high priority process](#)

Frame requests and frame faults

D3 services requests for frames by the D3 Virtual Machine as follows. When the Virtual Machine requests a frame, the Monitor scans through all the frames currently in RAM to see if the requested frame already exists in a buffer. If it does, the Monitor hands the buffer over to the Virtual Machine. If it does not, a frame fault condition occurs. The Monitor goes to the disk, retrieves the frame, and places it in a buffer. It then releases the buffer over to the Virtual Machine for its use.

- How is the buffer (frame) written back out to disk?

When the Virtual Machine releases a buffer, the Monitor has the option to write the buffer back out to disk at its leisure. Usually, this happens only when the Monitor needs the buffer for some other purpose such as another frame.

- Which frame should the Monitor dispose of?

The Monitor employs a method of *least recently used* buffers to decide which ones to write out to disk. The newer the frame, the more likely it is to remain in RAM. The older the frame, the more likely it is demoted back to the disk. This scheme permits frequently used frames to stay in RAM due to the probability that they are needed again shortly.

The system RAM capacity determines the number of frames that can remain in RAM. The larger the system RAM, the greater the number of frames that can remain in RAM, requiring less frame shuffling and improving system performance. The more active processes running on the system, the more likely frames are requested that are not in RAM. *All* the buffers are flushed to disk before the system stops. The D3 flusher also is active at predetermined intervals, or whenever system activity lets up.

All of this means the information in RAM is *different* from the information on disk. The D3 file system is protected from corruption by mechanisms such as Halt Tolerance, transaction logging and transaction bracketing.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Memory management](#)

[Concept of a process](#)

[Process identification blocks](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Scheduling algorithm](#)

[Super high priority process](#)

Concept of a process

A process in D3 is a program, subroutine, or job that requires the resources of the D3 system to perform some kind of manipulation of data, and perhaps requires input and produces output. The technical definition of a process is a continuing operation on a set of functional elements (frames).

An internal D3 system routine is a process. A logged on user is a process. A program that a user runs can be considered a process, too. D3 can have multiple processes running at any time to facilitate use of the system.

The maximum number of processes is determined by the system configuration for each site. Processes are normally attached to terminals or personal computers running terminal emulation programs, but also may be attached to other devices such as bar code readers, or may not be attached to any device at all (background or *phantom* processes). The D3 Monitor controls these processes and services their I/O requests.

A typical interactive process deals with only one terminal. Each process uses the CPU for a certain amount of time called a timeslice and runs D3 Virtual code until:

- all of its allocated time is used.
- process executes a D3 Monitor call.
- disk request frame fault occurs.

Because a D3 Monitor call or frame fault service would use much CPU time that could be used for other processes, system efficiency is better served if the current process is suspended and another process is granted the use of the CPU.

As processes run, eventually a frame is requested that is not in memory. At this point, a frame fault condition occurs. The current process is suspended and the Process Identification Block (PIB) is marked accordingly (waiting for disk). The Monitor goes off to get the frame. Rather than wait for the disk, the Monitor activates another process that was ready and waiting, and lets it execute for its timeslice. Eventually, the Monitor gets a signal from the disk drive that the frame is in and the PIB status of the first process is changed to ready and waiting.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Memory management](#)

[Frame requests and frame faults](#)

[Process identification blocks](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Scheduling algorithm](#)

[Super high priority process](#)

Process identification blocks

The Pib is the ID badge for each user. It tells the system all the internal administrative information it needs to track, such as what account the user is in, what kind of permissions are allowed, and how much time has been used.

Some of the D3 commands display PIB status bytes, which are described below:

Pib Major Byte (XX...)

0		Sleeping	Page Fault	Echo not Enabled	I/O Bound	Outputting	Inputting	Active
1	Activatable			Full Duplex				Comatose
	7	6	5	4	3	2	1	0

Pib Minor Byte (...XX)

0		Sleeping	Page Fault	Echo not Enabled	Debug Attention Codes			
1	In Vir. Debugger	XOFF in effect	Typeahead enabled	XON/XOF supported	Debug Attention Codes			
	7	6	5	4	3	2	1	0

Examples

3Fxx = Sleeping

5Fxx = Page Fault (Waiting for Disk I/O)

7Bxx = Terminal is outputting

FBxx = Terminal is outputting

FDxx = Terminal is inputting

7Fxx = Active, or Ready to Go

FFxx = Active, or Ready to Go

Pibs need additional space to store information about the processes it is running, and a place to store the process data and variables. This area is called the Process Control Block (PCB).

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Memory management](#)

[Frame requests and frame faults](#)

[Concept of a process](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Scheduling algorithm](#)

[Super high priority process](#)

User pibs and phantom pibs

The two kinds of processes that vie for a place in the system resources queue are user processes and phantom processes. The user process is associated with a serial port or a networked device and is expected to require some kind of input/output through that port. An example is a simple data entry program run on a dumb terminal.

The phantom process usually is not associated with a serial port and is *not* expected to require any kind of input/output through that port. A typical example of such a process is the Spooler, which handles print jobs. Another example is the Scheduler, which determines when jobs get processed.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Memory management](#)
[Frame requests and frame faults](#)
[Concept of a process](#)
[Process identification blocks](#)
[Scheduler](#)
[Scheduling algorithm](#)
[Super high priority process](#)

Scheduler

In a single user system such as MS-DOS, the user has all of the computer system available all of the time. In a multiuser system, resources must be shared with many users. This is accomplished by giving one user the entire resources of the system for a short time, then stopping the job in process, saving the current state of the job, loading another user's job, and continuing. Some time later, the first user's job is recalled and processed a bit more. Eventually, each user's job gets done.

Slicing up the CPU time and dividing it among several users is called *time sharing*. Management of the CPU and each user's job is called *scheduling*.

Information about each process, user, or job is stored in a PIB. Along with the PIB is a PCB, which contains the current state of the job in progress. All the jobs (pibs) wait in line to be processed. This line is a queue, similar to the checkout line at the supermarket.

Related concepts

[Linking frames](#)
[I/O devices](#)
[Monitor](#)
[Memory management](#)
[Frame requests and frame faults](#)
[Concept of a process](#)
[Process identification blocks](#)
[User pibs and phantom pibs](#)
[Scheduling algorithm](#)
[Super high priority process](#)

Scheduling algorithm

The Monitor maintains a queue of processes currently in the system. Only one job is active and the others are paused, awaiting the services of the CPU. The processes in the queue are maintained in increasing order of priority, with the head of the queue having the highest priority.

A process priority is determined by two factors:

- Total processing time expected (based on the amount of CPU processing time and disk I/O already done by that process)
- Weighting

Interactive processes are favored by increasing their priority. As a process outputs characters to the terminal, it

migrates up the priority queue. When a process receives terminal input, it is moved to the head of the queue for immediate activation.

When the number of units of a resource consumed reaches the weighting factor, the process is moved up or down in the priority queue one position and the resource unit count is reset to zero. As a process consumes system resources (CPU time and disk reads), it gets pushed down the queue. If a process is currently waiting for a system resource to become available, for example, a frame fault, the process is said to be roadblocked, and is pushed down one position in the queue.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Memory management](#)

[Frame requests and frame faults](#)

[Concept of a process](#)

[Process identification blocks](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Super high priority process](#)

Super high priority process

One process in the System may be designated the Super High Priority Process (SHPP) in order to receive special handling in the process scheduling mechanism. The SHPP has top priority to all system resources, allowing it to run without interference from other processes. This is implemented for BISYNC communications.

The following rules apply to the SHPP:

- SHPP is reactivated when its CPU processing time quantum is used up.
- Frame faults for the SHPP are processed as soon as the necessary disk is available.
- Disk interrupts, which signal completion of a disk read for the SHPP, cause the SHPP to be activated immediately.
- Interrupts from devices with addresses in the range 0x10—0x13, which are for the SHPP, cause the SHPP to be activated immediately.
- Voluntary RQMs (process is suspended by terminating its timeslice) by the SHPP allow two other processes to run before the SHPP is activated again.

Related concepts

[Linking frames](#)

[I/O devices](#)

[Monitor](#)

[Memory management](#)

[Frame requests and frame faults](#)

[Concept of a process](#)

[Process identification blocks](#)

[User pibs and phantom pibs](#)

[Scheduler](#)

[Scheduling algorithm](#)

Major parts of the system

This section describes the boot record, ABS, process control blocks and workstations.

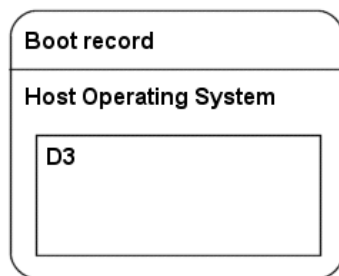
Related concepts

[Frames](#)

Boot record

When a machine is turned on, it goes through a Power On Self Test, POST, and reads the Master Fixed Disk Boot Record (MFDBR) from disk drive zero to get the proper boot partition. It goes to that partition, loads the system and executes. After the host is powered up, the D3 Monitor can be invoked.

Hosted D3



Related concepts

[ABS](#)

[Process control blocks and workspaces](#)

ABS

The ABSolute (ABS) addressed frames file contains the D3 virtual code and is the first few hundred frames of the D3 disk area.

Lock Tables

The first few frames of the ABS file are reserved for the *Lock Tables*, which begin at FID one. The number of frames reserved for lock tables is 18 but additional frames may be linked-in from overflow if necessary.

Locking Data

Since D3 is a multiuser system, special arrangements must be made so that a file is not concurrently modified by several users. To accomplish this, D3 uses *locks*. Individual records can be locked, as well as an entire file. In addition

to locking, restrictions can be implemented for a file for each user's retrieval (reading) and update (writing) abilities.

D3 stores data in *files*. The files are evenly divided into *groups*. Groups contain one or more *items* (or *records*). When an item is updated, the new information is appended to the end of the group, and the old, outdated information is marked as deleted. The system then moves all the items forward, squeezing out any deleted items. When this happens, the items are said to be *shifting* in the group. D3 can lock data at the file, group or individual item level.

Group Update Lock

When the entire group is locked, no other process is allowed into the group for reading or writing. The group is allowed to shift if there are no Group Retrieval Locks or Group Read-Only Locks on the group. This means that writes are allowed in the group to append a new item, delete an existing item, or to replace an existing item.

Group Retrieval Lock

When a Group Retrieval Lock is placed on a group, other processes are still allowed to lock the group, but the group is not allowed to shift. This means more than one Group Retrieval Lock is allowed per group. When placing Group Update Locks on a group with Group Retrieval Locks already in place, no more Group Retrieval Locks are allowed. Since the group cannot shift due to having Group Update and Group Retrieval Locks, any writes have to be appended to the end of the group. After the last lock is released the group is cleaned up.

A possible scenario would be a Retrieval Lock followed by an Update Lock from another process. The second process might file an existing item in the group. The group cannot shift, therefore, the new item is tacked on to the end of the group and the old copy is marked as deleted. Later, when the final lock is released, the deleted item is removed and the group shifted.

Group Read-Only Lock

When a Read-Only Lock is placed on a group, no Group Update Locks are allowed. The group is not allowed to shift. However, other Read-Only Locks and Retrieval Locks are allowed on the group.

Item Lock

The record (item) is locked and all other records in the file are accessible.

- Group Lock is placed on the group to prevent shifting.
- Item Lock is placed on the item, and the Group Lock is released.

Item Locks are used by the FlashBASIC `readu`, `readvu`, and `matreadu` statements and the Update Processor.

ABS Symbolic Names

The ABS, or actual D3 System Virtual code, is next. D3 references frames by a symbolic name, and the system refers to a cross-reference table. The FID in D3 is referenced as:

```
process{subprocess}:offset
```

The ABS file contains information on what is in each frame.

ABS Processors

D3 has its different tasks broken into separate subprograms called *processors*, the code of which resides in the ABS file. These processors perform the various functions of the D3 system. Some of these processors are listed below:

Processor	Description
AQL	Performs nonprocedural queries against the database.
Macro, Proc, Menu	Front-end tools to organize processes.
Phantom Processor	Handles user-like jobs that do not have terminal I/O, thus freeing terminals.
FlashBASIC	Compiles and runs FlashBASIC programs.
Scheduler	Schedules user processes for a time-sharing environment.
Spooler	Handles output to the printers.
TCL	Handles input from terminals and nondatabase commands.
Update Processor	Full screen editor permits direct access to update and retrieve data within files.
Other processors:	• Bootstrapping and cold start • Administration of user accounts • Handling of terminal characteristics • Level pushing • Interuser message communications • System accounting • Mathematical functions • Block printing • Virtual memory dumping • Tracking file statistics • File save/restore functions

Processor	Description
	• Magnetic tape unit functions

Related concepts

[Boot record](#)

[Process control blocks and workspaces](#)

Process control blocks and workspaces

PCBs are associated with each job running on the system. They tell the system all the variables, what they currently are, and which resources are allocated to it. The system saves this information during round robin timeslices, so it can restart jobs.

Each PCB uses about four frames beyond the ABS. More is linked as necessary. Since the workspace is a series of linked frames that get attached to the user as he logs on, it follows that available frames are required in order to log on. There is always the danger that the overflow table is exhausted, and *no one*, not even the console, is able to log on, rendering the system useless. A *reserve overflow system* is used to wrap up processes and recover disk space.

Sysbase

Following the D3 system frames (locks, ABS, and User Workspaces) are the rest of the file system. This area is called Sysbase or Base of the D3 file system.

File System

The D3 file system uses the remainder of the disk. This may consist of multiple collections of related files called accounts, or part of the pool of available frames called overflow.

Maxfid

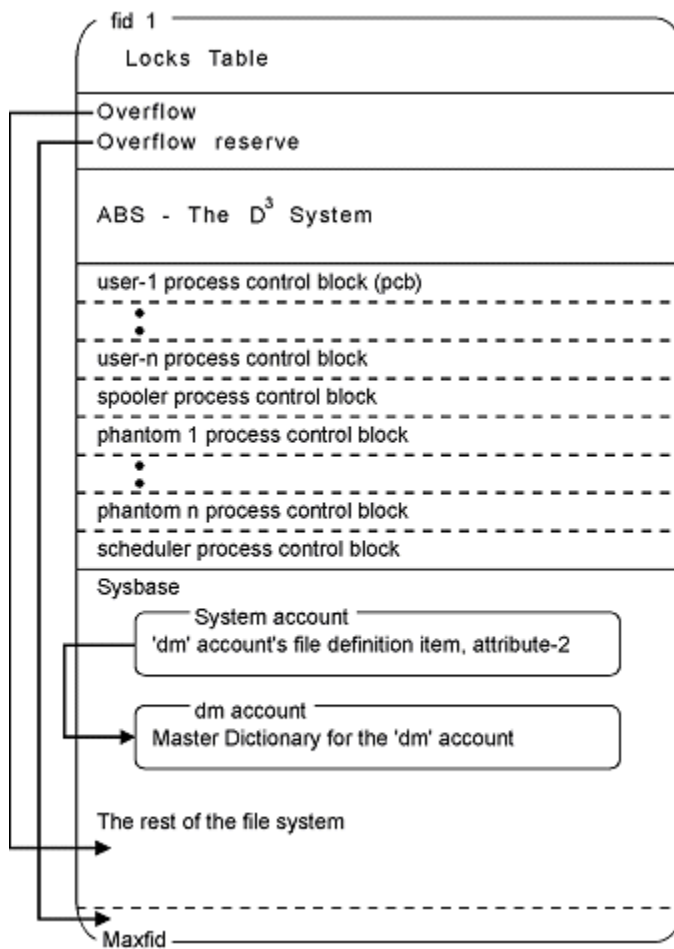
The maximum limit of the frame addresses (essentially, the end of the D3 file system) is kept in a special value called maxfid, or maximum FID. This is the largest frame address available on D3. The number is dependent upon how much disk space a system has, and what already has been allocated. It is possible to allocate additional disk space to a system, assuming more is available, though this requires that the system be shut down and rebooted.

Physical Limitations

The D3 data structure resides either in a separate partition from the host and is accessed by D3 or as a file system under control of the host.

For UNIX: There is a file that holds a table of filenames where the D3 file system resides. Currently there are four table entries available, so disk capacity is limited to the handling capacity of the host operating system for those four table entries. On UNIX, the limit is 4,000 GB.

D3 System Pictorial Diagram



Related concepts

[Boot record](#)

[ABS](#)

Proc

Proc, short for procedure, allows executing a series of TCL and other commands from a single word command. Everything that can be done in TCL can be done in Proc. Procs also are used to avoid repetitively typing the same series of commands, interact with the user, and to test or verify input information from the user.

Proc has the following features:

- Interactive terminal prompting
- Four variable length I/O buffers
- Extensive I/O and buffer control commands
- Parameter passing between the buffers
- Free-field and fixed-field character manipulation
- Relational character testing
- Pattern matching
- Conditional and unconditional branching
- Optional command labels
- User defined subroutine linkage
- Inter-Proc linkage

Use of the PROC processor is much the same as using a JCL on mainframe systems. However, Proc language is more versatile since it can interactively prompt the user for responses and can branch on conditional statements. In addition, Proc can verify the user's input.

Procs are executed interpretively by the PROC processor; they do not need to be compiled. Procs are stored as dictionary items, or as items in a data file. Procs that are in the current master dictionary can be executed by typing the name at the TCL prompt. While the Proc definition must be in the master dictionary, the actual body of the Proc may reside as an item in any dictionary or data file. Attribute 1 of a Proc is always the pq code.

Once a Proc is invoked, it remains in control until it completes or until it is explicitly terminated. A Proc can temporarily relinquish control to another processor, such as a subroutine or the Update Processor, but it regains control when that process ends. The system only returns to TCL when the Proc is deliberately terminated or when the Proc ends.

Since FlashBASIC can do everything Proc can, and much more, Proc is less used today than it was in earlier implementations.

Structure of the PROC processor

The PROC processor uses four variable length I/O buffers—a primary and secondary input buffer and a primary and secondary output buffer. These buffers are the stack. Basically, the function of a Proc is to move data from the input buffers to the output buffers. At any time, one of the input buffers is considered active, and so is one of the output buffers. Buffers are selected as active by certain Proc commands.

The primary input buffer contains the Proc name and optional arguments, if any. The primary output buffer is used to build the command that ultimately is submitted to TCL for processing. The secondary input buffer contains data subsequently entered by the user in response to an **IN** command. Usually, the data in this buffer is tested before it is transferred to the secondary output buffer. When all data has been moved to the secondary output buffer, control is passed to the primary output buffer via a **P** or a **PP** command. The command that resides in the primary output buffer is executed at the TCL level. The data in the secondary output buffer, if any, is used to feed other processors, such as AQL or the Update Processor. When the process is finished, control returns to the Proc and new data can be moved to the output buffers.

Related reference

[Proc command format](#)

[Proc command summary](#)

Proc I/O buffers

The main operation of a Proc is to move data from one of the two input buffers into one of the two output buffers to perform the desired TCL commands.

When a Proc starts or when control returns to a Proc, the primary input buffer is selected automatically as the active buffer. It contains the Proc name and any additional arguments entered when the Proc was invoked. The contents of this buffer remain the same throughout the execution of the Proc, unless specifically modified.

The secondary input buffer contains data subsequently entered by the user. The data in this buffer is volatile and overwritten by subsequent input commands. Usually, the data in this buffer should be tested for correctness, then moved to the secondary output buffer.

Data from several system processors, such as the spooler, are also placed in the secondary input buffer. For example, the last hold file entry number is placed in this buffer. The secondary input buffer is a very temporary entity, and if its contents are to be used, they should be used immediately subsequent to the execution of the processor that loaded the buffer.

The primary output buffer is used to construct a single TCL command. A carriage return is placed automatically at the end of the command in the buffer when the **p** command is issued.

The secondary output buffer (stack) is used to hold zero or more lines of data, which are used by other processors called by the Proc generated TCL command. Each request for terminal input is satisfied with a line of data from the stack. In the event that the called processor requests more data than exists in the stack, data is requested from the terminal. Note that the stack is really a first in, first out queue. Each line of data must be terminated by a carriage return and must be placed in the stack explicitly via an **h** command.

After all data has been moved to the buffer, control is passed to the primary output buffer via the **p** command. The command stored in the primary output buffer is executed at the TCL level and the data in the secondary output buffer, if any, is used to feed processors such as AQL. When the process ends, control returns to the Proc at which

time new data may be moved to the output buffers.

Related concepts

[Parameters](#)

Parameters

Data is moved between buffers in terms of parameters. A parameter is defined as a string of characters surrounded by spaces or single quotes and residing in one of the buffers. To keep track of the parameters, each buffer has a pointer that points to the current position in the buffer.

The end of the parameter is designated by the first space encountered, unless the entire parameter is enclosed in single quotes, in which case the entire string in quotes is moved. The following Proc example displays the contents of the mds or system file.

```
A Proc 'show-me':  
000 show-me  
001 pq  
002 hlist only  
003 a2  
004 p
```

Type **show-me mds** to execute this Proc. This invokes the Proc show-me, and puts the string show-me mds in the primary input buffer.

Primary Input Buffer	Primary Output Buffer
show-me mds	
Secondary Input Buffer	Secondary Output Buffer

The **h** command puts the text **list only** in the primary output buffer:

Primary Input Buffer	Primary Output Buffer
show-me mds	list only
Secondary Input Buffer	Secondary Output Buffer

The **a** command picks up the second parameter in the input buffer from the primary input buffer and places it in the primary output buffer.

Primary Input Buffer	Primary Output Buffer
show-me mds	list only mds
Secondary Input Buffer	Secondary Output Buffer

Finally, the last line of the Proc (the **p** command) submits the primary output buffer to TCL for processing.

Related concepts

Proc command format

The general syntax for Proc commands is presented below.

Syntax

```
{ label } command { argument... }
```

Parameters

Parameter	Description
<i>label</i>	A numeric statement label.
<i>command</i>	A one or two character Proc command. See Proc command summary for a list of available commands.
<i>argument</i>	Command arguments. Arguments are command specific. Please refer to the specific command to find out about the command arguments.

Related concepts

[Structure of the PROC processor](#)

Related reference

[Proc command summary](#)

Proc command summary

The table below lists the supported Proc commands.

Command	Description
a	Move data argument from input buffer to output buffer.
b	Back up input pointer.
bo	Back up output pointer.
c	Comment.
d	Display an input buffer to the terminal.
f	Move the input pointer forward.
go	Go to a label.
h	Move text string to an output buffer.
if	Conditional statement.
ih	Move a text string to an input buffer.

Command	Description
ip	Inputs from the terminal to an input buffer.
is	Inputs from the terminal to the primary input buffer.
it	Inputs from tape to the primary input buffer.
o	Output a text string to the terminal.
p	Executes a command in the primary output buffer.
ph	Executes a command in the primary output buffer, but suppress all terminal output for the command.
pp	Executes a command in the primary output buffer, but displays the output of the output buffers.
pw	Executes a command in the primary output buffer, but waits for a user response before continuing.
px	Executes a command in the primary output buffer, but returns to TCL, not Proc, after processing.
ri	Reset (clear) the input buffers.
ro	Reset (clear) the output buffers.
s	Sets the position of the input buffer and optionally selects the primary input buffer.
sp	Select the primary input buffer.
ss	Select the secondary input buffer.
ston	Select the secondary output buffer (stack on).
stoff	Select the primary input buffer (stack off).
t	Formatted terminal output.
u	User exit.
x	Exits back to TCL or to the calling Proc.
+	Adds a decimal number to a parameter in the input buffer.
-	Subtracts a decimal number from a parameter in the input buffer.
()	Transfers control to another Proc.
[]	Subroutine call or call to another Proc.

Related concepts

[Structure of the PROC processor](#)

Related reference

[Proc command format](#)

Editor

D3 has two item editors: the Editor (a line editor) and the Update Processor (a full screen editor).

The Editor permits online interactive modification of any item, except binary items, in the database. The editor is used to create or modify FlashBASIC:

- Programs
- Data files
- File dictionaries
- Menus
- Macros
- Procs
- Assembly source code

The Editor includes the following features:

- Two variable length temporary buffers
- Absolute and relative current line positioning
- Line number prompting on input
- Merging of lines from the same or other items
- Character string location and replacement
- Conditional and unconditional line deletion
- Input/Output formatting
- Prestoring of commands

Entering the Editor

The Editor is entered by invoking the edit command at TCL.

Syntax

```
ed{it} { dict} file.name{ item.list} {( options)}
```

Parameters

Parameter	Description
<i>dict</i>	Specifies items in the dictionary section. If omitted, the specified items in the data section of the specified file are edited.
<i>file.name</i>	Name of the file.
<i>item.list</i>	Item-IDs separated by spaces. To specify all items in the file, enter either an asterisk or leave the item list blank. If multiple item-IDs are specified, the items are edited in the order specified. If a select list is in effect and the item list is omitted, the item-IDs are obtained from the select list.
<i>options</i>	The following options are available:
	a Turns on the assembly code formatting option.
	d Allows the file definition item to be edited.
	m Turns on the macro expansion flag.
	p Sends all system output to the line printer.
	s Turns on the suppress line numbers or suppress object code option (see the Editor command summary command).
	z Suppresses TOP and EOI messages.

Related reference

[Editor command summary](#)

[Editor error messages](#)

Editor command summary

The table below lists the Editor commands.

Command	Description
^	Toggle wildcard character
Enter	List next line
?	Display current line number
.command	Suppress display of command results

Command	Description
a	Search again
as	Assembler format ON/OFF
b	Bottom of item
c	Display column numbers
de{n}	Delete lines
de{n}'string' {p{-q}}	Delete lines with string
ex{k}	Exit item without filing it
fd	Delete item
fi {{file.name} item-ID}	File item
fs	File item and remain in the item
{g}n	Goto line
i	Input multiple lines
i data	Insert single line
l{n}	List lines
l{n}'string' {p{-q}}	Locate string
m	Macro expansion
me{n}/ {item-ID}/ {m}	Merge from current file
me{n} {file.name item-ID} {m}	Merge from specified file
n{n}	Move line pointer down
pn commands	Define prestore <i>n</i> command
p{n}	Invoke prestore
pd	Display prestore commands
r{n}	Replace lines
r{u}{n}'string1/string2' {p{-q}}	Replace strings
s?	Size of current item
s	Suppress line number or object code
tb <i>n,n,n,...</i>	Set tabs
t	Top of item
u{n}	Move line pointer up
x{f}	Undo last replace, delete, or insert

Command	Description
<code>z{p{-q}}</code>	Set zone for list command

Related concepts

[Entering the Editor](#)

Related reference

[Editor error messages](#)

Editor error messages

The table below presents a list of potential error messages returned by the editor.

Error Message	Description
CMND?	Illegal Editor command.
STRING?	Illegal specification, or missing string (for example, missing required string for merge). Also occurs as a result of an invalid numeric parameter specification, which causes a part of the numeric parameter to appear as if it were a string.
COL#?	Illegal characters follow the recognized end of the command, illegal format for a column specification, or non-numeric characters used for <code>start.col</code> and <code>end.col</code> in <code>locate</code> , <code>replace</code> , <code>delete</code> , or <code>merge</code> commands.
SEQN?	Out of sequence update. Item modification must be done in an ascending line number sequence until an F command is entered.
EOI <i>nn</i>	End of item reached at line <i>nn</i> .
TOP	Top of item (line 0) reached.
L <i>nn</i>	Indicates the current line pointer number, or after an X command, indicates update action on line <i>nn</i> was undone.
NOT ON FILE	Item specified in <code>merge</code> command is not on the file.
'item-ID' EXITED	Editor exited via EX command. The item is untouched.
'item-ID' DELETED	Item has been deleted from the file.
'item-ID' FILED	Item has been updated to the file.

Related concepts

[Entering the Editor](#)

Related reference

[Editor command summary](#)

Update Processor

D3 has two item editors: Update Processor (a full screen editor) and the the Editor (a line editor).

The Update Processor is a dictionary-driven, full-screen editor, and data processor. Used in conjunction with the Output processor it operates as a document processor. Update Processor can be used to view, create, and manipulate items in files. Update Processor facilitates editing of text, dictionaries, programs and data files, as everything in D3 is an item in a file. When used as a data processor, attribute-defining items in the file dictionary are employed to verify and translate data as they are entered and retrieved.

B-tree indexes permit cruising from item to item on an attribute-value sequence. The B-tree mechanism also allows entry to the index on a partial key. FlashBASIC subroutines can be called from dictionaries allowing absolute control of all data entry using the Update Processor and dictionaries. Without any attribute-defining items specified, Update Processor functions as a full screen editor. When evoked, Update Processor copies the specified item to a workspace and displays as much of the item on the screen as it can. In most applications, each input screen fits within a real screen. Update Processor places the cursor at the upper left corner of the item (top) on the first character of the first value of the first attribute and waits for input.

Once the item has been loaded, any character typed overwrites whatever is on the screen. The CTRL key is used to perform editing functions, but as long as the CTRL key is not pressed, all characters are entered as typed. An automatic word wrap feature moves whole words to the next line as they are typed when the end of the current line is overrun. An optionally activated spelling checker beeps and rejects input when a character is typed that does not lead to a predefined word.

Text is normally entered in the insert mode. CTRL+R is used to toggle between the insert and overtype modes.

While in the input mode	Pressing ENTER creates a new attribute or paragraph.
When in the overtype mode	Same key does not create a new attribute, but instead goes to the next attribute. The default mode is overtype.

Update Processor commands begin with a key combination by simultaneously pressing the CTRL key and one of the character keys. Some Update Processor commands require more command characters, while others require text input. The second Update Processor command can be entered as straight alphanumeric or with CTRL pressed. On some terminals a CTRL+S leaves the terminal locked. To unlock the terminal, press CTRL+Q.

Until the item is saved, changes made using Update Processor have no affect on the actual item in the file—only Update Processor workspace is changed. The item can be exited at any time without filing by pressing CTRL+XE or CTRL+XX.

Update Processor as text editor

In the absence of any attribute list, Update Processor functions as a text editor.

Syntax

```
[u{p} | update] file.reference{ item.list} {( options)}
```

Parameters

Parameter	Description
<i>file.reference</i>	The name of the file to update.
<i>item.list</i>	The item-ID to update.
<i>options</i>	The list of options.
	i Include item-ID in Update Processor workspace.
	s Toggle off spelling checker.
	l Look only; item cannot be filed or deleted.
	c Clear screen before bringing item into Update Processor workspace.

Related concepts

[Update Processor as a data processor](#)

Related reference

[Prestore commands](#)

[Exiting items](#)

[Overloaded commands for data files](#)

[Update processor command summary](#)

Cursor movement

The cursor is the underline or block that moves around the screen when characters are typed.

Cursors have various forms depending on the terminal:

- Highlighted
- Blinking
- Reverse video

In all cases, the cursor indicates where the next character typed displays on the screen.

Cursor movement commands are used to position the cursor anywhere in the item without altering the text. The cursor actually moves on the screen, but since the screen is only a window into the item, the cursor may be moved off the screen forcing a screen redisplay (scrolling).

Command	Description
CTRL+B	Move the cursor up one line maintaining the same column position.
CTRL+D	Move the cursor back to the beginning of the sentence.
CTRL+F	Move the cursor forward to the beginning of the next sentence.

Command	Description
CTRL+G	Move the cursor to the end of the current paragraph.
CTRL+I	Move the cursor forward to the next tab stop in the line. If the cursor is past the last tab, the command has no affect. Tabs may be set at TCL or by using the Update Processor set tab command CTRL+ZT.
CTRL+J	Move the cursor back (left) one character. If the cursor is at the beginning of the line, it moves to the last data position on the previous line.
CTRL+K	Move the cursor forward (right) one character. If the cursor is at the end of the screen or at the end of an attribute then CTRL+K moves it to the first position on the next line.
CTRL+M	(Also pressing ENTER) Move the cursor to the beginning of the next paragraph if in overtype mode. If in insert mode, it creates a new attribute (paragraph). If the cursor is on the last line of the item, it creates a new attribute (paragraph) whether in insert or overtype mode.
CTRL+N	Move the cursor down one line maintaining the same column position.
CTRL+T	Move the cursor to the top of the current item.
CTRL+U	Move the cursor forward (right) one word. If the cursor is at the end of a line or paragraph, it moves to the first word of the next line or paragraph.
CTRL+Y	Move the cursor back (left) one word. If the cursor is at the beginning of a line, it moves to the beginning of the last word in the previous line or paragraph.
CTRL+ZA	Redisplay the screen with the current line at the top of the screen.
CTRL+ZB	Redisplay the screen with the current line at the bottom of the screen.
CTRL+ZE	Move the cursor to the end of the current item.
CTRL+ZH	Move the cursor to the first line of the next screen, then move the display up twelve lines, approximately half of a screen.
CTRL+ZN	Move the cursor to the top of the next screen.
CTRL+ZP	Redisplay (repaint) the screen.
CTRL+ZQ	Move the cursor to the first line of the next screen, then move display up one quarter of a screen.
CTRL+ZY	Move the cursor to the top of the previous screen.
CTRL+Z <i>number</i>	Go to the specified line number and redisplay the screen. The line number may be before or after the current line number. After pressing CTRL+Z and the first character of the line number, the prompt go to line #: displays, along with the number typed. Finish entering the line number and press ENTER. The cursor moves to the specified line.

Related concepts

[Overtyping and insert modes](#)

[Inputting text using the spelling checker](#)

[Deleting text](#)

Related reference

[Searching and replacing text](#)

[Cutting and pasting text](#)

Overtyping and insert modes

When Update Processor is first entered it is in overtyping mode. This means that characters typed overwrite the characters on the screen. Insert mode is used to insert new text just in front of the text beginning at the cursor. When text is entered in the insert mode, the following text is opened and shown on the next line until insert mode is exited or a cursor movement command is typed.

The letter **O** displays at the top right corner of the screen when in overtyping mode; the letter **I** displays when in insert mode. As text is entered in either mode, Update Processor word wraps the text as necessary. That is, if a character string is longer than the remaining space on the current line, the entire string is moved to the next line. This means typing can be done continuously within a paragraph (attribute) without pressing the ENTER.

 **Note:** The terminal line width is determined by a value specified in the term command.

CTRL+R	Toggle between overtyping and insert modes. When insert mode is entered, the text following the cursor is moved to the next line and as many lines of new text as needed may be entered. To create a new attribute (paragraph), press CTRL+M or ENTER, while in insert mode. To return to overtyping mode, press CTRL+R again.
Paragraphs	To create a new paragraph in the middle of existing text, enter the insert mode and press ENTER. This terminates the current paragraph and prompts with the next attribute number. If the cursor is at the end of an item, CTRL+M creates a new paragraph whether in input mode or not.
CTRL+M	Same as ENTER. Always moves the cursor to the beginning of the next attribute. If in insert mode or at the end of an item, a CTRL+M creates a new paragraph. If Update Processor is being controlled by a dictionary, CTRL+M does not create a new attribute mark, rather the cursor goes to the next attribute.

Related concepts

[Cursor movement](#)

[Inputting text using the spelling checker](#)

[Deleting text](#)

Related reference

[Searching and replacing text](#)

[Cutting and pasting text](#)

Inputting text using the spelling checker

Update Processor uses a spelling checker to verify text as it is entered. The spelling checker looks at each character to see if those typed are a legitimate word or can become a legitimate word. Words used by the spelling checker are kept in the words file in the dm account. In order to activate the spelling checker, a Q-pointer must be present on the current account and an index must be generated on attribute 0 of the words file.

If an error is detected, the terminal beeps and does not display the last character typed. If an incorrect letter is typed, type the correct one and continue entering text. To view words adjacent to the misspelled word in the words file, use CTRL+U to get the next legitimate word or CTRL+Y to get the previous word. Otherwise, press one of the following:

Command	Description
CTRL+A	Override the spelling checker and accept the characters.
CTRL+A	Insert the word into the words file and continue.

Command	Description
CTRL+A	
CTRL+A CTRL+A CTRL+A	Turn off the spelling checker.
CTRL+ZS	Toggle the spelling checker on and off from any point in the item. If the spelling checker is on, CTRL+ZS toggles it off. If the spelling checker is off, CTRL+ZS toggles it on.

Related concepts

[Cursor movement](#)

[Overtyping and insert modes](#)

[Deleting text](#)

Related reference

[Searching and replacing text](#)

[Cutting and pasting text](#)

Deleting text

As text is deleted, the remaining characters in the paragraph are moved to the left. The cut and paste commands also delete text.

Related concepts

[Cursor movement](#)

[Overtyping and insert modes](#)

[Inputting text using the spelling checker](#)

Related reference

[Searching and replacing text](#)

[Cutting and pasting text](#)

Searching and replacing text

The search commands in Update Processor offer the capability to search for text, search for text and replace it, and search for text then determine whether or not to replace it.

Command	Description
CTRL+A	Search again for the string specified by the last search command (for example, repeat last search command). If a search string is not specified, the terminal beeps and displays a message.
CTRL+A <i>string</i>	Search for the first occurrence of the specified string. If a match is found, the cursor displays on the first character of the matched string; otherwise, the cursor remains at the current location. When CTRL+A is pressed, Update Processor prompts for the search string at the top of the screen. The search string is not case sensitive. To search for letters that are case specific, enclose the string in single or double quotation marks. If the string itself contains quotation marks, enclose those quotes in quotes.

Command	Description	
CTRL+A <i>string1</i> CTRL+R <i>string2</i> CTRL+N	Search for and replace every occurrence of the first string with the second string from the current position of the cursor to the end of the item. When CTRL+R is pressed, Update Processor prompts for the replacing string. After entering the new text, press CTRL+N to indicate all occurrences. The text is replaced as entered. However, if the replacement is at the beginning of a sentence, the initial letter is replaced as an uppercase letter. To specify an exact replacement of uppercase and lowercase letters, enclose the text in single or double quotation marks.	
CTRL+A <i>string1</i> CTRL+R <i>string2</i>	Search for the first occurrence of <i>string1</i> . If a match is found, the cursor displays on the first character of the matched string and Update Processor waits for one of the following commands:	
	CTRL+A	Do not replace <i>string1</i> ; search again for next occurrence.
	CTRL+N	Replace <i>string1</i> with <i>string2</i> ; replace all further matches.
	CTRL+R	Replace <i>string1</i> with <i>string2</i> ; search again for <i>string1</i> .
	CTRL+X	Stop search and return to edit mode.

Related concepts

[Cursor movement](#)

[Overtyping and insert modes](#)

[Inputting text using the spelling checker](#)

[Deleting text](#)

Related reference

[Cutting and pasting text](#)

Cutting and pasting text

Cut commands are used to move or copy data from one place in an item to another. Text may be cut and deleted, or cut leaving the original left in place. After the text is cut, it is placed in a cut buffer replacing the previous contents of the cut buffer. The cut text in the cut buffer may be pasted in the current document or it can be pasted in a specified item in a file. Text also may be pasted from a specified item and file.

Command	Description
CTRL+CC	Mark end of text to cut or copy from the beginning cut, for example, CTRL+CD or CTRL+CL to current cursor position, and put the text in the cut buffer. If a CTRL+CL is performed without a previous CTRL+CD or CTRL+CL, then the terminal beeps and the command is not accepted.
CTRL+CD	Mark the beginning location of text to be cut and deleted. Update Processor displays a backslash to mark the beginning location of the cut.
CTRL+CI	Copy the internal contents of the current value into the cut buffer.
CTRL+CL	Mark the beginning of text to cut and leave, that is do not remove the cut from the current location in the text and make a copy for the cut buffer. Update Processor displays a backslash to mark the beginning location of the cut.

Command	Description
CTRL+CO	Copy the entire contents of the current value into the cut buffer.
CTRL+CP	Paste contents of cut buffer at current cursor position. Once text has been put in the cut buffer it remains there for the duration of the current update session in the Update Processor workspace.
CTRL+CR <i>{{file.reference}}</i> <i>item-ID {n{-m}}</i>	Read contents of specified item and paste at current cursor location. The optional <i>n</i> parameter is an integer that specifies the first attribute number to copy; the optional <i>m</i> parameter is an integer that specifies the last attribute number to copy. If a range is not specified, all of the attributes from beginning to end of the item are copied.
CTRL+CW <i>{{file.reference}}</i> <i>item-ID</i>	Write the contents of the current item cut buffer to the item specified by <i>item-ID</i> .

Related concepts

[Cursor movement](#)

[Overtyping and insert modes](#)

[Inputting text using the spelling checker](#)

[Deleting text](#)

Related reference

[Searching and replacing text](#)

Prestore commands

Prestore commands are used to store and execute a sequence of Update Processor commands. The prestore commands can be used during the current update session or saved as an item in a file and recalled for use in subsequent update sessions. Prestore commands can contain both Update Processor commands and text. To specify an Update Processor command, enter the letter portion of the editing command without pressing CTRL. To specify text, enter the text enclosed within single or double quotation marks.

Prestores can be created in the prestore buffer or they can be created as an item in a file. Prestore items can exist in any file and must have a **p** in the first attribute. The following attribute values contain prestore commands and text:

Command	Description
<CTRL>P	Execute the current command in the prestore buffer. The default prestore command is <CTRL>ZN to advance a page.
<CTRL>ZL	Create or edit prestore command in prestore buffer. Most Update Processor commands are available to input and edit the prestore buffer.
<CTRL>ZR <i>{{file.reference}}</i> <i>item.id</i>	Load specified item into the prestore buffer but do not execute it. The first attribute must be the character 'p'.
<CTRL>ZR <i>{{file.reference}}</i> <i>item.id</i> <CTRL>P	Load the specified item into the prestore buffer and execute it. The first attribute in the item must be a 'p'. The default file is the master dictionary.
<CTRL>ZW <i>{{file.reference}}</i> <i>item.id</i>	Write the prestore buffer into the specified item. The character 'p' is written as the first attribute of the item. The default file is the master dictionary.
<CTRL>V	Insert a value mark. The value mark causes any characters that follows it in that attribute to be moved to the next line. A value mark has an ASCII decimal value of 253 and is not capable of being displayed in the Update Processor. (See the

Command	Description
	following section on using UP as a data processor.) In an attribute with a v1 processing code and an index you may use <CTRL>V to create a new item with the same key.
<CTRL>ZC	Displays column positions across top line of the screen (a ruler).
<CTRL>ZT	Sets tab stops. The Update Processor prompts to enter tab stops separated by blanks.

Related concepts

[Update Processor as a data processor](#)

Related reference

[Update Processor as text editor](#)

[Exiting items](#)

[Overloaded commands for data files](#)

[Update processor command summary](#)

Exiting items

The CTRL+X commands offer different methods of exiting an item. CTRL+X indicates exiting this item and performing one of a series of options. For example, an item can be exited and the information filed (saved), or the item can be exited without the information being filed.

If an item is exited without being filed, the changes made during the editing session are not saved. To save the changes, the item must be filed. Several options that exit an item do not file it. If changes are made to an item during the current editing session and an option is selected that exits the item without filing it, the system double checks with the user to make sure that the changed information is not to be saved. All Update Processor exiting commands are a minimum of two characters. When CTRL+X is pressed, the current size of the item, file name and item-ID are displayed on the top line of the terminal. Update Processor waits for an additional command.

Command	Description
CTRL+XB	Exit the item and go back to the previous item in the item list or active select list. If there is no previous item, the terminal beeps and remains in the current item.
CTRL+XC	File the current item then compile and catalog it. This option is used with FlashBASIC programs.
CTRL+XE	Exit the item without filing it.
CTRL+XF	File the item, then exit it.
CTRL+XI{ <i>{file.reference} {item-ID}</i> }	Rename and file the new item. This command prompts for a standard file item specification where the item is copied. After entering the file item specification information, the system asks if the old item from which the new item was created is to be deleted. Enter y to remove the old item, otherwise the original item is unchanged.
CTRL+XK	Exit the item and return to the calling process. If editing with a select list, the select list is terminated.
CTRL+XN	Exit current item and create a new null item. If a name is not entered when the new item is filed, it can be named using the xi option. If there is no ID processing code,

Command	Description
	Update Processor uses the current date concatenated with a system wide sequence number as the item-ID.
CTRL+XO	Exit and delete the item. If any changes were made to the item, the system prompts to verify that the item is to be deleted.
CTRL+XP	File the item and print it using the Output Processor.
CTRL+XR	File the item, compile it and run it if compilation is successful. This option is used with FlashBASIC programs, macros, and menus.
CTRL+XS	File (save) the item and remain in Update Processor with the current item.

Related concepts

[Update Processor as a data processor](#)

Related reference

[Update Processor as text editor](#)

[Prestore commands](#)

[Overloaded commands for data files](#)

[Update processor command summary](#)

Update Processor as a data processor

When Update Processor is called with an attribute list it functions as a data processor. Each attribute name in the Update Processor statement becomes a data entry location on the screen identified by the named attribute. Attribute names are listed along the left of the screen in half intensity. Dependent attribute names are displayed horizontally across the screen with the controlling attribute. Each attribute-defining item creates a cell on the screen through which data can be accessed and modified under the control of dictionary processing codes. Since a FlashBASIC program can be called using a processing code, complete application flexibility is provided.

The cursor can be moved, using the previously defined cursor movement commands, from attribute name to attribute name. Some of the commands function in a different manner. Data entered or changed next to an attribute name is filed in that attribute when the edited item is saved. Stored data is subject to output processing codes defined in attribute-defining items in the file dictionary prior to display. Data entered using Update Processor is subject to input processing codes defined prior to storage on disk. By invoking a single Update Processor command, an entire data access and update screen may be defined. When that command is saved as a macro, it may be recalled any time for file access.

Syntax

```
[u|up|update] file.reference item-ID attribute.list{( options)}
```

Parameters

Parameter	Description
<i>file.reference</i>	Name of the file to update.
<i>item.list</i>	Item-ID to update.

Parameter	Description	
<i>attribute.list</i>	List of attributes (by their attribute names) to update.	
<i>options</i>	i	Include item-ID in Update Processor workspace.
	s	Toggle off spelling checker.
	l	Look only; item cannot be filed or deleted.
	c	Clear screen before bringing item into Update Processor workspace.

Update Processor may be used to scan data using file indexes. When attributes are indexed using the **create-index** command, Update Processor may be used to scan or cruise the index. The key to cruising is understanding the Balance Tree indexing feature of D3. B-tree is a technique used to keep an ordered list, such as names, and the ability to search through the list very quickly with a minimal number of disk Inputs and Outputs. It is a method for balancing. Any attribute in a file can contain an index. Indexes are defined using a-correlatives. Once the B-tree has been defined, the system automatically maintains the index.

B-trees return the ability to access data in ways not described before. The words describing the new capabilities are cruising using the file index, cruising using the file related index, zooming and double clutching.

Feature	Description
Cruising	Using the file index (local file) is the ability to move from item to item within the current file. You can move forward through the items on any indexed attribute (CTRL+F) or backward (CTRL+D). For example, in an entity file you could look for all items whose name (indexed attribute) begins with ABC. Cruising using the related file index (remote file) provides the ability to view valid data from another file. For example, a sales order file could have an attribute called name that points to the entity file. From the orders file, one could view the valid names in the entity file and select the one desired.
Double clutching	Means the combined abilities of searching the remote file for valid values and searching the local file for specific items. For example, in an order file, the name attribute could have an index pointing to the entity file allowing for searching of valid people. A local index on the name attribute would allow for searching for all of the orders already on file for that person.
Zooming	Refers to moving through a defined door to another file. Once in the other file, the user has the option to cruise from item to item in the new file, return to the original file or zoom through a door into yet another file. CTRL+G is the command to zoom to another file.

When the cursor is residing in a cell belonging to an indexed attribute, the CTRL+D and CTRL+F commands can be used to display the data associated with the item containing an adjacent value in the indexed attribute. The CTRL+F command causes the data associated with the next value in the index to be displayed. The CTRL+D command causes the data associated with the previous value in the index to be displayed.

For example, if an item containing customer name and order-number indexed attributes is being displayed on the screen and the cursor is at the order-number prompt, the CTRL+F command displays the data associated with the next order number in the order-number index. If the cursor is residing adjacent to customer name, CTRL+F causes the data associated with the next customer name in the index to be displayed; customer name is the key. All of the data in the new item associated with the attribute names displayed on the screen displays.

When the input processing code attribute in an attribute-defining item specifies an index pointing to another file, that file can be accessed (zoomed to) by using the CTRL+G command. This command pushes the current process one level and accesses the item-ID in the pointed to file that matches the current value in the cell where the cursor resides. Attributes to be displayed with that item-ID must be specified in attribute 15 of the original attribute-defining item. Several logical views may be defined as macros in attribute 15 of the D-pointer. A BASIC program may be called to

specify the actual logical view to use for the current data item. It does this by assigning the logical view number into the pseudo variable AQL (18). If no attributes are specified, the entire item displays. After viewing the new item, the original item can be returned to the screen by a file exit command such as CTRL+XE.

When the input processing code attribute in an attribute-defining item specifies a second index as well as one pointing to another file, the commands CTRL+U and CTRL+Y allow the user to cruise on indexes in the pointed to file. The attribute pointed to in the second file must have been previously indexed by the **create-index** command. The command CTRL+U shows the next attribute value in the pointed to file. Only this value is updated; it is shown in the cell where the cursor is residing. What displays is subject to the processing codes specified in the dictionary of the first file. The command CTRL+Y shows the preceding item-ID in the pointed to file.

In order for Update Processor to function in this manner, the attribute-defining items must contain the proper index processing codes. These are specified in the input processing code attribute of the attribute-defining item in the file dictionary.

Related reference

[Update Processor as text editor](#)

[Prestore commands](#)

[Exiting items](#)

[Overloaded commands for data files](#)

[Update processor command summary](#)

Overloaded commands for data files

Most Update Processor commands work the same whether or not dictionary attributes are in control. Certain commands function differently when certain conditions are met. These commands provide the functionality for browsing on indexes. The cruising commands can be used in a double clutching fashion by specifying the index for the current file I and a second index for another file. CTRL+U and CTRL+Y cruise on legal instances in the pointed to file staying in the same item. CTRL+F and CTRL+D cruise on other items in the same file using the index for the attribute as a guide.

Command	Description
CTRL+D	On an attribute with an index, the sentence backward command goes to the previous sequential index and gets the corresponding item into the Update Processor workspace. If the current item has been changed, the user is prompted to exit the current item or to cancel the request and go to another item.
CTRL+F	On an indexed attribute, the sentence forward command goes to the next sequential index and gets the corresponding item into the Update Processor workspace. If the current item has been changed, the user is prompted to either exit the item or to cancel the request and go to another item.
CTRL+G	When in an attribute which indexes to another file or has a translate code in the input processing code, this command pushes the current process one level and calls Update Processor again with the item pointed to in the new file as the argument. If the macro attribute (attribute 15) in the primary file attribute definition item is used, this list is appended to the update statement. This is known as zooming.
CTRL+U	If the attribute indexes to another file, the word forward command gets the next sequential item from the pointed to file as the value for this attribute.
CTRL+Y	If the attribute indexes to another file, the word backward command gets the previous sequential item from the pointed to file as the value for this attribute.

Related concepts

[Update Processor as a data processor](#)

Related reference

[Update Processor as text editor](#)

[Prestore commands](#)

[Exiting items](#)

[Update processor command summary](#)

Update processor command summary

The Update processor commands are summarized below.

Command	Description
CTRL+A	Search or search again.
	(Search/Replace) Do not replace text, search again for <i>string1</i> .
	Override spelling checker.
CTRL+A <i>string</i>	Search for specified <i>string</i> .
CTRL+A <i>string1</i> CTRL+R <i>string2</i>	Search for <i>string1</i> , if found, replace with <i>string2</i> , wait for further input.
CTRL+A <i>string1</i> CTRL+R <i>string2</i> CTRL+n	Replace every occurrence of <i>string1</i> with <i>string2</i> .
CTRL+A CTRL+A	Insert word into words file.
CTRL+A CTRL+A CTRL+A	Turn off spelling checker.
CTRL+B	Move cursor up one line.
CTRL+CC	Cut text marked by CTRL+CD or CTRL+CL and place in cut buffer.
CTRL+CD	Mark beginning of text to copy; delete original text.
CTRL+CI	Copy the internal contents of the current value into the cut buffer.
CTRL+CL	Mark beginning of text to cut; leave original text in place.
CTRL+CO	Copy the entire contents of the current value into the cut buffer.
CTRL+CP	Paste contents of cut buffer at current cursor position.
CTRL+CR{(file.ref)item-ID{n{-m}}}	Paste specified item.
CTRL+CW{(file.ref)item-ID}	Write cut buffer into specified item.
CTRL+D	Move cursor to first character of previous sentence or retrieve previous sequential index.
CTRL+E	Delete text from cursor to end of sentence.
CTRL+F	Move cursor to first character of next sentence or retrieve next sequential index.
CTRL+G	Move cursor to end of attribute (paragraph).
CTRL+G CTRL+G	Zooms to related item in related file.

Command	Description
CTRL+H	Backspace and replace character with blank. If character is at end of attribute, delete character.
CTRL+I	Move cursor to next tab position.
CTRL+J	Move cursor left one character.
CTRL+K	Move cursor right one character.
CTRL+L	Delete character under cursor.
CTRL+M	In overtype mode, same as Enter, and move cursor to beginning of next attribute. In insert mode, create an attribute and move the cursor to it.
CTRL+N	Move cursor down one line.
	(Search/Replace) Replace all matches with <i>string2</i> .
CTRL+O	Delete from cursor to end of word.
CTRL+P	Execute prestored command. Default prestore is CTRL+ZN.
CTRL+R	Toggle between overtype and insert mode.
	(Search/Replace) Replace current match with <i>string2</i> then search again for <i>string1</i> .
CTRL+Y	Move cursor to top of item.
CTRL+U	Move cursor right to next word or retrieve next sequential index in remote file.
CTRL+V	Insert value mark.
CTRL+W	Insert one space.
CTRL+X	Abort search return to edit mode.
CTRL+XB	Exit item, then return to previous item.
CTRL+XC	File item, then compile and catalog it.
CTRL+XE	Exit item without filing.
CTRL+XF	File item.
CTRL+XI{{file.ref}}item-ID	Rename item and file.
CTRL+XK	Exit item, then return to calling process.
CTRL+XN	Exit item, then create a new item.
CTRL+XO	Delete item.
CTRL+XP	File item and print using Output processor.
CTRL+XR	File item, then compile and run.
CTRL+XS	File item (save) and remain in item.
CTRL+XX	Exit item without filing.
CTRL+Y	Move cursor left one word or retrieve previous sequential index in remote file.

Command	Description
CTRL+ZA	Redisplay item with current line at top.
CTRL+ZB	Redisplay item with current line at bottom.
CTRL+ZC	Display terminal columnar positions.
CTRL+ZD	Delete from cursor to end of item.
CTRL+ZE	Move cursor to end of item.
CTRL+ZG	Move cursor to end of previous paragraph.
CTRL+ZH	Move cursor to first line of next screen, display half of next screen.
CTRL+ZL	Load prestore buffer.
CTRL+ZN	Move cursor to next page.
CTRL+ZO	Delete from cursor to end of attribute (paragraph).
CTRL+ZP	Redisplay screen.
CTRL+ZQ	Move cursor to first line of next screen, display quarter of next screen.
CTRL+ZR{{file.ref}} item-ID	Load <i>item-ID</i> into the prestore buffer; do not execute.
CTRL+ZR{{file.ref}} item-ID CTRL+P	Load <i>item-ID</i> into the prestore buffer; execute it.
CTRL+ZS	Toggle spelling checker off and on.
CTRL+ZT <i>n1</i> { <i>n2</i> { <i>n3</i> {...}}}	Set tabs at indicated tab positions.
CTRL+ZW{{file.ref}} item-ID	Write contents of the prestore buffer to the specified item.
CTRL+ZY	Move cursor back one screen.
CTRL+Z <i>attrib.number</i>	Move cursor to the <i>attrib.number</i> .
CTRL+ZZ	Undo last delete.

Related concepts

[Update Processor as a data processor](#)

Related reference

[Update Processor as text editor](#)

[Prestore commands](#)

[Exiting items](#)

[Overloaded commands for data files](#)